

An Almost-Linear Approximation Scheme for Metric-Weighted Edit Distance

Jihan Wang

October 7, 2026

Abstract

We give an almost-linear randomized approximation scheme for edit distance under arbitrary metric weights. For every fixed rational $\varepsilon > 0$, the algorithm returns an estimate in $[D, (1 + \varepsilon)D]$ with probability at least $5/6$, where D is the weighted edit distance. Its worst-case expected running time is $N^{1+O(\log \log \log N / \log \log N)} = N^{1+o(1)}$ for total string length N , with no dependence on the ratio between positive costs. The same running time gives factor $1 + O((\log \log N)^{-3})$ when the accuracy is chosen from the input length. The model permits constant-time exact metric queries and exact real arithmetic. Finite binary computation and random sampling are charged. We adapt the unit-cost refinement of OpenAI to metric weights. Gap-mass coordinates control the cost of moving alignment endpoints. A coarse estimator answers a substring query with source X in $(1 + |X|)N^{o(1)}$ expected time, without a factor for the target fragment's length. Approximate singleton comparisons contribute error once, at the leaves. A single capped and rounded integer metric reduces arbitrary real costs to the bounded-cost setting.

1 Introduction

Edit distance is the minimum cost of changing one string into another by insertions, deletions, and substitutions. It is a basic measure of string similarity, with a long history in coding and sequence alignment [1, 2, 3, 4]. In the unit-cost setting, each nontrivial edit costs one. Weighted edit distance allows the cost to depend on the edited symbols. We study the metric setting: a metric w is given on the alphabet together with a gap symbol $\perp$. Deleting a costs $w(a, \perp)$, inserting b costs $w(b, \perp)$, and substituting a by b costs $w(a, b)$. The triangle inequality makes the edit-script definition equivalent to the minimum cost of a monotone alignment path.

For strings x, y , let $N = |x| + |y|$ and $D = \text{WED}_w(x, y)$. Our goal is a $(1 + \varepsilon)$ approximation in almost-linear time. The weights may have arbitrary positive ratios. This setting differs substantially from unit costs. Moving a target endpoint by one position can cross an arbitrarily expensive symbol. A long target fragment can also consist mostly of symbols whose gap costs are very small. An efficient algorithm must measure alignment errors by their cost, while charging its computation by the number of input symbols.

Recent approximation schemes for unit-cost edit distance make this question particularly timely. Mao and Rubinstein [5] obtained a $(1 + \varepsilon)$ approximation in quasi-strongly subquadratic time. OpenAI [6] subsequently gave an almost-linear randomized approximation scheme. For arbitrary metric weights, the recent algorithms of Mader, Tavasoli, and Wang [7] and Das, Kipouridis, and Kociumaka [8] achieve factor $3 + \varepsilon$ in strongly subquadratic time. We extend the unit-cost refinement to arbitrary metric weights and obtain a $(1 + \varepsilon)$ approximation in almost-linear expected time.

The metric assumption is essential for general approximation with oracle access to costs. For growing alphabets, Gorbachev [9] proved an unconditional $\Omega(N^2)$ expected oracle-query lower bound

for approximating nonmetric minimum-cost alignment within any factor depending only on N . The obstruction persists under symmetry alone or the triangle inequality alone. Particular nonmetric classes can still admit fast algorithms, including symmetric costs over a binary alphabet [9].

1.1 Our result

The algorithm uses a metric oracle and exact real arithmetic. Metric queries and real addition, subtraction, multiplication, division, and comparison take constant time. We charge the finite binary computation and random sampling used by the algorithm. The recent arbitrary-metric algorithms also assume constant-time metric queries and exact cost arithmetic [7, 8]. For a string Z , write $M_w(Z) = \sum_{a \text{ in } Z} w(a, \perp)$, where the sum counts occurrences.

Theorem 1.1. *There is one randomized algorithm in the metric-oracle model of Section 3 that takes strings x, y and a rational accuracy $\varepsilon > 0$ as input. It returns an estimate $\hat{D}$ satisfying*

$$\mathbb{P}\{D \leq \hat{D} \leq (1 + \varepsilon)D\} \geq 5/6.$$

For every fixed accuracy, its worst-case expected running time is

$$N^{1+O(\log \log \log N / \log \log N)} = N^{1+o(1)},$$

uniformly over the input strings and metric weights. It uses polynomial workspace and terminates on every sequence of random bits. It returns zero exactly when $x = y$, and its output always belongs to $[0, M_w(x) + M_w(y)]$.

The running time has no dependence on the ratio between positive edit costs. It includes executions on which an intermediate estimate fails. The running-time bound is asymptotic. The large-input analysis requires $\chi = \log_2 H \geq 2^{1000}$, where $H = \Theta(\log_2 N)$. Thus the cutoff is roughly $\log_2 \log_2 N \geq 2^{1000}$. Additional cutoff conditions depend on the accuracy. Below the cutoffs, the algorithm uses exact dynamic programming. Its finite accuracy encoding is specified in Section 3.

The accuracy can also be chosen from the input length.

Corollary 1.2. *There is a randomized algorithm that, for strings of total length $N \geq 4$, returns an estimate $\hat{D}$ satisfying*

$$\mathbb{P}\left\{D \leq \hat{D} \leq \left(1 + \frac{30}{(\log_2 \log_2 N)^3}\right) D\right\} \geq 5/6.$$

Its worst-case expected running time is

$$N^{1+O(\log \log \log N / \log \log N)},$$

uniformly over the input strings and metric weights. The constant in the running-time bound is absolute.

We construct a coarse table for polynomially bounded integer metric costs. After shared preprocessing, a query with source X has expected evaluation cost $(1 + |X|)N^{o(1)}$, uniformly in the target fragment. The same bound holds conditional on the history before the entry's first request. Gap-mass geometry and heavy-symbol methods from the metric-weighted literature [7, 8] provide the starting tools. Nested nets of actual target boundaries control endpoint rounding. The weighted connection inequality bounds the parent distance by the sum of the child distances and their connection cost.

The refinement uses approximate singleton estimates that are 3-Lipschitz in endpoint distance. Their total error is charged once at the comparison-tree frontier. A single capped and rounded integer metric transfers the construction to arbitrary real costs and keeps the work independent of positive cost ratios. These constructions implement the interfaces needed by OpenAI’s unit-cost refinement [6].¹ That refinement improves a coarse table through a hierarchy of local comparisons. We adapt its curvature, monotone-loss, and first-crossing arguments to the weighted setting and give proofs of the statements used here. The weighted theorem does not use the unit-cost theorem as a black box.

We retain the parameter forms of the unit-cost refinement [6], with $H = \Theta(\log N)$ and $\chi = \log_2 H$. The choices $p_* = \chi$ and $\ell = \Theta(\chi^3)$ give the explicit subpolynomial overhead in Theorem 1.1. The cited unit-cost theorem states $N^{1+o(1)}$ without this rate. The same explicit expected-time bound also applies to unit-cost edit distance on explicitly stored strings with $O(\log N)$ -bit integer symbols on the word RAM with $O(\log N)$ -bit words (Remark 9.6).

The weighted scheme improves both approximation and asymptotic running time for arbitrary metric costs. For equal-length strings, Kuszmaul [10] gave an $O(N^\delta)$ approximation in $\tilde{O}(N^{2-\delta})$ time for each fixed $0 < \delta < 1$. Mader, Tavasoli, and Wang [7] obtained factor $3 + \varepsilon$ in $\tilde{O}(N^{7/4}/\varepsilon^8)$ time. Das, Kipouridis, and Kociumaka [8] improved this to $\tilde{O}(N^{8/5}/\varepsilon^{16/5})$. Gorbachev [9] subsequently achieved factor 3 in $\tilde{O}(N^{40/21})$ time.

1.2 Related work

Exact alignment and small distance. The classical dynamic program computes weighted edit distance in quadratic time [2, 3, 4]. Linear-space methods show how to recover alignments without storing the whole dynamic-programming table [11, 12]. Masek and Paterson [13] obtained a logarithmic improvement for a fixed alphabet and commensurate edit costs. Crochemore, Landau, and Ziv-Ukelson [14] gave a subquadratic alignment algorithm for unrestricted scoring matrices over a constant-size alphabet. For unit costs, small-distance algorithms exploit the fact that a low-cost alignment stays close to the diagonal. This principle underlies Ukkonen’s algorithm [15], the Landau–Vishkin algorithms [16, 17], and incremental string comparison [18]. Myers’s difference algorithm [19] gives another classical distance-sensitive approach for insertion and deletion distance. Related distance-sensitive methods have also been used for DNA alignment under specific scoring rules [20]. These algorithms supply exact answers when the relevant distance parameter is small. Their running times do not give an almost-linear bound on unrestricted inputs.

Backurs and Indyk [21] showed that exact unit-cost edit distance has no strongly subquadratic algorithm under the Strong Exponential Time Hypothesis. Bringmann and Künnemann [22] established related conditional lower bounds for string similarity and dynamic time warping. Abboud, Hansen, Vassilevska Williams, and Williams [23] connected sufficiently large logarithmic improvements for exact edit distance and LCS with new circuit lower bounds. These results motivate approximation. Exact-distance hardness does not itself exclude a fixed-factor or a $(1 + \varepsilon)$ approximation.

Approximation for unit costs. The early subquadratic and near-linear algorithms traded approximation quality for running time [24, 25]. Metric embeddings provided another route:

¹We cite the September 24, 2026 manuscript at the repository revision listed in the bibliography. The catalogue describes a [Lean formalization](#) whose stated scope covers unit-cost edit distance on explicitly stored integer strings with polynomially bounded symbols. For fixed rational $\varepsilon \in (0, 1)$, that scope gives success probability at least $2/3$ and expected work at most $(N + 2)^{1+\eta}$ for each fixed $\eta > 0$ and sufficiently large N . This is the qualitative $N^{1+o(1)}$ bound. The scope does not cover our explicit rate or the weighted construction in the exact-real oracle model.

Ostrovsky and Rabani [26] embedded edit distance into ℓ_1 with subpolynomial distortion. Andoni and Onak [27] obtained a near-linear algorithm with a comparable approximation factor. For every fixed $\delta > 0$, Andoni, Krauthgamer, and Onak [28] achieved a polylogarithmic factor in $N^{1+\delta}$ time. Their source decomposition and precision sampling are important predecessors of the coarse estimator used here.

Chakraborty, Das, Goldenberg, Koucký, and Saks [29] gave the first classical constant-factor approximation in strongly subquadratic time. Goldenberg, Rubinstein, and Saha [30] developed stronger tradeoffs, including a $(3 + \varepsilon)$ approximation. Fox, Mader, Quanrud, Tavasoli, and Wang [31] gave a distance-sensitive $(3 + \varepsilon)$ approximation in $\tilde{O}_\varepsilon(N^{6/5}k^{2/5})$ time, where k is the unit-cost edit distance. Brakensiek and Rubinstein [32] and Koucký and Saks [33] obtained near-linear constant-factor approximations for sufficiently distant pairs. Andoni and Nosatzki [34] gave a constant-factor approximation on all input pairs in $N^{1+\delta}$ time for every fixed $\delta > 0$. The factor depends on δ . Approximation schemes require more than these constant-factor guarantees. The recent results of Mao and Rubinstein [5] and OpenAI [6] reach factor $1 + \varepsilon$ for unit costs. The latter refinement is the direct starting point of this paper.

Weighted and bounded edit distance. Weighted edit distance also admits exact algorithms parameterized by the distance or the weight range. Here the weights are normalized so that every nontrivial edit costs at least one, and k is a bound on the distance. Das, Gilbert, Hajiaghayi, Kociumaka, and Saha [35] gave an $O(N + k^5)$ algorithm. Cassis, Kociumaka, and Wellnitz [36] obtained $\tilde{O}(N + \sqrt{N}k^3)$ time. Gorbachev and Kociumaka [37] obtained improved algorithms for small integer weights. They also gave a $(1 + \varepsilon)$ approximation for normalized real weights in $O(N + (k/\varepsilon)^{5/2} \log^3 N)$ time. This distance-sensitive bound differs from the length-only bound sought here.

Dynamic versions reuse work across updates [38, 39]. Weighted pattern matching asks where a pattern approximately occurs inside a longer text [40]. Structured geometric costs permit additional exact speedups, as shown by Gold and Sharir [41] for geometric edit distance and dynamic time warping. Our metric is given by an oracle and need not have such a geometric representation.

For arbitrary metrics, Kuszmaul’s simplification deletes inexpensive symbols at a sampled cost scale [10]. The recent factor- $3 + \varepsilon$ algorithms [7, 8] combine this idea with weighted local comparisons and cost-based decompositions. Our normalization also uses this simplification. The main weighted refinement keeps the original input occurrences and measures endpoint movement by their gap mass in the normalized metric. This is needed when a large gap cost occurs beside a very cheap substitution.

Queries, sketches, and related similarity problems. Sublinear algorithms distinguish promised distance gaps using random access to the input strings [42, 43, 44, 45]. Recent adaptive algorithms narrow the gap to $(k, k^{1+o(1)})$ [46, 47], whereas non-adaptive queries have a separate optimal tradeoff [48]. Sketching algorithms encode each string separately and recover the exact distance, and sometimes an optimal edit script, when the distance is at most a prescribed threshold [49, 50, 51, 52]. Streaming algorithms restrict input access to forward scans. Randomized embeddings and low-distance algorithms also work with this access restriction [53, 49, 50]. Independent preprocessing can reduce the cost of repeated comparisons [30]. Our algorithm receives both strings explicitly and accesses metric costs through an oracle. Related questions arise for LCS [22, 5] and dynamic time warping [22, 10]. Reductions between edit-distance variants require care with alphabets and additive baselines [54].

2 Technical overview

The proof has one central task: make the unit-cost refinement work when alignment cost and string length describe different scales. The refinement needs geometric control of candidate alignments and fast local distance estimates. It then combines those estimates through a source hierarchy. With arbitrary weights, a position-based geometric estimate can be wrong by an unbounded factor, and a local query can take time proportional to a much longer target fragment. We explain how to restore these two properties and how they support the refinement. The exact parameter choices are deferred to the formal construction.

2.1 One bounded metric for the whole proof

The numerical range must first be separated from the number of input symbols. Searching successive cost scales between the smallest and largest positive weights would make the running time depend on their ratio. Instead, we find one interval containing the distance, with a width controlled by N , and normalize the entire metric once.

Let $g(a) = w(a, \perp)$ be the gap cost of a symbol. Deleting all occurrences with gap cost at most a threshold gives simplified strings. Kuszmaul’s simplification [10], as used by Das, Kipouridis, and Kociumaka [8], has two useful features. Removing cheap symbols costs at most the number of removed occurrences times the threshold. For a threshold sampled in a constant-width interval of scales, the expected distance of the surviving strings is controlled by D . If all surviving gap costs exceed a tested scale, any alignment below that scale has no gap operations and can be tested by pairing equal positions.

These tests search a finite set of input gap costs and their halves. The resulting bracket $[L_{\text{br}}, U_{\text{br}}]$ satisfies

$$0 < L_{\text{br}} \leq U_{\text{br}}, \quad U_{\text{br}}/L_{\text{br}} \leq 60N, \quad D \leq U_{\text{br}}.$$

The upper endpoint is valid on every outcome. With probability at least 99/100, the lower endpoint also satisfies $L_{\text{br}} \leq D$. The upper endpoint follows from a feasible alignment of the original strings. It remains valid when a randomized scale test misses a close pair. This allows every later computation to use a bounded metric.

Set $\eta_{\text{norm}} = \varepsilon L_{\text{br}}/(3N)$ and define

$$\bar{w}(a, b) = \left\lceil \frac{\min\{w(a, b), 2U_{\text{br}}\}}{\eta_{\text{norm}}} \right\rceil.$$

Both capping and upward rounding preserve the triangle inequality. Capping at $2U_{\text{br}}$ preserves the distance exactly. An optimal original alignment uses no edge above $D \leq U_{\text{br}}$. Any alignment affected by the cap has an edge of capped cost $2U_{\text{br}} > D$, so it cannot improve the optimum. An alignment has at most N edges, and rounding adds at most $N\eta_{\text{norm}}$. Consequently,

$$D \leq \eta_{\text{norm}} \text{WED}_{\bar{w}}(x, y) \leq D + \varepsilon L_{\text{br}}/3.$$

On the good bracket event, the final term is at most $(1 + \varepsilon/3)D$.

Every positive normalized cost is bounded by $\lceil 360N^2/\varepsilon \rceil$. For each fixed accuracy and sufficiently large N , this is at most N^3 , and the total gap mass is at most N^4 . We evaluate a normalized query by binary search over bounded integers, without taking the floor of an arbitrary real cost. The coarse estimator and every refinement pass use this same normalized metric. Their geometry and their distance estimates refer to the same edit costs.

2.2 Cost coordinates and fast singleton comparisons

For the rest of the overview, w denotes the bounded integer metric, and D denotes the distance in that metric. With $g(a) = w(a, \perp)$, define its prefix gap masses by

$$M_x(i) = \sum_{k \leq i} g(x_k), \quad M_y(j) = \sum_{k \leq j} g(y_k), \quad R = M_x(|x|) + M_y(|y|).$$

An alignment vertex (i, j) has shift $M_y(j) - M_x(i)$. The reverse triangle inequality gives $|g(x_i) - g(y_j)| \leq w(x_i, y_j)$. Thus a diagonal step changes the shift by at most its substitution cost. A gap step changes it by exactly that step's cost. The total variation of shift along any alignment segment is bounded by the segment's cost [8]. This is the weighted replacement for the position-based shift used in the unit-cost refinement.

String positions are numbered from one, and boundaries from zero. A state compares a source interval $I = (l, u]$ with a target endpoint pair $q = (q^-, q^+)$, where q^- and q^+ are boundary indices. Write $x[I] = x_{l+1} \cdots x_u$, $y[q] = y_{q^-+1} \cdots y_{q^+}$, and $c(I, q) = \text{WED}_w(x[I], y[q])$. The endpoint distance between two such pairs is

$$d(q, r) = |M_y(q^-) - M_y(r^-)| + |M_y(q^+) - M_y(r^+)|.$$

Adding or deleting the target pieces between corresponding endpoints shows that $c(I, \cdot)$ is 1-Lipschitz in d . We also need to combine child comparisons whose target intervals do not meet. Their connection cost is the gap mass crossed between successive endpoints. A connection inequality bounds the parent distance by the child distances plus this charge. It applies even when the child intervals overlap. Each added source-deletion cost is assigned to a distinct backward target crossing. The added cost is bounded by the connection charge.

Rounding must respect this geometry. A grid of target indices can jump across a very expensive symbol. We instead use nets of actual target boundaries, nested across dyadic mass scales. Nearby net points are separated in gap mass, and each boundary rounds downward with a controlled mass displacement. A large gap between consecutive prefix masses contributes a boundary to the net. We never split a target symbol into fractional positions. The same separation gives short local candidate lists, while nesting lets different stages round to compatible boundaries.

The second local difficulty is comparison against a singleton source. For a source symbol a and a target fragment V , the exact cost is

$$g(a) + M_w(V) - \max(\{0\} \cup \{g(a) + g(b) - w(a, b) : b \text{ occurs in } V\}).$$

The maximum is the saving obtained by pairing a with one target occurrence rather than deleting and inserting both symbols. For unit costs, an occurrence query finds this saving. For arbitrary metrics, it may require many different substitution queries. Scanning V at every source leaf would lose the desired running time.

The heavy-symbol reduction [8] avoids that scan. Keep the occurrences of V with the largest gap costs and query their substitution costs. If a potentially better occurrence is omitted, its possible saving is bounded by twice its gap cost. The retained occurrences then force enough remaining insertion cost to absorb the missed saving. For a chosen leaf accuracy $\varepsilon_{\text{leaf}} \in (0, 1]$, the resulting estimate Q satisfies

$$c \leq Q \leq (1 + \varepsilon_{\text{leaf}})c, \quad |Q(a, V) - Q(a, V')| \leq 3d(V, V').$$

It uses only $O(1/\varepsilon_{\text{leaf}})$ substitution queries, after range-reporting preprocessing. The Lipschitz bound is as important as the approximation bound: later comparisons move target endpoints. We retain these actual approximate leaf values throughout the proof.

2.3 A coarse table charged by source length

Refinement starts from a coarse estimate at every state it may request. Its requests are adaptive, so an estimate only for the full strings would not suffice. We need one random table that is accurate simultaneously on the predetermined state family. We also need to evaluate a requested entry quickly when its target fragment is much longer than its source fragment.

For a source fragment X , build a balanced tree of its positions. A tree node records a mass shift. A leaf uses that shift to predict the mass interval opposite its source symbol. It chooses either a deletion or a substitution against an allowed target occurrence, and pays for the discrepancy of the chosen endpoints. Internal nodes connect the shifts of their children. This gives a min-plus recurrence with a penalty for changing shift.

The recurrence is a geometric relaxation of alignment. Any assignment of shifts and leaf operations can be converted to an alignment after paying its connection charges and the difference between the total source and target gap masses. Conversely, an optimal alignment supplies a shift at every source cut. The mass-variation bound controls the total penalty of those shifts over the tree. The resulting comparison factor depends on tree branching and depth, rather than on the weights or on the target length. This is the weighted analogue of the source hierarchy used in the unit-cost coarse estimator [28, 6].

Evaluating the recurrence on every integer shift would still be too expensive. Precision sampling gives each node an additive error allowance. A larger allowance permits a coarser shift grid and cheaper leaf evaluation. A smaller allowance reveals more detail. At a singleton, a target symbol whose gap cost is small relative to the allowance can be replaced by an empty-target candidate with a controlled additive error. Only the heavier occurrences need actual substitution queries.

The deterministic restriction on allowed leaf starts confines these occurrences to a mass interval of length proportional to the distance guess. It contains only $O(1 + k/a)$ occurrences with gap cost above a fixed fraction of the allowance a , where k is the guess. This count does not depend on the number of cheap symbols in the fragment. Range reporting finds the heavy occurrences. Piecewise-linear envelopes evaluate their leaf costs on the grid, and two sweeps evaluate the min-plus shift transforms. These operations produce costs for many shifts without scanning the target fragment.

Child estimates are reconstructed through the tree using these random allowances. The analysis fixes a node's incoming allowance before exposing its children's fresh draws. It then proves the same approximation invariant on every relevant grid point. Parent and child grids need not be nested. Their transform error is charged to the chosen mesh sizes. A union bound over the predetermined entry family gives a simultaneous coarse table with factor A_{init} .

Each entry uses its own random stream and completed shared preprocessing. Before an entry's first request, its stream is unexposed. Thus its expected evaluation cost remains bounded after conditioning on the request history. The expected inverse allowance grows by a polylogarithmic factor at each source level. The growing branching factor shortens the tree enough to keep the accumulated overhead subpolynomial. These bounds give expected work $(1 + |X|)N^{o(1)}$ for an entry with source X , uniformly in the target fragment. This source-length bound is the computational counterpart of the gap-mass geometry. It lets the refinement ask for many different target comparisons while charging the total time through the source tree.

2.4 Why refinement can concentrate on narrow bands

Fix a complete good coarse table U , with $c \leq U \leq Ac$ at every state. One refinement pass reduces this factor by a growing parameter F . Put $a = \max\{1, A/F\}$. Scaled coarse estimates and endpoint

cones supply initial values between c/a and $O(Fc)$. A cone adds the gap-mass cost of moving its center's endpoints. The minimum of these cones provides controlled estimates at nearby states as well.

Repeated child comparisons decrease these values. Their average provides a stable cost profile for the next stage. The refinement uses this profile to predict how an alignment's shift evolves as cost accumulates. Searching all child endpoint combinations would be expensive. We instead search near this prediction. To justify that restriction, we must bound both the prediction error and the cost of nodes where the prediction fails.

To explain this, fix an optimal alignment for analysis and split it recursively at the source cuts. Round its target anchors to the mass nets. The child comparisons form a comparison tree. Their costs sum to the parent cost, apart from a controlled endpoint rounding allowance. At one nonterminal, let m_i be the mean cost estimates of its comparison children, let z_i be their mass imbalances, and put $W = \sum_i m_i$. For $m_i > 0$, the ratio z_i/m_i describes shift change per unit of estimated cost. If $m_i = 0$, the imbalance bound gives $z_i = 0$; we take $0/0$ to be zero. The comparison tree stops at physical leaves and at states of true cost zero. The comparison bounds ensure $W > 0$ at a nonterminal.

The main obstruction is variation in these ratios. It is measured by

$$K = \sum_i \frac{z_i^2}{m_i} - \frac{(\sum_i z_i)^2}{W}.$$

This quantity is nonnegative by Cauchy–Schwarz. It vanishes when the child ratios agree. We call K the curvature. The average shift change per unit of estimated cost is the velocity $v = (\sum_i z_i)/W$. The same inequality gives

$$\sum_i |z_i - vm_i| \leq \sqrt{KW}.$$

Thus small K makes every prefix shift close to a common velocity times its accumulated mean cost. There is also a discrepancy G between the comparison-child sum and the cheapest wide comparison. A wide comparison can use different target endpoints at a source cut and pays their connection cost. The computed prefix minimum lies between the comparison prefix's mean cost minus G and that mean cost. Small G lets the algorithm use this computed profile in place of the unknown comparison profile.

The differences between a node's energy z^2/m and the sum of its children's energies telescope over the comparison tree. Together with the averaged child comparisons, this bounds the sum of $K + G$ over all nonterminals. The algorithm need not know which nodes have low curvature. The analysis uses the bound to charge the nodes where the narrow-band prediction fails. If a node instead has nearly extreme velocity, little decrement remains available and a separate bound charges it. At the other nodes, the comparison action lies in the algorithm's narrow band.

A band consists of child comparisons with shared target endpoints. Its defining constraint uses the computed prefix-cost profile. Away from extreme velocity, that constraint is contractive, so the allowed shifts at each cut have a small diameter. This small diameter lets the algorithm search a structured region even though the full set of child actions is large. This curvature-and-band mechanism is inherited from the unit-cost refinement [6]. The weighted geometry makes its endpoint and connection estimates valid for arbitrary metric costs.

Approximate leaves require a change in the telescope. At a terminal, the table value is Q , while the true cost is c . The energy is still bounded by c , since $Q \geq c \geq |z|$. The telescoping sum of mean costs must use the actual terminal value Q . The terminal true costs sum to $O(D)$, so replacing them by $(1 + \varepsilon_{\text{leaf}})$ approximations adds only $O(\varepsilon_{\text{leaf}}D)$. The leaf error is charged once at the frontier, rather than at every level of the source hierarchy.

2.5 Choosing comparisons and reducing their error

Two sampling steps have different roles. Shared samples compare canonical rounded actions for nearby representatives in one mass cell. They group children according to dyadic upper bounds on their costs. Larger cost bounds receive larger sample budgets. The winning representative is then evaluated at its actual current cost. This gives a candidate that is reliable throughout the cell.

The online step chooses one whole band action from a distribution guided by earlier child values. The entropy objective balances those values against uncertainty in the choice of action. Sampling a whole action preserves its shared-endpoint constraints.

The regret analysis uses probability vectors with fixed mass in each child block. It normalizes a child's cost by its dyadic bound. The normalized costs decrease with the refinement index. The monotone-loss inequality from the unit-cost refinement [6] controls the excess cost of the online choices in terms of these decrements. The fixed block masses cancel a common decrement within a block. Only variation between its candidate child states remains. This is what makes the bound fit the narrow-band geometry. We prove the fixed-block version used here.

Intermediate passes need a further argument. When $a > 1$, the refinement's lower bound is c/a . A child estimate can be below its true cost, so a comparison action cannot always be treated as an upper estimate. The baseline is the minimum of endpoint cones for the current group candidates and the wide comparison. The update clips the online candidate between this baseline and a slightly smaller fraction of it. It preserves temporal monotonicity and the lower bound c/a .

For analysis, replace each current value d_t by $\bar{d}_t = \max\{c, d_t\}$. Fix the complete child tables and consider the first index at which a band action, including its endpoint cone, falls below the true parent cost. Before this crossing, the monotone-loss bound controls the excess cost of the sampled actions. The extra cost from clipping an update above its sampled action is charged to the action's child-value decrements. At the crossing, successful shared sampling leaves only a small clipping error. After it, child monotonicity keeps the crossing action's earlier-index Bellman cost below the true cost. The clipped analysis value is then exactly c and contributes no further positive regret. In the final pass, $a = 1$ and every child value is an upper estimate; no crossing is needed.

Summing these bounds over the comparison tree gives one pass with

$$d_S \geq c/a \quad \text{on every outcome,} \quad \mathbb{E}(d_S - c)_+ \leq c/\ell,$$

where ℓ grows with input length. The expectation conditions on the complete fixed good seed table. The curvature budget handles the excluded nodes, the online bound handles covered nodes, and the terminal estimate supplies the leaf bias. Together they reduce the approximation factor.

Independent copies and medians turn this expected error bound into another simultaneous good table. The process repeats until the final pass has $a = 1$. At that point Markov's inequality gives the required root approximation. Complete-table conditioning justifies each transition even though the implementation requests only selected entries. It is distinct from the exposed-history conditioning used to charge their evaluation cost.

2.6 Why the total time stays almost linear

The mathematical tables have a polynomial state domain, but evaluating all their entries would be too expensive. The implementation evaluates only requested values and caches each under one complete key. Requests for the same table entry share that key, even when they come from different representatives or sampled actions. Private random streams give these cached values the same joint distribution as the complete tables used in the proof.

The decisive count distinguishes two kinds of recursive request. A request at an earlier refinement index can have a relatively large number of children. A request at the same index must descend one physical source level and has a much smaller number of children. Action lists and entropy calculations use values already gathered from earlier indices. Current child values are requested only for sampled or exact group terms and selected actions. Candidate enumeration uses these gathered arrays, so it does not multiply the recursive cost by the size of the full action list.

The evaluator orders records by rank, physical height, and phase. Same-node auxiliary reads at equal rank decrease phase. For the dependency count, expand auxiliary records into their scalar requests while retaining their owner decisions. Every resulting edge decreases rank or descends one physical source level at equal rank. These two decreases bound every scalar dependency chain. At a fixed physical depth, the branching of the source tree is balanced by the shrinking source lengths. This balance lets us sum the coarse table's $(1 + |X|)N^{O(1)}$ entry bound over requests with a leading factor of N . All request counts hold without conditioning on a good seed, so the expected-time bound also includes inaccurate intermediate tables.

For the quantitative estimate, let $H = \Theta(\log N)$ and $\chi = \log_2 H$. The parameters stay fixed through a bounded-integer core run. We use coarse branching factor $B = H^\chi$. Its source tree has height $O(H/\chi^2)$, so precision allowances accumulate over fewer levels. Accounting for both the allowances and the branching gives overhead factor $2^{O(H/\chi + \chi^2)}$.

For refinement, we choose $\ell = \Theta(\chi^3)$. This choice supports $p_* = \chi$ while keeping the error budget within $O(1/\ell)$. It also reduces the number of refinement steps to $S = H/\ell^2$, which controls the number of earlier-index dependencies. A request at the same index descends the physical source tree, whose height is $J = O(H/\chi)$. Each level contributes a factor $\ell^{O(1)}$ beyond the source branching already charged to string length. The combined overhead from these levels is $2^{O(J \log \ell)} = 2^{O(H \log \chi / \chi)}$. Earlier-index dependencies across all passes, local action enumeration, finite entropy optimization, and sampling fit within this bound. Thus the core takes expected time

$$N 2^{O(H \log \chi / \chi)} = N^{1+O(\log \log \log N / \log \log N)}.$$

The finite optimizer uses rational approximations and samples from a finite distribution with fair bits. Its mixture selects an action; it does not average recursive costs into the returned table values. This keeps the bit lengths of those values under control.

Finally, condition on the history that selected the normalization bracket. Every such history fixes a valid bounded integer metric. Fresh core runs have the same approximation and expected-time bounds uniformly for that metric. On a good bracket, the normalization and core losses multiply to at most $(1 + \varepsilon/3)^2 \leq 1 + \varepsilon$. A constant number of runs raises the conditional success probability, and combining it with the good-bracket probability gives 5/6. A failed bracket still supplies a polynomial-range metric, so it does not introduce dependence on the original weight ratios into the expected running time.

3 Preliminaries

3.1 Input and computational model

The input consists of strings x, y of total length $N = |x| + |y|$ and a deterministic metric oracle w on their alphabet together with a gap symbol $\perp$. The strings do not contain $\perp$. The metric has nonnegative real values, with $w(a, b) = 0$ exactly when $a = b$. Deleting a , inserting b , and substituting a by b cost $w(a, \perp)$, $w(\perp, b)$, and $w(a, b)$, respectively. We write $\text{WED}_w(x, y)$ for the minimum cost of an edit script.

A metric query takes constant time. The model also permits exact real addition, subtraction, multiplication, division, and comparison. Integer indices and finite rational control data are represented in binary, and their actual bit operations are charged. Random sampling uses independent fair bits. The model provides no floor operation or digit extraction on an arbitrary real register and no ideal random real draw. Space used by real values is measured in registers.

Each input occurrence has an $O(\log(N + 2))$ -bit positional handle. The oracle accepts two handles, or a handle and $\perp$. It returns the distance between the underlying symbols. Different handles can name the same symbol, so equality is tested by a zero metric value. The algorithm outputs a distance estimate. The running-time bound assumes oracle access to costs; it does not include reading an explicitly supplied quadratic distance table.

The accuracy input is a rational $\varepsilon = p_{\text{acc}}/q_{\text{acc}} > 0$, with coprime positive integers $p_{\text{acc}}, q_{\text{acc}}$. For a positive integer u with k binary digits, define its code as $0^{k-1} \text{bin}(u)$. The accuracy encoding concatenates the numerator and denominator codes. For the fixed-accuracy running-time guarantee, constants and thresholds can depend on ε .

3.2 Intervals, masses, and alignments

Characters are indexed from one and boundaries from zero. For boundaries $a \leq b$ of a string Z , write $Z(a, b] = Z_{a+1} \cdots Z_b$, with $Z(a, a]$ empty. For a source interval $I = (l, u]$ and a target endpoint pair $q = (q^-, q^+)$, use the shorthands $x[I] = x(l, u]$ and $y[q] = y(q^-, q^+]$. These boundary conventions follow [6].

Following the gap-mass notation of [7], put

$$g(a) = w(a, \perp), \quad M(Z) = \sum_{a \text{ in } Z} g(a), \quad M_Z(j) = M(Z(0, j]).$$

The sum counts occurrences. We write M_w when the metric must be specified. Unqualified masses use the metric fixed in the current construction. In a local query on fragments X, V , their prefix masses $M_X(i), M_V(j)$ start at their own first boundaries. For arbitrary intervals, put

$$c(I, q) = \text{WED}_w(x[I], y[q]), \quad D = \text{WED}_w(x, y), \quad R = M(x) + M(y).$$

An alignment is a monotone grid path with deletion, insertion, and substitution edges. Its cost is the sum of the corresponding edit costs. An alignment gives an edit script of the same cost. Conversely, trace the occurrences through an edit script. Each surviving original occurrence contributes a diagonal. Deleted original occurrences and surviving inserted occurrences contribute gap edges. The triangle inequality replaces successive changes to an occurrence by one direct operation of no greater cost. An inserted occurrence that is later deleted can be omitted. The surviving original occurrences retain their order, so these operations form an alignment. Thus minimum alignment cost equals edit-script distance.

We use logarithms to base two unless $\ln$ is written. For an integer $k \geq 1$, $[k] = \{1, \dots, k\}$. Empty minima are $+\infty$.

3.3 Exact dynamic programming

For strings of lengths n, m , let $D(i, j)$ be the minimum alignment cost for $x(0, i]$ and $y(0, j]$. Then

$$\begin{aligned} D(0, 0) &= 0, & D(i, 0) &= \sum_{h=1}^i g(x_h), & D(0, j) &= \sum_{h=1}^j g(y_h), \\ D(i, j) &= \min\{D(i-1, j) + g(x_i), D(i, j-1) + g(y_j), \\ &\quad D(i-1, j-1) + w(x_i, y_j)\}. \end{aligned}$$

Every nonempty alignment ends in one of these three operations. Removing it gives the corresponding prefix alignment. Conversely, each prefix alignment extends by that operation. Induction on $i + j$ proves the recurrence.

The computation uses $O((n + 1)(m + 1))$ metric queries and exact arithmetic operations, with polynomial-in- $\log(N + 2)$ indexing overhead per cell. Two rows suffice for storage. Empty inputs and a source of length at most one take $O(N + 1)$ operations. We use this routine for the exact branches of the algorithm.

4 Normalization

The exact routine handles inputs with $N < 2$. In the constructions below, assume $N \geq 2$. We construct one bounded integer metric on the original alphabet. Its distance approximates the original distance on the good bracket event. The bounds on its costs hold on every outcome.

4.1 A bounded-randomness bracket with a certified upper endpoint

Lemma 4.1. *Suppose every character of strings S, T has gap cost at least $b > 0$. Then $\min\{\text{WED}_w(S, T), b\}$ can be computed in $O(1 + |S| + |T|)$ operations. If the lengths differ, return b . Otherwise, return*

$$\min \left\{ b, \sum_j w(S[j], T[j]) \right\}.$$

Proof. Any alignment of cost strictly below b contains no insertion or deletion. It must pair equal positions. If no such alignment exists, both formulas return the cap b . $\square$

For $r > 0$, write $Z_{>r}$ for the string obtained from Z by deleting every occurrence whose gap cost is at most r .

Lemma 4.2. *For every scale $b > 0$ and an ideal uniform $r \in [b, 2b)$, used only for analysis,*

$$\mathbb{E}_r \text{WED}_w(x_{>r}, y_{>r}) \leq 3D.$$

Proof. Project an optimal original alignment to the surviving characters. Original gap edges do not increase in cost. For a diagonal pairing a_1, a_2 , put $\alpha = \min\{g(a_1), g(a_2)\}$ and $\beta = \max\{g(a_1), g(a_2)\}$. Let $s = w(a_1, a_2)$. If both symbols survive, the projected cost is s . If neither survives, it is zero. If only one survives, it is the heavier symbol and the projected cost is β . The reverse triangle inequality gives $\beta - s \leq \alpha$. A mixed outcome has probability at most $(\beta - \alpha)/b \leq s/b$. If such an outcome has positive probability, then $\alpha < 2b$. Hence the expected projected diagonal cost is at most

$$s + \mathbb{P}(\text{mixed}) \max\{\beta - s, 0\} \leq s + (s/b)\alpha \leq 3s.$$

If mixing is impossible, the upper bound s suffices. Sum over the fixed original alignment. Its projection is a feasible filtered alignment, so the expected filtered optimum is at most $3D$. $\square$

Lemma 4.3. *Fix a scale $b > 0$. There is a fair-bit test with the following properties:*

1. *Every CLOSE output certifies $D < 3Nb$.*
2. *If $D < b/10$, then the probability of a FAR output is at most $1/(200N)$.*

3. It uses deterministically $O(N \log N)$ metric/real operations and $N \text{poly}(\log N)$ finite-control work. Its number of fair bits is $O(\log^2 N)$.

Proof. Put $q = \lceil \log_2 N \rceil + 2$. Draw a uniform integer $J \in \{0, \dots, 2^q - 1\}$ using exactly q fair bits. Its cell is

$$I_J = \left[b + \frac{Jb}{2^q}, b + \frac{(J+1)b}{2^q} \right).$$

If no input symbol has gap cost strictly inside I_J , take its left endpoint as the threshold r . Otherwise take $r = b$. Sort the input gap costs once and use predecessor searches to implement this test. A gap cost at a cell's left endpoint has already been removed there; one at its right endpoint is outside the half-open cell. Thus filtering is constant on every cell declared constant.

At most N cells contain a symbol's gap cost in their interior. The probability of choosing such a cell is at most $N2^{-q} \leq 1/4$. Couple the uniform cell with an ideal uniform point inside it. On every other cell, the finite threshold and the ideal threshold produce the same filtered strings. If $D < b/10$, Lemma 4.2 gives expected filtered distance less than $b/2$ under the ideal threshold. Markov's inequality gives distance below b with probability greater than $1/2$. Removing the bad-cell probability leaves probability greater than $1/4$ for the finite sampler.

For each sampled threshold compute $h = \min\{\text{WED}_w(x_{>r}, y_{>r}), b\}$ by Lemma 4.1. Return CLOSE if any sample has $h < b$. Such a sample gives

$$D \leq h + M_w(x \setminus x_{>r}) + M_w(y \setminus y_{>r}) < b + 2Nb \leq 3Nb.$$

Run $t = 4\lceil \log_2(200N) \rceil$ independent samples. Since $(3/4)^4 < 1/2$, their common failure probability is at most $1/(200N)$. All logarithmic counts here are computed from bounded integer bit lengths. Each sample costs $O(N + \log N)$ operations and exactly q fair bits. $\square$

Lemma 4.4. *For distinct nonempty strings, a fair-bit algorithm uses $O(N \log^2 N)$ metric/real operations and $N \text{poly}(\log N)$ finite-control work, and returns $0 < L_{\text{br}} \leq U_{\text{br}}$ such that*

$$\begin{aligned} U_{\text{br}}/L_{\text{br}} &\leq 60N \quad \text{on every outcome,} \\ D &\leq U_{\text{br}} \quad \text{on every outcome,} \\ \mathbb{P}\{L_{\text{br}} \leq D \leq U_{\text{br}}\} &\geq 99/100. \end{aligned}$$

It uses $O(N)$ real registers, $\text{poly}(\log N)$ -bit control data, and $O(\log^3 N)$ fair bits. It uses no floor on real registers.

Proof. Sort the distinct candidate scales

$$\mathcal{S} = \{g(a)/2, g(a) : a \text{ occurs in } x \text{ or } y\}.$$

Let $A = \min \mathcal{S}$ and $B = \max \mathcal{S}$. Every original gap costs at least $2A$. Use Lemma 4.1 to compute $e_0 = \min\{D, 2A\}$. If $e_0 < 2A$, return $[e_0, e_0]$. This is a positive exact singleton because the strings are distinct. Otherwise $2A \leq D \leq NB$.

While A, B are not consecutive in $\mathcal{S}$, test a median candidate strictly between them. Replace A by that candidate on FAR, and replace B by it on CLOSE. Every CLOSE answer is certified. Each update preserves $D < 3NB$, so this bound holds throughout the search on every outcome. For analysis, assign an independent random packet to every candidate scale. The algorithm draws a packet only when it tests that scale. There are at most $2N$ scales. By Lemma 4.3, with probability at least $99/100$, every possible FAR answer also has its valid implication $A/10 \leq D$. On this event the search maintains

$$A/10 \leq D < 3NB.$$

This union is over fixed candidate scales, so adaptive median choices do not affect it.

If the final endpoints have $B \leq 2A$, return $[A/10, 3NB]$. The width is at most $60N$, its upper endpoint is valid by the bound $D < 3NB$, and its lower endpoint is valid on the stated event.

Otherwise $B > 2A$. No input symbol has gap cost in $(A, 2B)$: such a cost in (A, B) would itself be an intervening candidate, while a gap cost in $[B, 2B)$ would have an intervening half. The simplification at A has all surviving gap costs at least $2B$. Compute

$$e = \min\{\text{WED}_w(x_{>A}, y_{>A}), B\}, \quad L_{\text{br}} = \max\{A/10, e/5\}, \quad U_{\text{br}} = 15NL_{\text{br}}.$$

The width is $15N$ on every outcome. If $e = B$, then the bound $D < 3NB$ gives $D < 3NB = 3Ne \leq 15NL_{\text{br}}$. If $e < B$, restoring deleted characters costs at most NA , so

$$D \leq e + NA \leq 5L_{\text{br}} + 10NL_{\text{br}} \leq 15NL_{\text{br}}.$$

These two upper-bound arguments do not use the FAR implications. They prove the asserted upper bound on every outcome. For the lower bound, all thresholds in $[A, 2A)$ give the same filtered strings. Lemma 4.2 consequently gives $e \leq 5D$. Together with the event $A/10 \leq D$, this proves $L_{\text{br}} \leq D$.

The number of scale tests is $O(\log N)$. Lemma 4.3 gives the claimed deterministic running time and bit count. Thresholds and bracket endpoints use only exact arithmetic, original real values, and bounded integer multipliers or divisors. $\square$

The bracket construction follows Das, Kipouridis, and Kociumaka [8]. We retain the upper-endpoint guarantee from its proof on every outcome. This guarantee allows normalization even when the lower endpoint is inaccurate.

4.2 One capped and rounded metric

Lemma 4.5. *Let $[L_{\text{br}}, U_{\text{br}}]$ be any output of Lemma 4.4. Define*

$$\eta_{\text{norm}} = \frac{\varepsilon L_{\text{br}}}{3N}, \quad \bar{w}(a, b) = \left\lceil \frac{\min\{w(a, b), 2U_{\text{br}}\}}{\eta_{\text{norm}}} \right\rceil, \quad \bar{D} = \text{WED}_{\bar{w}}(x, y).$$

Then $\bar{w}$ is an integer metric with

$$1 \leq \bar{w}(a, b) \leq \left\lceil \frac{360N^2}{\varepsilon} \right\rceil \quad (a \neq b).$$

On every bracket outcome,

$$D \leq \eta_{\text{norm}} \bar{D} \leq D + \varepsilon L_{\text{br}}/3.$$

On the good bracket event $L_{\text{br}} \leq D$, this becomes

$$D \leq \eta_{\text{norm}} \bar{D} \leq (1 + \varepsilon/3)D.$$

Proof. The function $x \mapsto \min\{x, 2U_{\text{br}}\}$ is nondecreasing and subadditive on nonnegative reals, so capping preserves the triangle inequality. Upward rounding after scaling preserves it because $\lceil x + y \rceil \leq \lceil x \rceil + \lceil y \rceil$. Positive costs remain positive, and zero costs remain zero. The deterministic width bound gives

$$2U_{\text{br}}/\eta_{\text{norm}} = 6NU_{\text{br}}/(\varepsilon L_{\text{br}}) \leq 360N^2/\varepsilon.$$

By Lemma 4.4, $D \leq U_{\text{br}}$ on every outcome. An original optimal alignment uses no edge more expensive than D , and is unchanged by capping at $2U_{\text{br}}$. Any alignment changed by capping has an

edge of capped cost $2U_{\text{br}} > D$, so it cannot improve the optimum. The capped distance equals D . A monotone alignment has at most N edges. Rounding increases the cost of an optimal capped alignment by at most $N\eta_{\text{norm}} = \varepsilon L_{\text{br}}/3$, while it never decreases any capped cost. This proves the asserted transfer bound. The good-event claim follows from $L_{\text{br}} \leq D$. $\square$

A fixed integer search range. Use exact dynamic programming whenever $\varepsilon N < 361$. In the remaining branch,

$$\frac{360N^2}{\varepsilon} \leq \frac{360}{361}N^3 < N^3.$$

Since N^3 is an integer, every normalized cost is at most N^3 . There is no need to calculate a real ceiling to find the search bound. For one normalized query, obtain $c = \min\{w(a, b), 2U_{\text{br}}\}$ and binary-search for the least integer $j \in \{0, \dots, N^3\}$ with $c \leq j\eta_{\text{norm}}$. This returns exactly $\bar{w}(a, b)$ using $O(\log N)$ exact real comparisons and arithmetic operations. Its integer control data have $O(\log N)$ bits. The construction never extracts digits of c/η_{norm} . To avoid an implicit integer-to-real conversion at each search step, maintain exact real copies of the integer lower and upper bounds. If their integer sum is odd, subtract $1/2$ from half their real sum; otherwise use that half-sum directly. This is the real copy of the binary integer midpoint. Update both copies together, adding one when the lower bound advances. Initial real copies are obtained by scanning the binary encoding of N with repeated doubling, then cubing it. The integer parity test uses finite binary control. Each search step uses a constant number of permitted real operations. The total normalized gap mass $\bar{R}$ is at most N^4 and at least N .

5 Weighted geometry and local queries

Fix an integer metric with total input gap mass $R \leq N^C$ for a fixed constant C . Positive gap costs are at least one. Endpoint coordinates count gap mass, but each endpoint is an original character boundary. We use the gap-mass geometry to compare arbitrary source and target fragments.

5.1 Two weighted geometric facts

In the weighted alignment grid, a horizontal step inserts a target symbol b at cost $g(b)$. A vertical step deletes a source symbol a at cost $g(a)$. A diagonal step pairs a with b at cost $w(a, b)$. The metric assumptions imply that shortest alignment cost equals weighted edit distance. The following gap-mass bound is also used by Das, Kipouridis, and Kociumaka [8].

Lemma 5.1. *Along an alignment, the change in target prefix mass minus source prefix mass has absolute value at most the cost of the corresponding subpath.*

Proof. A gap edge changes this potential by its gap cost in absolute value. A diagonal pairing a, b changes it by $g(b) - g(a)$, whose absolute value is at most $w(a, b)$ by the reverse triangle inequality. Sum along the subpath. $\square$

Lemma 5.2. *Let $X_1, \dots, X_r$ be consecutive source fragments, where $r \geq 1$. Fix an ambient target string Y and a parent fragment $Y(a, b]$. For each i , choose a fragment $V_i = Y(a_i, b_i]$, possibly empty, and put $u_i = M_Y(a_i)$ and $v_i = M_Y(b_i)$. The fragments may overlap or leave the parent fragment. Then*

$$\text{WED}_w(X_1 \cdots X_r, Y(a, b]) \leq \sum_i \text{WED}_w(X_i, V_i) + |u_1 - M_Y(a)| + \sum_{i < r} |u_{i+1} - v_i| + |M_Y(b) - v_r|.$$

Proof. Place optimal child alignments in source order. Join their target endpoints by monotone walks along target boundaries, starting at a and ending at b . Each connecting walk pays the gap masses of the occurrences it crosses. Its cost equals the absolute prefix-mass difference between its endpoints.

Select the first forward crossing of each occurrence in $Y(a, b]$. These selected crossings occur in target order. Keep every child deletion. Keep a diagonal only when its target crossing is selected; otherwise replace it by deletion of the same source occurrence. Keep selected child insertions and omit the others. For each selected connecting crossing, insert that target occurrence. The resulting alignment consumes the source in order and emits exactly $Y(a, b]$.

The concatenated walk crosses an occurrence inside the parent once more forward than backward. For an occurrence outside the parent, the two counts are equal. After removing the selected crossings, the remaining forward crossings can be paired bijectively with backward connecting crossings.

Replacing a diagonal (x_i, y_j) by deletion increases cost by at most $g(y_j)$, since $g(x_i) \leq w(x_i, y_j) + g(y_j)$. Charge this increase to the paired backward crossing. Charge each selected connecting insertion to its own forward crossing. These charges use distinct crossings and total at most the displayed connection mass. Omitted child insertions only save cost. $\square$

5.2 Mass geometry and endpoint nets

Define

$$z(I, q) = (M_y(q^+) - M_y(q^-)) - (M_x(u) - M_x(l)), \quad d(q, r) = |M_y(q^-) - M_y(r^-)| + |M_y(q^+) - M_y(r^+)|.$$

The shift at a path point (i, j) is $M_y(j) - M_x(i)$.

Lemma 5.3. *We have $c \geq |z|$. Both $c(I, \cdot)$ and $|z(I, \cdot)|$ are 1-Lipschitz in d . Partition a source interval I into $k \geq 1$ consecutive intervals $I_1, \dots, I_k$, allowing empty intervals. For a target endpoint pair r and target states $s_1, \dots, s_k$, define*

$$\text{conn}_r(s) = |M_y(s_1^-) - M_y(r^-)| + \sum_{i < k} |M_y(s_{i+1}^-) - M_y(s_i^+)| + |M_y(s_k^+) - M_y(r^+)|.$$

Then

$$c(I, r) \leq \sum_i c(I_i, s_i) + \text{conn}_r(s), \quad |z(I, r)| \leq \sum_i |z(I_i, s_i)| + \text{conn}_r(s).$$

The variation of shift along any alignment segment is at most that segment's cost.

Proof. Lemma 5.1 proves the shift-variation statement. Applying it to an optimal alignment gives $c \geq |z|$.

An endpoint change inserts or removes the target symbols crossed and costs at most their total gap mass. The triangle inequality for edit distance proves the Lipschitz bound for c . The displayed formula for z proves its bound.

The first connection inequality is Lemma 5.2, with ambient target y and parent fragment $y[r]$. For the second, the signed target-mass changes telescope. The source children partition the parent, so

$$z(I, r) - \sum_i z(I_i, s_i) = M_y(r^+) - M_y(s_k^+) + \sum_{i < k} (M_y(s_{i+1}^-) - M_y(s_i^+)) + M_y(s_1^-) - M_y(r^-).$$

The scalar triangle inequality bounds this difference by $\text{conn}_r(s)$. $\square$

Lemma 5.4. Let $I = (l, u]$ and $k \geq 1$. Fix prescribed source cuts $l = l_0 \leq l_1 \leq \dots \leq l_k = u$ and put $I_i = (l_{i-1}, l_i]$. For every target pair q , there are boundaries $q^- = v_0 \leq v_1 \leq \dots \leq v_k = q^+$ such that

$$c(I, q) = \sum_{i=1}^k c(I_i, (v_{i-1}, v_i)).$$

The cuts may repeat, so some source intervals may be empty.

Proof. Take an optimal parent alignment as a chronological sequence of vertices. Fix its start and end times for cut occurrences zero and k . For every internal cut occurrence, choose the first arrival at its source row. These times are weakly ordered even when numeric cuts repeat. Consecutive chosen times partition every path edge into valid child alignments with original occurrence identities. Each subpath is optimal for its child endpoints, since a cheaper replacement would reduce the parent cost. Their costs sum to the parent optimum. Only the outside cuts are pinned to the path's endpoints. Internal cuts at the same source row share its first-arrival vertex. Horizontal edges after that vertex belong to the child after the last internal cut at that row. If the source is empty, all internal cuts use the start vertex and the final child receives the entire horizontal path. $\square$

Lemma 5.5. There are nested sets $\mathcal{N}_h$ of target boundaries, for dyadic integers $h \geq 1$, such that $\mathcal{N}_1$ is the complete boundary set, consecutive points of $\mathcal{N}_h$ are separated by mass at least h , and downward rounding pred_h satisfies

$$0 \leq M_y(q) - M_y(\text{pred}_h(q)) < 2h.$$

At $h = 1$ the error is zero. After prefix preprocessing, all levels through mass R require $O(N \log R)$ comparisons and stored boundary records, with $O_C(N \log^2(N + 2))$ binary work and stored bits. A mass interval of length b contains $O(1 + b/h)$ net points. Reporting k points uses $O(\log N)$ mass comparisons and k record accesses, with $O_C((\log N + k) \log(N + 2))$ binary work.

Proof. Starting with $\mathcal{N}_1$, construct each $\mathcal{N}_h$ from $\mathcal{N}_{h/2}$. Retain boundary zero, then greedily select the first old-net point whose mass is at least h beyond the previous selected point. This gives nesting and separation.

Let $p = \text{pred}_{h/2}(q)$. Induction gives $M_y(q) - M_y(p) < h$. Let r be the selected predecessor of p . The greedy rule gives $M_y(p) - M_y(r) < h$; the difference is zero if $r = p$. No old-net point lies between p and q , so $r = \text{pred}_h(q)$. Adding the two bounds proves the rounding error below $2h$. This argument also covers an arbitrarily large gap between consecutive prefix masses. Each level is one scan. Binary search in the sorted net array gives range reporting. $\square$

5.3 Integer leaf queries

An empty source interval has exact cost equal to target gap mass. At a singleton source a , write $u = g(a)$ and $G = \sum_{b \in V} g(b)$ for the target interval V . Define the saving

$$B(a, b) = u + g(b) - w(a, b), \quad 0 \leq B(a, b) \leq 2g(b).$$

The exact cost is $u + G - \max(\{0\} \cup \{B(a, b) : b \in V\})$.

Fix a positive integer k . Retain the k largest-gap positions in V , or all positions if fewer exist. Among positions with equal gap cost, retain the earlier original position first. Define

$$Q_k(a, V) = u + G - \max(\{0\} \cup \{B(a, b) : b \text{ is retained}\}).$$

For an empty source, let Q_k be the exact answer. For target fragments V, V' , $d(V, V')$ denotes the distance between their endpoint pairs.

Lemma 5.6. *The value Q_k is an integer in $[0, R]$, is zero exactly at zero-cost states, and satisfies*

$$c \leq Q_k \leq (1 + 2/k)c, \quad |Q_k(a, V) - Q_k(a, V')| \leq 3d(V, V').$$

Preprocessing uses $O(N)$ comparisons and records, with $O_C(N \log(N + 2))$ binary work and stored bits, in addition to gap queries. A query uses $O(k \log N)$ bounded-word operations and $O(k)$ metric calls. Its binary control and arithmetic cost is $O_C(k \log^2(N + 2))$, in addition to the metric-query cost.

Proof. If the maximum saving is attained by zero or a retained position, the value is exact. Otherwise, let b be a maximizing position that is not retained. All k retained positions have gap cost at least $g(b)$, so $g(b) \leq G/(k + 1)$. The exact cost satisfies

$$c = u + G - B(a, b) \geq G - g(b) \geq kG/(k + 1),$$

where the first inequality uses $B(a, b) \leq u + g(b)$. The missed saving is at most $2g(b)$. Its ratio to c is at most $2/k$. A zero-cost singleton state has one matching target symbol, so it is exact.

Insert an endpoint symbol of gap cost t . If it enters the retained set, it displaces at most one old symbol, whose gap cost is at most t . The maximum retained saving can then increase or decrease by at most $2t$. Since G increases by t , the change in Q_k lies in $[-t, 3t]$. Deletion reverses this operation. Telescoping the endpoint operations proves the regularity bound.

A range-maximum tree on target gap costs is built in linear time. Its maximum records choose the earliest position among ties. A priority queue reports an interval's maximum and replaces the interval by its two remaining sides. This gives the k heaviest positions in $O(k \log N)$ time. Das, Kipouridis, and Kociumaka [8] use this reporting structure. Evaluate the reported positions' savings with k metric calls. $\square$

Remark 5.7. Use the line metric with coordinates $\mathbf{c} = -1$, $\perp = 0$, $\mathbf{b} = 1$, and $\mathbf{a} = 2$. Let the ambient target be $Y = \mathbf{c}^k \mathbf{b}$. Let V be its suffix of length k , and let $V' = Y$. For source $\mathbf{a}$, the estimator retains every position of V and returns $Q_k(\mathbf{a}, V) = k$. In V' , the earlier-position tie rule retains its first k symbols, all equal to $\mathbf{c}$, giving $Q_k(\mathbf{a}, V') = k + 3$. The endpoint distance is $d(V, V') = g(\mathbf{c}) = 1$, while the true distances are k and $k + 1$. Thus the Lipschitz constant 3 is sharp.

6 The weighted coarse estimator

We construct a random coarse table for a fixed family of fragment pairs. The table must be accurate simultaneously, although the evaluator can request its entries adaptively. A separate work bound must remain valid conditional on the history before an entry's first request. We do not condition that work bound on approximation success.

The source hierarchy, precision reconstruction, and conditional first-request accounting are adapted from [6]. The mass potential and heavy-character reporting follow [8]. The weighted implementation uses allowance-dependent grids and leaf envelopes that do not scan the target fragment.

6.1 Model, parameters, and shared preprocessing

Fix constants $C, s > 0$. The strings X_0, Y_0 have total length at most $N \geq 2$. The metric w is integer-valued on their alphabet together with $\perp$, and every positive metric value belongs to $[1, N^C]$.

A metric query takes constant time and its bounded integer answer is used in binary. If the answer is supplied in an exact real register, choose a fixed integer $d \geq C$ and binary-search $\{0, \dots, N^d\}$ for its value. This recovers its binary representation with polylogarithmic overhead. Maintain real copies of the integer search bounds, as in the query implementation of Lemma 4.5. Each occurrence has an $O(\log N)$ -bit handle to its original input position. Different handles may name the same symbol. Test equality by a zero metric query. Local prefix-mass offsets do not change the handles supplied to substitution queries.

Write

$$g(a) = w(a, \perp), \quad M(Z) = \sum_{a \text{ in } Z} g(a), \quad R_0 = M(X_0) + M(Y_0).$$

The sum counts occurrences. For a positive integer z , let $\text{leastpow}_2(z)$ be the least power of two at least z . Set

$$H = \text{leastpow}_2(\lceil \log_2(N + R_0 + 2) \rceil), \quad \chi = \log_2 H,$$

and

$$p_* = \max\{1, \chi\}, \quad B = H^{p_*}, \quad A_{\text{init}} = BH^4, \quad Q_0 = H^4, \quad \lambda = \frac{1}{64H}.$$

These parameters use the full input, even for a shorter queried source. They are computable by bit lengths and integer comparisons. Both B and Q_0 are powers of two. Since $R_0 \leq N^{C+1}$, we have $H = \Theta_C(\log N)$ and $B = N^{o(1)}$. We analyze the fixed large-input branch $\chi \geq 2^{1000}$, where $p_* = \chi$ and $\log_2 B = \chi^2$. In this branch, $128H^5(1 + \ln B) \leq H^9$. Exact dynamic programming handles the remaining finite input-size range.

Before exposing random bits, fix an indexed mathematical family $\mathcal{E}$ of at most N^s fragment pairs (X, V) of X_0, Y_0 . Distinct indices may name the same pair. Assign each index its own private stream of independent fair bits. Streams for different indices and table instances are independent. The evaluator does not enumerate the family. It allocates entries and samples their bits only as needed. An entry may read only its full index, its private random bits, its local workspace, and the completed deterministic shared data. It does not call another entry or read another entry's auxiliary state.

Shared preprocessing queries every gap cost and stores both strings' prefix masses. It also builds a segment tree on the target positions. Each tree node stores a position of maximum gap mass in its interval, with ties resolved by original target position. These structures require $O(N \log N)$ bounded-word operations and $O(N)$ metric queries; their charged binary work and storage are $N \text{ poly}(\log N)$. The structures are read-only during entry evaluation.

For an entry, put $m = |X|$. Obtain a local target prefix mass by subtracting the fragment's initial global prefix mass. This takes constant-time accesses and does not scan V . Local source prefix masses require $O(1 + m)$ source work. If X is nonempty, set $K = \text{leastpow}_2(M(X) + M(V))$ and use all guesses $k = 1, 2, 4, \dots, K$. Thus $K \leq 2N^{C+1}$, and the guess set is fixed before any computed estimate is used.

6.2 A mass-shift tree with restricted leaf candidates

Fix a nonempty source $X = x_1 \dots x_m$ and target $V = y_1 \dots y_n$. Write

$$M_X(i) = \sum_{h=1}^i g(x_h), \quad M_V(j) = \sum_{h=1}^j g(y_h), \quad M_X(0) = M_V(0) = 0, \quad \Delta = |M_X(m) - M_V(n)|.$$

Let $k \geq 1$ be an integer power of two and put $\mathcal{D}_k = [-k, k] \cap \mathbb{Z}$.

For character x_i , put $g_i = g(x_i)$. At shift $v \in \mathcal{D}_k$ its predicted target mass interval is

$$[M_X(i-1) + v, M_X(i) + v].$$

The following leaf envelope uses actual target operations:

$$F_i(v) = \min \left\{ \min_{0 \leq j \leq n} [g_i + |M_V(j) - M_X(i-1) - v| + |M_V(j) - M_X(i) - v|], \right. \\ \left. \min_{\substack{1 \leq j \leq n \\ |M_V(j-1) - M_X(i-1)| \leq k}} [w(x_i, y_j) + |M_V(j-1) - M_X(i-1) - v| + |M_V(j) - M_X(i) - v|] \right\}.$$

The first minimum deletes x_i and selects an actual target boundary. The second pairs x_i with an actual target occurrence. The two absolute-value terms compare its fragment's mass endpoints with the predicted interval. If no singleton satisfies the start restriction, its minimum is $+\infty$; the deletion minimum is always finite. For fixed (X, V, k) , the candidate set does not depend on v . Each candidate is 2-Lipschitz in v , so the minimum F_i has the same bound.

For a source node $(l, u]$, set $c_i = l + \lfloor i(u-l)/B \rfloor$ for $i = 0, \dots, B$. Its children $(c_{i-1}, c_i]$ are consecutive and have widths differing by at most one. Apply these cuts recursively, stopping at width zero or one. The maximum depth is

$$D_0 = \lceil \log_B \max\{1, m\} \rceil,$$

and there are $O(Bm)$ nodes. Set $T_w = 0$ at empty nodes and $T_w = F_i$ at a singleton node for x_i . Otherwise define

$$T_w(v) = \sum_{u \text{ child of } w} \min_{z \in \mathcal{D}_k} [T_u(z) + 2|v - z|].$$

Lemma 6.1. *Let $d = \text{WED}_w(X, V)$ and let o be the root. For every guess k ,*

$$d \leq T_o(0) + \Delta.$$

If $k \geq d$, then

$$T_o(0) \leq (2 + 2BD_0)d.$$

Every node table T_w is $2B$ -Lipschitz on $\mathcal{D}_k$.

Proof. Each minimum transform is 2-Lipschitz. An internal node sums at most B such transforms, a singleton table is 2-Lipschitz, and an empty table is constant. This proves the regularity assertion.

For the lower bound, expand the recurrence as a minimum over assignments of shifts to tree nodes, with root shift zero, and choices of leaf operations. Fix such an assignment. For leaf i , let (u_i, v'_i) be the mass endpoints of its actual chosen target fragment, and let z_i be its shift. The leaf penalty is

$$e_i = |u_i - (M_X(i-1) + z_i)| + |v'_i - (M_X(i) + z_i)|.$$

The connection expression from Lemma 5.2 is at most

$$\sum_i e_i + |z_1| + \sum_{i < m} |z_{i+1} - z_i| + |z_m| + \Delta.$$

Traverse the source tree from the root through its nonempty leaves in left-to-right order and back to the root. Triangle inequality bounds the shift variation in this display by twice the sum of absolute shift differences on tree edges. Empty nodes and their leaves can be skipped. The connection lemma

converts the chosen operations into an alignment of cost at most the recurrence's value plus Δ . Minimize over the assignment to obtain the bound on d .

For the upper bound, fix an optimal alignment. Each node will receive its shift at the node's left source boundary. At every source cut i choose an alignment vertex (i, b_i) , in order, with $b_0 = 0$ and $b_m = n$. For example, use the first vertex on each internal source-cut row. These points partition the alignment into leaf subpaths of costs d_i whose sum is d . Assign shift

$$z_i = M_V(b_i) - M_X(i)$$

to every tree node starting at cut i . Lemma 5.1 gives $|z_i| \leq d \leq k$.

If the subpath for x_i deletes it, the empty candidate at boundary b_{i-1} has value $2g_i \leq 2d_i$ at shift z_{i-1} . Otherwise suppose the subpath pairs x_i with y_j . Every vertex of the full alignment has mass discrepancy at most d , so $|M_V(j-1) - M_X(i-1)| \leq k$ and this singleton is allowed in the leaf envelope. Put $a = M_V(j-1) - M_V(b_{i-1}) \geq 0$. Its value is

$$w(x_i, y_j) + a + |a + g(y_j) - g_i| \leq 2w(x_i, y_j) + 2a \leq 2d_i.$$

Here a is the cost of insertions before the diagonal, and the reverse triangle inequality bounds $|g(y_j) - g_i|$.

For a parent and child, the absolute change of their assigned shifts is at most the cost of the alignment segment between their chosen starting vertices. That segment lies inside the parent's segment, whose cost also bounds the shift change. There are at most B children per parent. At each depth, parent segments partition the alignment, so the sum of edge differences at that depth is at most Bd . Sum over the D_0 depths and add the leaf bound $2d$. $\square$

6.3 Fast approximate leaf arrays

Use the shared target-prefix array and maximum tree. Subtracting the queried fragment's initial prefix mass supplies all local coordinates $M_V(j)$ without scanning the fragment.

Lemma 6.2. *Fix a source character x_i , an allowance $a > 0$, a guess k , and a sorted list $G \subseteq \mathcal{D}_k$. In*

$$O\left(\left(1 + \frac{k}{a} + |G|\right) \log N\right)$$

operations, up to finite-arithmetic overhead, one can compute values $\hat{F}_i(v)$ for $v \in G$ satisfying

$$F_i(v) \leq \hat{F}_i(v) \leq F_i(v) + a/2.$$

The bound has no dependence on the target fragment's character count.

Proof. Set $t = a/4$. Retain only allowed singleton candidates with $g(y_j) > t$; retain all empty-boundary candidates. Compare a discarded singleton with the empty candidate at its starting boundary. The latter minus the former is at most

$$g_i - w(x_i, y_j) + g(y_j) \leq 2g(y_j) \leq 2t = a/2.$$

This proves the approximation bound.

Allowed retained starts lie in an interval of mass width $2k$. For any two successive retained starts, their difference exceeds t , since the first retained character itself has gap cost greater than t . There are $O(1 + k/t)$ such characters. Binary searches locate the allowed target position interval, and the maximum tree reports its characters heavier than t in $O((1 + k/t) \log N)$ time.

For each reported character, let

$$u = M_V(j-1) - M_X(i-1), \quad z = M_V(j) - M_X(i), \quad c_j = w(x_i, y_j).$$

Its value as a function of v is $c_j + |v - u| + |v - z|$. Let $l_j = \min\{u, z\}$ and $r_j = \max\{u, z\}$. This function has the three pieces

$$\begin{cases} c_j + l_j + r_j - 2v, & v \leq l_j, \\ c_j + r_j - l_j, & l_j \leq v \leq r_j, \\ c_j - l_j - r_j + 2v, & v \geq r_j. \end{cases}$$

Sort the breakpoints. Suffix and prefix minima evaluate the minimum left and right pieces at every queried shift. A sweep with an active minimum heap evaluates the plateau pieces. The work is linear in the number of reported candidates plus queried shifts, with a logarithmic factor.

For an empty candidate, put $s = M_X(i-1) + v$. Its minimum value is

$$g_i + \min_j (|M_V(j) - s| + |M_V(j) - s - g_i|) = 2g_i + 2 \operatorname{dist}([s, s + g_i], \{M_V(0), \dots, M_V(n)\}).$$

A binary search tests whether the interval contains a target boundary. Otherwise its predecessor and successor give the distance. This costs $O(\log N)$ per queried shift, including when the predicted interval extends beyond the target. $\square$

The procedure **ADDITIVE-LEAF-ARRAY** implements the construction in Lemma 6.2 and returns its array in the supplied grid order. All substitution queries use the original occurrence handles. Reporting and heap ties use original target positions.

6.4 Adaptive integer grids and precision reconstruction

Set the root allowance $a_o = k$. Independently for each child u of a nonempty internal node w , draw R_u uniformly from $\{1, \dots, B\}$ and set

$$t_u = \frac{a_w R_u}{BQ_0}, \quad a_u = \frac{\lambda}{2} t_u = \frac{a_w R_u}{128BH^5}.$$

Use independent draws for different entry indices, guesses, and tree edges. All draws can be defined in advance, including unused values at empty nodes. A draw uses exactly $\log_2 B$ fair bits.

At a nonempty node w , let h_w be the largest power of two at most $a_w/(16B)$ if that power is at least one, and otherwise let $h_w = 1$. Equivalently,

$$h_w = \max\{1, 2^{\lfloor \log_2(a_w/(16B)) \rfloor}\}.$$

The implementation uses bit lengths and integer comparisons, not a real-arithmetic logarithm. Define the integer grid

$$G_w = (h_w \mathbb{Z} \cap [-k, k]) \cup \{-k, 0, k\}.$$

An empty node is represented by the identically zero function, with no allocated grid. At a nonempty leaf, evaluate Lemma 6.2 with allowance a_w on G_w , obtaining $\tilde{T}_w$.

For an internal node and $v \in G_w$, form

$$\hat{s}_u(v) = \min_{z \in G_u} (\tilde{T}_u(z) + 2|v - z|)$$

for each nonempty child; for an empty child set $\hat{s}_u(v) = 0$. Then define

$$\tilde{T}_w(v) = \sum_{u \text{ child of } w} \mathbf{1}_{\hat{s}_u(v) \geq t_u} \max\{\hat{s}_u(v), a_w/Q_0\}.$$

This algorithm computes all nonempty nodes. It does not prune by source mass, source width, or a claimed upper bound on true node costs.

For a sorted child grid, compute prefix minima of $\tilde{T}_u(z) - 2z$ and suffix minima of $\tilde{T}_u(z) + 2z$. Merge the parent coordinates with these two sweeps. Their minimum gives $\hat{s}_u(v)$ at every parent coordinate in $O(|G_w| + |G_u|)$ operations. The grids need not be nested.

Lemma 6.3. *Suppose a nonempty child has, on its grid,*

$$T_u(z)/f - a_u \leq \tilde{T}_u(z) \leq fT_u(z) + a_u, \quad 1 \leq f \leq 2.$$

Write $s_u(v) = \min_{z \in \mathcal{D}_k} (T_u(z) + 2|v - z|)$. Then

$$s_u(v)/f - \lambda t_u \leq \hat{s}_u(v) \leq fs_u(v) + \lambda t_u.$$

Proof. The lower bound with error a_u follows directly: restriction of the minimum can only increase it, and a nonnegative penalty dominates its value divided by f .

If $h_u = 1$, the child grid contains the full integer domain, so the upper bound with error a_u also follows directly. This case is essential when a_u is smaller than one.

If $h_u > 1$, round an integer minimizer to a child grid point at distance at most $h_u \leq a_u/(16B)$. By Lemma 6.1, its true value increases by at most $2Bh_u$. Its penalty increases by at most $2h_u$. Hence

$$\hat{s}_u(v) \leq fs_u(v) + a_u + (2Bf + 2)h_u \leq fs_u(v) + \frac{11}{8}a_u \leq fs_u(v) + \lambda t_u.$$

The penultimate bound uses $B \geq 1$ and $f \leq 2$; the last uses $a_u = \lambda t_u/2$. The lower bound is weaker than the already established error- a_u bound. Empty children satisfy both inequalities exactly. $\square$

Lemma 6.4. *For every fixed $c_0 > 0$, when N is sufficiently large as a function of C, s, c_0 , with probability at least $1 - N^{-c_0}$ the following holds simultaneously over $\mathcal{E}$, all guesses, all nodes, and their grid points:*

$$T_w(v)/f_w - a_w \leq \tilde{T}_w(v) \leq f_w T_w(v) + a_w, \quad f_w = \left(\frac{1 + \lambda}{1 - \lambda}\right)^{D_0 - \text{depth}(w)} \leq 2.$$

Proof. Fix an internal node and condition only on the root-to-node allowance draws, including its incoming edge. Then $a = a_w$, the node's grid, and each true child transform $s_i = s_i(v)$ at a fixed grid point are determined. The comparison lists below depend only on these fixed data. They are independent of the fresh child draws R_i . We apply concentration to them without conditioning on descendant success. Separately, the child invariants will place the computed transforms between these lists. Let f' be the factor for the children's structural depth. Write $\bar{a} = a/Q_0$ for this node's allowance scale, and put

$$t_i = \bar{a}R_i/B, \quad h_i^- = \frac{s_i}{f'(1 + \lambda)}, \quad h_i^+ = \frac{f's_i}{1 - \lambda}, \quad \phi_i(h) = \mathbf{1}_{h \geq t_i} \max\{h, \bar{a}\}.$$

If the child invariants hold, Lemma 6.3 gives $s_i/f' - \lambda t_i \leq \hat{s}_i \leq f's_i + \lambda t_i$. Consequently,

$$\phi_i(h_i^-) \leq \phi_i(\hat{s}_i) \leq \phi_i(h_i^+).$$

Indeed, $h_i^- \geq t_i$ implies $\hat{s}_i \geq h_i^-$; and $\hat{s}_i \geq t_i$ implies $\hat{s}_i \leq h_i^+$.

Concentrate each of the two fixed lists $h_i = h_i^-$ and $h_i = h_i^+$ without conditioning on descendant success. When $0 \leq h_i < \bar{a}$, $\phi_i(h_i)$ equals $\bar{a}$ with probability $\lfloor Bh_i/\bar{a} \rfloor/B$ and zero otherwise. When $h_i \geq \bar{a}$, it equals h_i deterministically. Thus

$$0 \leq h_i - \mathbb{E}\phi_i(h_i) \leq \bar{a}/B, \quad \text{Var}(\phi_i(h_i)) \leq \bar{a}h_i, \quad |\phi_i(h_i) - \mathbb{E}\phi_i(h_i)| \leq \bar{a}.$$

Write $V_h = \sum_i h_i$ and $Z = \sum_i \phi_i(h_i)$. The total bias is at most $\bar{a}$. Bernstein's inequality, applied with deviation $\lambda V_h + a/2$, gives

$$\mathbb{P}(|Z - V_h| > \lambda V_h + a \mid \text{ancestors}) \leq 2 \exp(-\lambda Q_0/12).$$

If $V_h \geq a$, the variance denominator is at most $3\bar{a}V_h$ and the squared deviation is at least $\lambda a V_h$. If $V_h \leq a$, the denominator is at most $3\bar{a}a$ and the squared deviation is at least $a^2/4$. The bias is absorbed by the remaining $a/2$ because $Q_0 \geq 2$.

On these two concentration events, the displayed sandwich inequality gives the factor update $(1 + \lambda)/(1 - \lambda)$ and additive error a . Induct from the leaves, where Lemma 6.2 gives an even smaller error. Early singleton leaves can use the displayed larger structural factor. Empty nodes are exact. Since $D_0 \leq H$ for sufficiently large N , the product factor is less than two.

It remains to justify the union bound for random grids. A grid is fixed after conditioning on the node's ancestors. Its cardinality is at most $2k + 1$, since it contains only integer shifts. The largest guess is at most $2(M(X) + M(V)) \leq 2N^{C+1}$. There are at most N^s entries, $O_C(H)$ guesses per entry, and $O(BN)$ nodes per guess. Thus the number of tested lists is bounded by $N^{s+C+2+o(1)}$. We may union-bound the uniform conditional bound at each fixed node over its polynomial maximum grid count, then remove the conditioning. No enumeration of all ancestor histories is required. Finally, $\lambda Q_0/12 = H^3/768$ dominates every fixed multiple of $\log N$. This proves the claimed simultaneous failure probability for sufficiently large N .

An explicit smaller event count is also available. A nonempty internal source node has at least two nonempty children, and the nonempty leaves are the source occurrences. Thus a source of width m has at most $m - 1$ internal nodes. Empty nodes and singleton leaves need no concentration event. There are at most $(C + 1)H + 2$ guesses per entry and at most $4N^{C+1} + 1$ integer shifts per grid. If $\mathcal{E}_{\text{rec}}$ is the intersection of both fixed-list events at every tested parent coordinate, then

$$\mathbb{P}(\mathcal{E}_{\text{rec}}^c) \leq 4N^s((C + 1)H + 2)(N - 1)(4N^{C+1} + 1) \exp(-H^3/768).$$

This counts the random grids only after the stated conditioning and then integrates the uniform conditional bound. For $C = 3, s = 6$ it is at most $100HN^{11} \exp(-H^3/768)$. $\square$

6.5 Standalone entry evaluation

The following procedures evaluate exactly the arrays just defined. Use the canonical balanced cuts, left-to-right child order, and increasing dyadic guesses. Index each allowance draw by the entry identifier, its guess, and its tree-edge path. The identifier includes the table-instance identifier and the complete family index. When a family is indexed by source-tree nodes and target endpoints, retain the full source-node label and original endpoint indices. Equal fragments with different labels retain different keys.

For a nonempty unequal pair, define the candidate at guess k by

$$U_k = \left\lceil 2(\tilde{T}_o(0) + k) + \Delta \right\rceil, \quad \Delta = |M(X) - M(V)|.$$

Algorithm 1 Adaptive coarse-node arrays

Require: A private coarse-tree node w , allowance $a_w > 0$, and fixed guess k .

Ensure: The sorted grid array $(G_w, \tilde{T}_w)$, or the zero function for an empty node.

```
1: procedure COARSE-NODE( $w, a_w, k$ )
2:   if  $w$  is empty then
3:     return the identically zero function
4:   end if
5:    $h_w \leftarrow \max\{1, 2^{\lfloor \log_2(a_w/(16B)) \rfloor}\}$ , using bounded binary arithmetic.
6:    $G_w \leftarrow$  the sorted list  $(h_w\mathbb{Z} \cap [-k, k]) \cup \{-k, 0, k\}$ .
7:   if  $w$  is a singleton then
8:      $\tilde{T}_w \leftarrow$  ADDITIVE-LEAF-ARRAY( $w, k, a_w, G_w$ ).
9:     return  $(G_w, \tilde{T}_w)$ .
10:  end if
11:  for each child  $u$  in left-to-right order do
12:    Draw  $R_u$  uniformly from  $\{1, \dots, B\}$  using the entry's private stream.
13:     $t_u \leftarrow a_w R_u / (BQ_0)$ ;  $a_u \leftarrow a_w R_u / (128BH^5)$ .
14:    Obtain the child array from COARSE-NODE( $u, a_u, k$ ).
15:  end for
16:  for each child  $u$  do
17:    if  $u$  is empty then
18:      Set  $\hat{s}_u(v) \leftarrow 0$  for every  $v \in G_w$ .
19:    else
20:      Use two merged sweeps to compute, for every  $v \in G_w$ ,
21:       $\hat{s}_u(v) \leftarrow \min_{z \in G_u} \{\tilde{T}_u(z) + 2|v - z|\}$ .
22:    end if
23:  end for
24:  for each  $v \in G_w$  do
25:     $\tilde{T}_w(v) \leftarrow \sum_{u \text{ child of } w} \mathbf{1}_{\hat{s}_u(v) \geq t_u} \max\{\hat{s}_u(v), a_w/Q_0\}$ .
26:  end for
27:  return  $(G_w, \tilde{T}_w)$ .
end procedure
```

Algorithm 2 A standalone initial-seed entry

Require: Fragments X, V , completed deterministic preprocessing, and the entry's private random stream.

Ensure: A nonnegative integer at most $M(X) + M(V)$; no other initial entry is requested.

```
1: procedure INITIAL-ENTRY( $X, V, \omega$ )
2:   if  $X$  is empty then
3:     return  $M(V)$ 
4:   end if
5:   if  $|X| = |V|$  and corresponding symbols have oracle distance zero then
6:     return 0
7:   end if
8:   if  $\chi < 2^{1000}$  then
9:     return the exact alignment dynamic-programming value.
10:  end if
11:  Form the local source prefix masses.
12:   $\Delta \leftarrow |M(X) - M(V)|$ ;  $U_{\text{gap}} \leftarrow M(X) + M(V)$ .
13:   $K \leftarrow \text{leastpow}_2(U_{\text{gap}})$ ;  $U_{\text{best}} \leftarrow U_{\text{gap}}$ .
14:  for  $k = 1, 2, 4, \dots, K$  do
15:    Build the private balanced  $B$ -ary source tree, with root  $o$ .
16:     $(G_o, \tilde{T}_o) \leftarrow \text{COARSE-NODE}(o, k, k)$ .
17:     $U_k \leftarrow \lceil 2(\tilde{T}_o(0) + k) + \Delta \rceil$ .
18:     $U_{\text{best}} \leftarrow \min\{U_{\text{best}}, U_k\}$ .
19:  end for
20:  return  $U_{\text{best}}$ .
21: end procedure
```

The entry answer is the minimum of all these candidates and $M(X) + M(V)$. Empty sources and equal strings are handled exactly before sampling.

Empty child transforms are zero and need no allocated grid. Draws for unused empty nodes may be included in the private stream; they are independent of all used draws. No grid point outside $[-k, k] \cap \mathbb{Z}$ is used. The endpoint K of the guess loop is fixed before U_{best} is updated.

The entry uses only its private workspace and the completed read-only prefix arrays and reporting tree. It never calls another initial entry or reads another entry's auxiliary state. On completion, the evaluator caches its integer answer. Repeated requests use that cached answer.

6.6 Finite arithmetic and adaptive work

Lemma 6.5. *Every entry evaluation uses polynomial space, random bits, and finite binary numbers of $O_C(H \text{ polylog } H)$ bits. Each arithmetic operation can be implemented with $\text{poly}(H)$ bit operations.*

Proof. The exact small-input branch has polynomial workspace and finite binary arithmetic by the alignment dynamic program. Consider the large-input branch.

All masses and candidate breakpoints are integers of polynomial magnitude. The guesses are at most $2N^{C+1}$, and every grid is a subset of the integer interval $[-k, k]$. The complete tree and all its grids have polynomial size.

All allowances and thresholds are dyadic. A common denominator for depth at most D_0 needs

$$O(D_0(\log B + 5 \log H + 7) + \log H) = O(H + \chi^2) = O(H)$$

bits. Products of the R_i have at most $D_0 \log B = O(H + \log B)$ bits. An empty-boundary candidate at zero bounds every true leaf value by $2M_X(m) + 2k$, which is polynomial in N . The computed values also have polynomial magnitude: each reconstruction adds at most a_w/Q_0 per child beyond a

chosen transformed value, and a transform can always choose a grid endpoint at penalty at most $4k$. Summing these polynomial contributions over the polynomial-size tree gives a polynomial bound. Minima select existing dyadic values, and sums preserve the common denominator. A draw of R_i uses exactly $\log_2 B$ unbiased bits. Ordinary binary arithmetic proves the assertion. $\square$

Lemma 6.6. *For one entry (X, V) , all guesses can be evaluated in expected $(1 + m)2^{O_C(H/\chi + \chi^2)} = (1 + m)N^{o(1)}$ work, uniformly over the target fragment and without conditioning on any accuracy event.*

Proof. From the allowance definition,

$$\mathbb{E} \left[\frac{a_w}{a_u} \middle| a_w \right] = 128H^5 \sum_{j=1}^B \frac{1}{j} \leq H^9$$

for all sufficiently large H . At depth r this implies

$$\mathbb{E}[k/a_w] \leq H^{9r}.$$

The dyadic grid convention gives

$$|G_w| = O(1 + Bk/a_w).$$

When $h_w > 1$, its mesh is greater than $a_w/(32B)$; when $h_w = 1$, $a_w < 32B$, so the same cardinality bound follows directly.

Internal work is $O(B|G_w| + \sum_u |G_u|)$, apart from arithmetic factors. Lemma 6.2 bounds leaf work by $O((1 + k/a_w + |G_w|) \log N)$. Summing over $O(Bm)$ nodes gives the conservative expected bound

$$(1 + m)B^{O(1)}H^{9D_0 + O(1)}$$

per guess, including logarithmic reporting factors. Lemma 6.5 absorbs finite arithmetic and sampling into a factor $H^{O_C(1)}$. There are $O_C(H)$ guesses. Moreover,

$$D_0 \leq 1 + \frac{H}{p_* \log_2 H}, \quad \log_2(B^{O(1)}H^{9D_0 + O(1)}) = O_C(H/p_* + p_* \log H) = O_C(H/\chi + \chi^2) = o(H).$$

Reading source gap costs, checking equality, and creating its local prefix array costs $O(1 + m)$. Target access uses the shared structures. All estimates depend only on the independent allowance draws for their work bounds, so the calculation does not require successful reconstruction. The exact branch lies in a fixed finite input-size range and is absorbed into the multiplicative constant. $\square$

Before an entry's first request, the exposed computation has not inspected its private random stream. The request can depend on previous entry answers, but those answers depend only on their other streams. Conditional on the complete preceding history and the first requested entry, its stream retains its original distribution. Lemma 6.6 gives the same expected bound under this conditioning. Repeated requests need only the cached-answer lookup.

The event in Lemma 6.4 concerns the entire fixed family $\mathcal{E}$, including entries that are never requested. This accuracy event and the conditional work argument use separate conditionings. No accuracy event is assumed in the work argument.

6.7 The coarse-table guarantee

Theorem 6.7. *After $N \text{poly}(\log N)$ charged binary preprocessing work and $O(N)$ metric queries, the construction gives a lazily evaluated random table U on any predetermined indexed family $\mathcal{E}$ of at most N^s fragment pairs. Distinct indices may name the same pair and have separate keys and private random streams.*

For every fixed $c_0 > 0$, when N is sufficiently large as a function of C, s, c_0 , with probability at least $1 - N^{-c_0}$,

$$\text{WED}_w(X, V) \leq U(X, V) \leq A_{\text{init}} \text{WED}_w(X, V)$$

simultaneously on $\mathcal{E}$. Every zero-distance entry returns zero on every outcome. Every returned value is a nonnegative integer at most $M(X) + M(V)$.

Conditional on the history before any entry's first adaptive request, its expected evaluation work, including finite binary computation and random sampling, is

$$(1 + |X|)2^{O_C(H/\chi + \chi^2)} = (1 + |X|)N^{o(1)},$$

uniformly over the entry and its target fragment. This bound includes inaccurate executions. Each entry evaluation terminates on every sequence of random bits, uses polynomial workspace, and performs finite binary arithmetic with $\text{poly}(\log N)$ work per operation. It reads only the completed deterministic shared data and its private state and does not request another entry.

Proof. If X is empty, its exact answer is $M(V)$. Otherwise the equality test examines corresponding symbols only when $|X| = |V|$ and costs $O(1 + |X|)$. It returns zero on every outcome when the strings are equal. For unequal strings, put $d = \text{WED}_w(X, V) \geq 1$.

On the simultaneous event of Lemma 6.4, the root estimate satisfies $T_o(0)/2 - k \leq \tilde{T}_o(0) \leq 2T_o(0) + k$. Every candidate satisfies

$$U_k \geq T_o(0) + \Delta \geq d,$$

by Lemma 6.1. The fixed guess set includes a power of two with $d \leq k < 2d$, since $d \leq M(X) + M(V)$. For this guess,

$$\begin{aligned} U_k &\leq 4T_o(0) + 4k + \Delta + 1 \\ &\leq (18 + 8BD_0)d \\ &\leq BH^4d = A_{\text{init}}d \end{aligned}$$

for sufficiently large H . Here Lemma 6.1 bounds $T_o(0)$, Lemma 5.1 gives $\Delta \leq d$, and $1 \leq d$. The trivial gap candidate $M(X) + M(V)$ is always at least d , so taking the minimum preserves the event's lower bound and gives the claimed upper bound. The simultaneous probability follows from Lemma 6.4.

On every outcome, the arrays are nonnegative and the answer is clamped by the trivial gap candidate. The output is a nonnegative integer at most $M(X) + M(V)$. The shared preprocessing has the bound stated at the start of the section. Lemma 6.5 gives finite execution and polynomial workspace. Lemma 6.6 and the private-stream argument give the conditional first-request work bound without conditioning on accuracy. Caching completed answers realizes the same random table for every adaptive request order. $\square$

7 The weighted refinement algorithm

One refinement pass takes a complete integer table U with values in $[0, R]$. Its accuracy analysis assumes that U is good. The algorithm never tests this assumption and applies every local guard. The mathematical tables have complete values before an adaptive evaluation schedule is chosen.

7.1 Global parameters and the physical source tree

Use the coarse-estimator parameter definitions with $X_0 = x$ and $Y_0 = y$, so $R_0 = R$. Reuse the parameters $H, \chi, p_*, B, A_{\text{init}}$ from the coarse estimator: H is the least power of two at least $\lceil \log_2(N + R + 2) \rceil$, $\chi = \log_2 H$, $p_* = \chi$, $B = H^{p_*}$, and $A_{\text{init}} = H^{p_*+4}$. No fragment request or refinement pass recomputes these parameters. We describe the branch $\chi \geq 2^{1000}$.

The parameter forms below follow the unit-cost refinement [6]. We use $p_* = \chi$ and $\ell = \Theta(\chi^3)$ to obtain the explicit overhead bound.

Definition 7.1. Let ℓ be the least power of two at least χ^3 . The source branching factor, factor reduction, warmup length, and refinement length are

$$M = 2^{\lfloor \chi/20 \rfloor}, \quad F = \ell, \quad T = 2^{\lceil 3\chi/4 \rceil}, \quad S = H/\ell^2.$$

For endpoint nets and whole-cell grouping, use

$$P = H^{p_*+20}, \quad \theta = \ell^{-40}, \quad L_0 = 32F, \quad L = 128F.$$

Here P sets the net resolution, θ sets the rounding and cell widths, and L_0, L are the cone slopes. The clipping slack, velocity margin, entropy parameters, sampling tolerance, and sample budget are

$$\delta = \ell^{-10}, \quad \tau = \kappa = \ell^{-4}, \quad \eta = \ell^5, \quad \gamma = \ell^{-20}, \quad Q_1 = \ell^{80}.$$

Finally, set $\varepsilon_{\text{leaf}} = \ell^{-3}$.

Build a complete M -ary source tree of physical height

$$J = \lceil \log_M \max(1, |x|) \rceil.$$

A node $I = (l, u]$ at positive physical height has cuts $l_i = l + \lfloor i(u - l)/M \rfloor$ and physical children $I_i = (l_{i-1}, l_i]$. Node labels distinguish repeated numeric intervals. Only nodes at physical height zero use the leaf rule. An internal node of width zero or one still uses the internal-node rule.

A state is a physical node I and an original target-boundary pair $q = (q^-, q^+)$ with $0 \leq q^- \leq q^+ \leq |y|$. Its true cost is $c(I, q) = \text{WED}_w(x[I], y[q])$. Prefix masses, z , the endpoint distance d , and connection charges use this fixed metric.

Lemma 7.2. *The parameters satisfy*

$$H = 2^\chi, \quad \chi^3 \leq \ell < 2\chi^3, \quad \ell^{200} \leq H^{1/100}, \quad p_* + 21 \leq \sqrt{\ell}, \quad p_*\chi \leq H/16.$$

Moreover,

$$J \leq 100H/\chi, \quad M^J \leq M \max(1, |x|), \quad \theta P \geq M, \quad 4MJ/P \leq \frac{1}{2}, \quad 64F\theta < 1.$$

Every physical leaf has source width zero or one. The number S is a positive integer, and $T = o(S)$. For every power of two $1 \leq A \leq A_{\text{init}}$ and $a = \max(1, A/F)$, we have $2a/P < 1$. There are fewer than e^{10H} states.

The clipping, entropy, and rounded-score error budgets satisfy

$$0 < \kappa < \frac{1}{2}, \quad 2\delta + 8L\theta \leq 3\delta \leq \kappa/\eta,$$

$$64F(6L + 2) + 2L \leq 512LF, \quad 2\gamma + 512LF\theta \leq \kappa\delta.$$

The label, group-failure, and action-coordinate bounds are

$$\log_2(64FM) + 2 \leq 2\ell, \quad 4\ell(P + 1)^2 e^{-\ell^{37}/3000} \leq H^{-4},$$

$$\ln(M(321P)^2) \leq 4(p_* + 21)\chi.$$

Proof. The definitions give the first two bounds. The third follows from $200 + 600 \log_2 \chi \leq \chi/100$. This inequality holds at the cutoff, and $\chi/100 - 600 \log_2 \chi$ increases beyond it. The remaining polynomial-versus-exponential comparisons follow from $H = 2^\chi$. In particular, $\chi - 2 \log_2 \ell$ is a nonnegative integer, so S is a positive integer. Also $T/S \leq 2\ell^2 H^{-1/4} \rightarrow 0$.

For the depth, $\log_2 M = \lfloor \chi/20 \rfloor \geq \chi/40$ and $\log_2 N \leq H$. The ceiling in J gives the displayed depth and leaf count. At physical depth j , each interval has width at most $\lceil |x|/M^j \rceil$: each split gives child width at most the ceiling of its parent width divided by M . Since $M^J \geq |x|$, leaf widths are at most one. The other comparisons follow from

$$2a/P \leq 2A_{\text{init}}/P = 2H^{-16}, \quad \frac{\theta P}{M} \geq \frac{H^{\chi+20-1/20}}{\ell^{40}}, \quad \frac{MJ}{P} \leq \frac{100H^{1+1/20}}{\chi P}.$$

The tree has fewer than $2M^J$ nodes and fewer than $(N + 1)^2$ target pairs per node. The preceding bounds imply the state-count claim.

For the error budgets, substitution reduces the two nontrivial checks to

$$1024\ell^{-39} \leq \ell^{-10}, \quad 2\ell^{-20} + 65536\ell^{-38} \leq \ell^{-14}.$$

Also $3\delta \leq \kappa/\eta$ follows from $3 \leq \ell$. The label and coordinate logarithms follow from $\log_2 M \leq \chi/20$, $\log_2 P = (p_* + 20)\chi$, and $\ell \geq \chi^3$. For the group-failure inequality, take logarithms. Its required bound is

$$\ln(4\ell) + 2\ln(P + 1) + 4\ln H \leq \ell^{37}/3000.$$

For $\chi \geq 2^{1000}$, the left side is at most $4\chi^2$, while the right side is at least $\chi^{111}/3000$. $\square$

7.2 The input table and leaf rule

Definition 7.3. A refinement invocation receives a complete integer table $U(I, q) \in [0, R]$ and a power of two $1 \leq A \leq A_{\text{init}}$. Put

$$a = \max(1, A/F), \quad \tilde{U}(I, q) = \max\{\lceil U(I, q)/a \rceil, |z(I, q)|\}.$$

The invocation has a *good input* if $c(I, q) \leq U(I, q) \leq Ac(I, q)$ at every state.

At a physical leaf use the generic leaf estimator of Lemma 5.6 with $k = 2\ell^3$, and denote it by $Q(I, q)$. Thus Q is integer-valued in $[0, R]$, is zero exactly at zero-cost states, and satisfies

$$c \leq Q \leq (1 + \varepsilon_{\text{leaf}})c, \quad |Q(I, q) - Q(I, q')| \leq 3d(q, q').$$

A supplied input table need not equal Q at a physical leaf.

For a good input, integrality of Fc gives $\max(c/a, |z|) \leq \tilde{U} \leq Fc$.

Definition 7.4. Let $\mathcal{D}_R = \{2^j : j \in \mathbb{Z}_{\geq 0}, 2^j \leq R\}$. Every scale range in the construction is intersected with $\mathcal{D}_R$, including on inaccurate inputs. For $b \in \mathcal{D}_R$, put

$$g_b = \max(1, b/(16P)), \quad r_b(q) = (\text{pred}_{g_b}(q^-), \text{pred}_{g_b}(q^+)).$$

A scale- b net pair is an ordered pair of boundaries in $\mathcal{N}_{g_b}$.

Lemma 7.5. For every state and every scale $b \in \mathcal{D}_R$, $d(q, r_b(q)) \leq 2b/P$.

Proof. If $g_b = 1$, rounding is exact. Otherwise Lemma 5.5 bounds each endpoint error by $2g_b = b/(8P)$. Their sum is at most the stated allowance. $\square$

Fix a core-run namespace ν , including the input and metric identities. An invocation has a pass label $p \geq 1$ and a copy label c_{copy} . Its local input U is the supplied complete table with seed label $p - 1$, denoted by $U^{(p-1)}$. Its factor A is fixed across the copies of this invocation. These labels identify values; they do not prescribe a procedure for constructing supplied seed tables. Within one invocation, we suppress ν, p, c_{copy} .

7.3 Initial cones and Bellman propagation

Definition 7.6. At a physical leaf set $w_0 = Q$. At an internal node and a query (I, q) , if $U(I, q) = 0$, set $w_0(I, q) = 0$. Otherwise enumerate scales

$$U(I, q)/(16A^2) \leq b \leq 4AU(I, q)$$

and scale- b net pairs r at mass sup-distance at most $U(I, q)$ from q . Retain exactly those satisfying $b/2 \leq U(I, r) \leq 4Ab$. Define $w_0(I, q)$ as the minimum of their cones

$$\tilde{U}(I, r) + L_0 d(q, r).$$

If no center is eligible, set $w_0(I, q) = R$.

Lemma 7.7. For a good input,

$$\max(c/a, |z|) \leq w_0 \leq 2Fc, \quad \text{Lip}(w_0) \leq L_0.$$

At an internal node, the local definition equals the global minimum of these cones over all eligible scale-net centers, together with all zero-seed centers. The local rule remains finite for every input table.

Proof. Lemma 5.3 makes every global cone dominate the two lower bounds. A leaf has these bounds and Lipschitz constant at most $3 \leq L_0$. For positive c , choose $b \leq c < 2b$ and $r = r_b(q)$. Lemma 7.5 gives $b/2 \leq c(I, r) \leq 4b$, so the center is eligible. Its cone is at most $Fc + (F + L_0)2b/P \leq 2Fc$.

The cone's nonnegative base cost puts a minimizing center at endpoint distance at most $2Fc/L_0 = c/16$. Its true cost lies in $[15c/16, 17c/16]$. A zero center cannot minimize at a positive-cost query, since its endpoint distance is at least c . Combining the cost interval with eligibility and $c \leq U(I, q) \leq Ac$ gives the stated local scale range. The local radius contains the minimizing center. At zero cost the query returns zero, which is also the global minimum. Global cone minima are L_0 -Lipschitz.

For every input, the tested scale range bounds the ratio of the radius to net spacing. Net separation gives $O((1 + PA^2)^2)$ raw centers per scale. The fixed scale set is finite, and an empty list has the explicit default R . $\square$

Definition 7.8. An eligible representative (b, r) is a scale- b net pair satisfying

$$b/(4a) \leq w_0(I, r) \leq 8Fb.$$

For a positive $w_0(I, q)$, its local scales satisfy

$$w_0(I, q)/(16F) \leq b \leq 8aw_0(I, q).$$

Partition the target endpoint-mass plane into half-open cells of side θb . The cell containing r has indices

$$e^\pm = \lfloor M_y(r^\pm)/(\theta b) \rfloor.$$

This floor acts on bounded rational integer-core data. Let $\mathcal{R}(I, q)$ contain all eligible representatives in every whole cell meeting the mass sup-square of radius $w_0(I, q)$ around q , for these local scales. It includes the rest of a meeting cell, even outside the square. If $w_0(I, q) = 0$, let $\mathcal{R}(I, q)$ be empty.

For a representative put $p_0 = M_y(r^-) - M_x(l)$. The wide list for physical child i consists of the ordered scale- b net states s_i satisfying

$$|M_y(s_i^-) - M_x(l_{i-1}) - p_0| \leq 10b, \quad |M_y(s_i^+) - M_x(l_i) - p_0| \leq 10b.$$

The wide action set $\mathcal{A}_{b,r}$ is the Cartesian product of these lists. Its child fragments may overlap or leave gaps. For a finite nonnegative child table f , put

$$V_r(\alpha; f) = \text{conn}_r(\alpha) + \sum_{i=1}^M f(I_i, s_i(\alpha)).$$

At an internal node define

$$\mathcal{B}_I(f)(q) = \min \left\{ w_0(I, q), \min_{(b,r) \in \mathcal{R}(I,q)} \left[Ld(q, r) + \min_{\alpha \in \mathcal{A}_{b,r}} V_r(\alpha; f) \right] \right\}.$$

An empty minimum is $+\infty$. At physical leaves, let $\mathcal{B}_I(f)(q) = Q(I, q)$ and set every w_j , m , and d_0 to Q . At internal nodes, set

$$w_j = \mathcal{B}(w_{j-1}) \quad (1 \leq j \leq T), \quad m = T^{-1} \sum_{j=0}^{T-1} w_j, \quad d_0 = w_T.$$

Lemma 7.9. Fix a good input table and an internal node I . For every parent state q and every eligible representative (b, r) ,

$$Ld(q, r) < w_0(I, q) \implies (b, r) \in \mathcal{R}(I, q).$$

Proof. Put $w_q = w_0(I, q)$ and $w_r = w_0(I, r)$. The hypothesis implies $w_q > 0$. By Lemma 7.7 and $L_0/L = 1/4$,

$$|w_r - w_q| \leq L_0 d(q, r) < w_q/4.$$

Eligibility and $3w_q/4 < w_r < 5w_q/4$ give

$$b > \frac{3w_q}{32F} > \frac{w_q}{16F}, \quad b < 5aw_q < 8aw_q.$$

These inequalities put b in the local scale range. Each endpoint-mass displacement is at most $d(q, r) < w_q/L \leq w_q$, so r lies in the query's mass square. Its cell meets that square, so the whole-cell rule includes the eligible representative (b, r) . $\square$

Lemma 7.10. *The Bellman operator is monotone in its nonnegative child table. For a good input, it is L -Lipschitz at the parent and preserves the lower bounds c/a and $|z|$ whenever the child table has them. For such an input, the warmup tables satisfy*

$$\max(c/a, |z|) \leq d_0 \leq m \leq w_0 \leq 2Fc, \quad \mathcal{B}(d_0) \leq d_0,$$

and all w_j and m are L -Lipschitz.

Proof. Fix a nonnegative child table. A Bellman cone that improves the fallback $w_0(I, q)$ has $Ld(q, r) < w_0(I, q)$, because its connection charge and child sum are nonnegative. Lemma 7.9 puts its center in the local representative list. A cone excluded from that list cannot improve the fallback. Thus the local rule equals the global minimum over all eligible representatives and wide actions, together with w_0 . This is a fixed family of L -Lipschitz functions, so its minimum is L -Lipschitz.

Monotonicity follows directly from the minima. The connection inequalities of Lemma 5.3 preserve the lower bounds. Start with $\mathcal{B}(w_0) \leq w_0$ and apply monotonicity to obtain $w_T \leq \dots \leq w_0$ and $\mathcal{B}(w_T) \leq w_T$. Averaging gives the stated bounds for m . Physical leaves retain Q throughout. $\square$

On arbitrary inputs, the local rule of Definition 7.8 is the definition. It always has the finite fallback $w_0(I, q)$.

7.4 Complete bands and rounded canonical actions

Definition 7.11. For an internal node, scale b , and candidate pair r , reject unless r is an ordered scale- b net pair and satisfies the eligibility test in Definition 7.8. For a passing candidate, put $Z = \min_{\alpha \in \mathcal{A}_{b,r}} V_r(\alpha; m)$. Reject its band if $Z \leq 0$ or $Z = +\infty$. Otherwise set $\hat{v} = z(I, r)/Z$ and reject unless $|\hat{v}| \leq 1 - \tau$.

For $1 \leq j < M$, define the prefix function

$$\begin{aligned} \Phi_j(p) = \min_{s_1, \dots, s_j} \bigg\{ & \sum_{i=1}^j m(I_i, s_i) + |M_y(s_1^-) - M_y(r^-)| \\ & + \sum_{i < j} |M_y(s_{i+1}^-) - M_y(s_i^+)| + |M_y(s_j^+) - (M_x(l_j) + p)| \bigg\}, \end{aligned}$$

where the minimum uses the first j wide child lists. This is a finite minimum of 1-Lipschitz functions of p .

The complete band $\mathcal{C}_{b,r}$ consists of shared-endpoint actions $s_i = (v_{i-1}, v_i)$ with $r^- = v_0 \leq \dots \leq v_M = r^+$ and each $v_j \in \mathcal{N}_{g_b}$. Every cut, including the two outside cuts, must satisfy $|M_y(v_j) - M_x(l_j) - p_0| \leq 6b$. At each internal cut put $p = M_y(v_j) - M_x(l_j)$ and require

$$|p - p_0 - \hat{v}\Phi_j(p)| \leq \tau\theta b/M.$$

Reject an empty complete band. The algorithm evaluates Φ_j only at the reported candidate shifts; it does not search a real interval.

For a nonempty band let

$$h_i = \text{ceil}_2 \left(\max \{ b/M, \max_{\alpha \in \mathcal{C}_{b,r}} w_0(I_i, s_i(\alpha)) \} \right), \quad h_\Sigma = \sum_i h_i.$$

Here $\text{ceil}_2(v)$ is the least power of two at least $v > 0$, allowing negative exponents. The maximum uses only states on complete band actions. Reject unless $h_\Sigma \leq 64Fb$.

Fix an action order once. Take its first action in $\mathcal{C}_{b,r}$ and round both endpoints downward separately at child i to $\mathcal{N}_{D_{h_i}}$, where $D_h = \max(1, \theta h/16)$. Call the resulting action $\beta(b, r)$. Reject unless

$$\beta(b, r) \in \mathcal{A}_{b,r}, \quad |z(I_i, s_i(\beta(b, r)))| \leq 2h_i \quad (1 \leq i \leq M).$$

A representative passing every test is retained. Its band record contains $\mathcal{C}_{b,r}$, all h_i , h_Σ , and $\beta(b, r)$.

The band constraints imply that two allowed shifts at a cut differ by at most $2\theta b/M$. Subtract their inequalities and use $|\hat{v}| \leq 1 - \tau$ and the Lipschitz bound for Φ_j . The physical cuts and original boundary indices remain part of every action, including at repeated numeric source cuts.

Lemma 7.2 and $h_i \geq b/M$ give $D_{h_i} \geq g_b$, so nesting preserves the parent-net membership. Each rounded endpoint moves by at most θh_i on every input. For a good input, the label bound gives $|z| \leq h_i$ before rounding. The rounded imbalance is then at most $h_i + 2\theta h_i \leq 2h_i$, and the shift radius is at most $6b + 64F\theta b < 10b$. Thus the final guards follow from the preceding ones on a good input.

Separate child rounding can make a formerly shared endpoint distinct. We charge each rounded action its full connection cost. A raw pair that cannot extend to a complete band action does not enter any label maximum. Rejection and missing-candidate sentinels occur only in auxiliary records, never in numerical tables.

7.5 Entropy under monotone costs with fixed block masses

For a probability vector define $\mathcal{H}(\rho) = -\sum_k \rho_k \ln \rho_k$, with $0 \ln 0 = 0$. The following inequality applies to a complete sequence of child costs.

Proposition 7.12. *Let $\mathcal{Q}$ be a nonempty compact convex set of probability vectors in $\mathbb{R}^d$. Suppose that $\mathcal{Q}$ contains a vector positive in every coordinate. Partition the coordinates into blocks and assume that every vector in $\mathcal{Q}$ has the same total mass in each block.*

Let $S \geq 1$ be an integer, let $\eta > 0$, and let $0 < \kappa < 1/2$. Let $f_0, \dots, f_S \in [0, 1]^d$ be coordinatewise nonincreasing. Put

$$\Delta_t = f_{t-1} - f_t, \quad \hat{g}_t = -f_{t-1}, \quad u_t = \hat{g}_t + (1 - 2\kappa)\Delta_t.$$

Suppose that the range of Δ_t within each block is at most κ/η . Let ρ_t maximize

$$\mathcal{H}(\rho) + \eta \langle \sum_{j < t} u_j + \hat{g}_t, \rho \rangle \quad \text{over } \rho \in \mathcal{Q}.$$

Then, for every $\rho^ \in \mathcal{Q}$ and every integer $0 \leq s \leq S$,*

$$\sum_{t=1}^s \langle f_t + \kappa \Delta_t, \rho_t \rangle \leq \sum_{t=1}^s \langle f_t, \rho^* \rangle + \frac{\ln d}{\eta} + 2\kappa.$$

If each left-hand vector ρ_t is replaced by a vector within $\zeta \geq 0$ in ℓ_1 distance, the right side increases by at most $s\zeta$.

Proof. Define the potential

$$\Psi(v) = \max_{\rho \in \mathcal{Q}} \{ \mathcal{H}(\rho) + \eta \langle v, \rho \rangle \}.$$

Compactness gives a maximizer. Strict concavity of entropy gives its uniqueness. To prove positivity, suppose a maximizing vector has a zero coordinate. Move distance $a > 0$ toward a positive feasible vector. The entropy contribution from zero coordinates increases by a positive multiple of $a \ln(1/a)$.

Contributions from previously positive coordinates and the linear objective change by $O(a)$. For sufficiently small a , the objective increases, contradicting optimality.

Let p be the maximizer for v . First-order optimality along every feasible segment gives

$$\langle \nabla \mathcal{H}(p) + \eta v, \rho - p \rangle \leq 0 \quad (\rho \in \mathcal{Q}).$$

For probability vectors,

$$\mathcal{H}(\rho) - \mathcal{H}(p) = \langle \nabla \mathcal{H}(p), \rho - p \rangle - D_{\text{KL}}(\rho \| p), \quad D_{\text{KL}}(\rho \| p) = \sum_k \rho_k \ln(\rho_k / p_k).$$

Consequently

$$\mathcal{H}(\rho) + \eta \langle v, \rho \rangle \leq \Psi(v) - D_{\text{KL}}(\rho \| p).$$

This argument works on the constrained hull $\mathcal{Q}$; it does not assume that its optimizer is an unconstrained softmax.

Put $v_{t-1} = \sum_{j < t} u_j$. Evaluating the old objective at the new maximizer ρ_t gives

$$\Psi(v_{t-1} + \hat{g}_t) - \Psi(v_{t-1}) \leq \eta \langle \hat{g}_t, \rho_t \rangle.$$

This remains valid when the increment has negative coordinates.

Let μ_k equal the minimum of Δ_t in the block containing k , and put $r = \Delta_t - \mu$. Then $\mu \geq 0$ and $0 \leq \eta r_k \leq \kappa$. Fixed block masses imply that $\langle \mu, \rho \rangle$ is the same for every $\rho \in \mathcal{Q}$. Applying the displayed KL penalty bound at $v_{t-1} + \hat{g}_t$, the potential increment produced by adding $(1 - 2\kappa)\Delta_t$ is at most

$$\eta(1 - 2\kappa) \langle \mu, \rho_t \rangle + \ln \left(\sum_k \rho_{t,k} e^{\eta(1-2\kappa)r_k} \right).$$

Indeed, after removing the constant block contribution, enlarge the KL-penalized optimization to the full probability simplex. If $Z = \sum_k \rho_{t,k} e^{\eta(1-2\kappa)r_k}$ and $q_k = \rho_{t,k} e^{\eta(1-2\kappa)r_k} / Z$, its objective is $\ln Z - D_{\text{KL}}(\rho \| q) \leq \ln Z$. Nonnegativity of KL follows, for example, from $\ln z \leq z - 1$, with zero coordinates treated by continuity.

For $0 \leq x \leq \kappa \leq 1/2$,

$$e^{(1-2\kappa)x} \leq 1 + (1 - \kappa)x.$$

To verify this, differentiate $\ln(1 + (1 - \kappa)x) - (1 - 2\kappa)x$. Its derivative is

$$\frac{\kappa - (1 - 2\kappa)(1 - \kappa)x}{1 + (1 - \kappa)x} \geq 0,$$

and the expression is zero at $x = 0$.

Apply the displayed scalar inequality with $x = \eta r_k$ and then use $\ln(1 + y) \leq y$. The bound for the remaining increment is at most

$$\eta(1 - 2\kappa) \langle \mu, \rho_t \rangle + \eta(1 - \kappa) \langle r, \rho_t \rangle \leq \eta(1 - \kappa) \langle \Delta_t, \rho_t \rangle,$$

where the final inequality uses $\mu \geq 0$. Combining this with the displayed prediction bound yields

$$\Psi(v_t) - \Psi(v_{t-1}) \leq \eta \langle \hat{g}_t + (1 - \kappa)\Delta_t, \rho_t \rangle = -\eta \langle f_t + \kappa\Delta_t, \rho_t \rangle.$$

Now $\Psi(0) \leq \ln d$, and $\mathcal{H}(\rho^*) \geq 0$. Also,

$$u_t = -f_t - 2\kappa\Delta_t, \quad 0 \leq \sum_{t=1}^s \Delta_t = f_0 - f_s \leq \mathbf{1}.$$

Summing the potential inequalities and evaluating the last potential at ρ^* gives

$$\begin{aligned}
\eta \sum_{t=1}^s \langle f_t + \kappa \Delta_t, \rho_t \rangle &\leq \Psi(0) - \Psi(v_s) \\
&\leq \ln d + \eta \sum_{t=1}^s \langle f_t, \rho^* \rangle + 2\eta\kappa \langle \sum_{t=1}^s \Delta_t, \rho^* \rangle \\
&\leq \ln d + \eta \sum_{t=1}^s \langle f_t, \rho^* \rangle + 2\eta\kappa.
\end{aligned}$$

Division by η proves the claimed inequality. The case $s = 0$ is immediate.

Finally, $f_t + \kappa \Delta_t = \kappa f_{t-1} + (1 - \kappa)f_t$ belongs to $[0, 1]^d$. Its inner product changes by at most ζ under an ℓ_1 perturbation of size ζ . Sum over the prefix. $\square$

7.6 Finite rational entropy optimization and exact sampling

Proposition 7.13. *Let $\mathcal{Q}$ be the convex hull of a nonempty finite list of rational probability vectors $a_1, \dots, a_L \in \mathbb{Q}^d$. Suppose that every coordinate is positive in at least one listed vector. Given $v \in \mathbb{Q}^d$ and an integer $h_{\text{opt}} \geq 4$, a finite rational algorithm constructs a mixture of the listed vectors whose mean ρ satisfies*

$$\|\rho - \rho^*\|_1 \leq h_{\text{opt}}^{-3}, \quad \rho^* = \operatorname{argmax}_{\rho \in \mathcal{Q}} \{\mathcal{H}(\rho) + \langle v, \rho \rangle\}.$$

The maximizer is unique. The construction cost in bit operations and the stored mixture size are polynomial in $G_0 = h_{\text{opt}} + d + 2$, L , and the maximum numerator or denominator bit length of an input rational. The returned mixture has dyadic probabilities and can be sampled exactly using a bounded number of fair random bits, with deterministic bit cost polynomial in the same parameters and a fixed space bound.

Proof. Averaging the input list gives a strictly positive feasible vector. The compactness, strict concavity, and positivity argument in the proof of Proposition 7.12 establishes existence, uniqueness, and positivity of ρ^* .

Set

$$\sigma = G_0^{-50}, \quad \varepsilon_{\text{grad}} = G_0^{-20}, \quad F(\rho) = \mathcal{H}(\rho) + \langle v, \rho \rangle, \quad F_\sigma(\rho) = - \sum_k (\rho_k + \sigma) \ln(\rho_k + \sigma) + \langle v, \rho \rangle.$$

For $x \in [0, 1]$, the increment $-(x + \sigma) \ln(x + \sigma) + x \ln x$ decreases with x . Its endpoint magnitudes are at most $\sigma \ln(1/\sigma)$ and $(1 + \sigma) \ln(1 + \sigma)$. Both are at most $4\sqrt{\sigma}$ for $0 < \sigma \leq 1$. Summing these coordinate bounds gives

$$\sup_{\rho \in \mathcal{Q}} |F_\sigma(\rho) - F(\rho)| \leq 4d\sqrt{\sigma}.$$

The Hessian of F_σ is diagonal with entries $-1/(\rho_k + \sigma)$, so its gradient is $1/\sigma$ -Lipschitz in Euclidean norm. The Euclidean diameter of $\mathcal{Q}$ is at most 2.

Start at any listed vertex $\rho^{(0)}$. At iteration $j \geq 0$, compute each gradient coordinate to absolute error at most $\varepsilon_{\text{grad}}$. Enumerate the list and choose a vertex v_j maximizing its inner product with the approximate gradient, resolving ties in a fixed order. Set

$$\xi_j = \frac{2}{j+2}, \quad \rho^{(j+1)} = (1 - \xi_j)\rho^{(j)} + \xi_j v_j.$$

The ℓ_1 distance between two probability vectors is at most 2. Thus v_j loses at most $2\varepsilon_{\text{grad}}$ in the true linearized objective relative to any feasible mixture.

Write $D_j = \max_{\mathcal{Q}} F_{\sigma} - F_{\sigma}(\rho^{(j)})$. Concavity and the Lipschitz gradient bound give

$$D_{j+1} \leq (1 - \xi_j)D_j + 2\varepsilon_{\text{grad}}\xi_j + \frac{2\xi_j^2}{\sigma}.$$

In particular, $\xi_0 = 1$ gives $D_1 \leq 2\varepsilon_{\text{grad}} + 2/\sigma$, independently of the magnitude of the rational vector v . Induction in the displayed recurrence yields

$$D_j \leq 2\varepsilon_{\text{grad}} + \frac{8}{\sigma(j+1)} \quad (j \geq 1).$$

For the induction step, the required scalar inequality is $j/(j+1) + 1/(j+2) \leq 1$.

Run $K_{\text{opt}} = G_0^{80}$ steps to obtain the preliminary mixture mean $\rho^{(K_{\text{opt}})}$. Using the uniform smoothing bound twice bounds its true objective gap by

$$F(\rho^*) - F(\rho^{(K_{\text{opt}})}) \leq 2G_0^{-20} + 8G_0^{-30} + 8dG_0^{-25} \leq 12G_0^{-20}.$$

The Hessian of $-\mathcal{H}$ at a positive probability vector has entries $1/\rho_k \geq 1$. Combining this bound with constrained first-order optimality at the positive maximizer gives

$$F(\rho^*) - F(\rho) \geq \frac{1}{2} \|\rho - \rho^*\|_2^2 \quad (\rho \in \mathcal{Q}).$$

For an endpoint with a zero coordinate this follows by continuity along the segment from ρ^* . Hence

$$\left\| \rho^{(K_{\text{opt}})} - \rho^* \right\|_1 \leq \sqrt{d} \left\| \rho^{(K_{\text{opt}})} - \rho^* \right\|_2 \leq \sqrt{24d} G_0^{-10} \leq \frac{1}{2} h_{\text{opt}}^{-3}.$$

For the last inequality use $d \leq G_0$, $h_{\text{opt}} \leq G_0$, and $G_0 \geq 7$: $\sqrt{24d} G_0^{-10} h_{\text{opt}}^3 \leq \sqrt{24} G_0^{-13/2} < 1/2$. The remaining half of the error allowance will cover dyadic rounding.

We now implement every step with finite rational arithmetic. After $K'_{\text{opt}} \geq 1$ iterations the weight of the vertex selected at iteration j , where $0 \leq j < K'_{\text{opt}}$, is

$$\frac{2}{j+2} \prod_{k=j+1}^{K'_{\text{opt}}-1} \frac{k}{k+2} = \frac{2(j+1)}{K'_{\text{opt}}(K'_{\text{opt}}+1)}.$$

The initial vertex has zero remaining weight because $\xi_0 = 1$. The displayed weights sum to one. Store the sequence of selected vertex indices, allowing repetitions.

Let D be the product of all denominators in the input vertex list. Its bit length is at most the sum of their bit lengths, hence polynomial in the input parameters. All coordinates of $\rho^{(K'_{\text{opt}})}$ have common denominator $DK'_{\text{opt}}(K'_{\text{opt}}+1)$. They can be computed from the cumulative weighted sum $\sum_{j < K'_{\text{opt}}} (j+1)v_j$ and this common denominator. This representation avoids multiplying denominators through successive iterations. Its bit length is polynomial in the stated parameters.

It remains to approximate the gradient, whose k th coordinate is $v_k - 1 - \ln(\rho_k + \sigma)$. For rational $r = \rho_k + \sigma \in [\sigma, 1 + \sigma]$, set

$$b_0 = \frac{r-1}{r+1}.$$

Since $\sigma = G_0^{-50} < 1/4$, we have $|b_0| \leq 1 - \sigma$ and $1 - b_0^2 \geq \sigma$. The identity

$$\ln r = 2 \sum_{j=0}^{\infty} \frac{b_0^{2j+1}}{2j+1}$$

follows by integrating the geometric series for $2/(1 - b_0^2)$ from zero to b_0 . Truncate after $K_{\log} = G_0^{60}$ terms. The absolute tail is at most

$$\frac{2|b_0|^{2K_{\log}+1}}{(2K_{\log}+1)(1-b_0^2)} \leq \frac{2e^{-2K_{\log}\sigma}}{(2K_{\log}+1)\sigma} < G_0^{-20} = \varepsilon_{\text{grad}}.$$

The final inequality follows from $K_{\log}\sigma = G_0^{10}$ and $G_0 \geq 7$.

Every summand is rational. The numerator and denominator lengths of powers of b_0 grow at most linearly in $K_{\log}$ times the bit length of b_0 . Multiplying the odd denominators contributes $O(K_{\log} \log K_{\log})$ bits. Direct integer arithmetic computes the truncated sum with polynomial bit cost. The rational linear term v_k and the constant -1 are added exactly. Enumerating the input list and comparing rational scores now implements the vertex choice. Repeating for G_0^{80} iterations still has a polynomial bit bound, with absolute constant exponents. The approximate logarithms affect only the selected vertex; they are not inserted into the exact mixture-coordinate representation.

Finally, let $\omega_j = 2(j+1)/(K_{\text{opt}}(K_{\text{opt}}+1))$ for $0 \leq j < K_{\text{opt}}$, and choose the least integer b_{samp} with $2^{b_{\text{samp}}} \geq 4K_{\text{opt}}h_{\text{opt}}^3$. Compute $q_j = \lfloor 2^{b_{\text{samp}}}\omega_j \rfloor$ by integer division. Add the integer residual $2^{b_{\text{samp}}} - \sum_j q_j$ to q_0 . The probabilities $\omega'_j = q_j/2^{b_{\text{samp}}}$ are nonnegative and sum to one. If $e_j = \omega_j - \lfloor 2^{b_{\text{samp}}}\omega_j \rfloor/2^{b_{\text{samp}}}$, their total change is

$$\sum_j |\omega'_j - \omega_j| = 2 \sum_{j>0} e_j < 2K_{\text{opt}}2^{-b_{\text{samp}}} \leq \frac{1}{2}h_{\text{opt}}^{-3}.$$

Each action vertex is a probability vector of ℓ_1 norm one. Consequently the new mixture mean $\rho = \sum_j \omega'_j v_j$ differs from $\rho^{(K_{\text{opt}})}$ in ℓ_1 norm by at most $h_{\text{opt}}^{-3}/2$. Together with the preceding optimization bound this proves $\|\rho - \rho^*\|_1 \leq h_{\text{opt}}^{-3}$. The new mean stays in $\mathcal{Q}$ and preserves every block mass shared by the action vertices.

Store integer prefix sums of the q_j . Draw a uniform integer $J_{\text{samp}} \in \{0, \dots, 2^{b_{\text{samp}}} - 1\}$ using exactly b_{samp} independent fair bits. Binary search for the first prefix sum strictly larger than J_{samp} , and return its vertex. This samples the returned dyadic mixture exactly; zero weights and repeated vertices cause no difficulty. Here $b_{\text{samp}} = O(\log K_{\text{opt}} + \log h_{\text{opt}})$, and construction, storage, and sampling have deterministic polynomial bit cost. There is no rejection loop. In particular, this implementation terminates for every fair-bit sequence. $\square$

7.7 Group choices, online choices, and the clipped update

Fix a pass, a copy, and its complete input U . Give every potential group choice (I, b, e, t) and every potential online choice (I, b, q, t) a separate private stream of independent fair bits. These streams are mutually independent and independent of U .

Definition 7.14. For a node, scale b , whole cell e , and index $1 \leq t \leq S$, let $\mathcal{R}_e$ be all retained representatives in the cell. If it is empty, the group has no candidate. Otherwise process dyadic labels $h \in [b/M, 64Fb]$ in increasing order, setting

$$k_h = \min\{M, \lceil Q_1 M h / b \rceil\}.$$

If $k_h = M$, use the list $J_h = (1, \dots, M)$. Otherwise draw k_h independent uniform child labels, with replacement, from the group's private stream. Use the same list for every representative in the whole cell. Write $h_i(r)$ for the label in r 's band record and define

$$\sigma_t(r) = \text{conn}_r(\beta(b, r)) + \sum_h \frac{M}{k_h} \sum_{i \in J_h: h_i(r)=h} d_t(I_i, s_i(\beta(b, r))).$$

The inner sum preserves sampled multiplicities. Choose the first representative r_* of minimum score in the fixed representative order, and return its actual cost $C_* = V_{r_*}(\beta(b, r_*); d_t)$, together with r_* . The returned cost is not the sampled score. In the exact and sampled sums, test $h_i(r) = h$ before requesting the corresponding scalar value. A failing test creates no current-index child request.

For a retained band at r , introduce one coordinate (i, s) for each child state on a complete action. Map action α to the rational probability vector $\rho(\alpha)$ with mass h_i/h_Σ at $(i, s_i(\alpha))$, and let $\mathcal{Q}$ be their convex hull. Every vector has fixed mass h_i/h_Σ in child block i . Set

$$\hat{g}_j(i, s) = -d_{j-1}(I_i, s)/h_i, \quad \Delta_j(i, s) = (d_{j-1}(I_i, s) - d_j(I_i, s))/h_i, \quad u_j = \hat{g}_j + (1 - 2\kappa)\Delta_j.$$

At index t , use Proposition 7.13 with $h_{\text{opt}} = H$ on the action list and the exact rational linear vector

$$v_t = \eta \left(\sum_{j=1}^{t-1} u_j + \hat{g}_t \right).$$

Its returned dyadic mixture has mean within H^{-3} in ℓ_1 distance of the unique maximizer of

$$\mathcal{H}(\rho) + \eta \left\langle \sum_{j=1}^{t-1} u_j + \hat{g}_t, \rho \right\rangle \quad \text{over } \rho \in \mathcal{Q}.$$

The online owner (I, b, q, t) uses $r = r_b(q)$. If its band is rejected, there is no candidate. Otherwise sample one whole action α_t exactly from this mixture using that owner's private stream. Return its actual current child sum $C = \sum_i d_t(I_i, s_i(\alpha_t))$ and retain the sampled action. The action has shared endpoints and outside endpoints r , so its connection charge is zero.

At a nonleaf, let $\Pi_t(I, q)$ be the minimum of $\mathcal{B}_I(d_{t-1})(q)$ and the selected actual group costs $C_* + \text{Ld}(q, r_*)$ for all distinct groups reached by $\mathcal{R}(I, q)$. Let $Y_t(I, q)$ be the minimum of $C + \text{Ld}(q, r_b(q))$ over the online candidates at the local scales of Definition 7.8. An empty online minimum is $+\infty$. Define

$$d_t(I, q) = \begin{cases} 0, & w_0(I, q) = 0, \\ \min\{\Pi_t(I, q), \max\{\Pi_t(I, q)/(1 + \delta), Y_t(I, q)\}\}, & w_0(I, q) > 0. \end{cases}$$

At physical leaves set $d_t = Q$ for every t .

Every online coordinate appears in a complete band action. Averaging the action list gives a positive feasible vector, as required by the finite optimizer. Its coefficients use only child indices $0, \dots, t-1$. Numerical iterations reuse their finite arrays; only the single sampled action requests current-index child values. The Bellman candidate makes Π_t finite, so the clipped update is a finite rational even when no online candidate is available. Do not clamp refinement values to R within a pass.

Definition 7.15. All source-node fields are physical labels. All target-index fields are original indices in $[0, |y|]$. Scales belong to $\mathcal{D}_R$ and indices t satisfy $1 \leq t \leq S$. Group keys use the cell indices from Definition 7.8. Their cells intersect the target endpoint rectangle $[0, M_y(|y|)]^2$. Leaf keys require physical height zero. Band, group, and online keys require an internal physical node. Use fixed deterministic orders for all finite enumerations. The full keys are

Leaf. (leaf, ν, I, q) specifies $Q(I, q)$. It has no pass or copy label.

Seed. $(\text{seed}, \nu, p, I, q)$ specifies $U^{(p)}(I, q)$. It names a supplied complete integer table in $[0, R]$. When the supplied table is the coarse table, this key identifies its corresponding standalone initial entry.

Warmup. $(\text{warmup}, \nu, p, c_{\text{copy}}, j, I, q)$ specifies w_j for $0 \leq j \leq T$.

Mean. $(\text{mean}, \nu, p, c_{\text{copy}}, I, q)$ specifies m .

Refinement. $(\text{refinement}, \nu, p, c_{\text{copy}}, t, I, q)$ specifies d_t for $1 \leq t \leq S$. The index-zero key aliases the warmup key at index T .

Band. $(\text{band}, \nu, p, c_{\text{copy}}, I, b, r)$ specifies rejection or a complete retained-band record.

Group. $(\text{group}, \nu, p, c_{\text{copy}}, I, b, e^-, e^+, t)$ specifies the label samples, winning representative, and actual cost for the whole cell. It omits the requesting state.

Online. $(\text{online}, \nu, p, c_{\text{copy}}, I, b, q, t)$ specifies rejection or the sampled whole action and its actual child cost. It includes the queried state.

The initial entries, group records, and online records are random owners. Assign mutually independent fair-bit streams to every potential owner, including owners that no evaluation reaches. Shared preprocessing for the concrete coarse estimator is deterministic. Different core runs, passes, and copies have disjoint owner namespaces.

These streams define the mathematical random experiment. An implementation can allocate bits when an owner is first reached and retain its decisions. An unreached stream need not be stored. Initial entries use only their own streams, identities, workspace, and completed read-only preprocessing. They request no other initial entry. These properties are part of the standalone coarse construction.

A scalar child key omits its caller, representative, group cell, sampled position, and label h . These data describe requests and owners, not new child values. Warmup, mean, and band keys retain the copy label even when they are deterministic functions of the same input U . A displayed $\text{key}_f(I, q)$ below abbreviates the corresponding full key.

Lemma 7.16. *Fix a good input table, an internal node I , and an index t . Fix all complete child tables and the parent group streams, with nonnegative child values. Let $\mathcal{G}_t$ contain every potential whole-cell group at I that returns a candidate, and write (r_g, C_g) for that group's winning representative and actual cost. Let b_g be its scale. For every parent state q ,*

$$\Pi_t(I, q) = \min \left\{ \mathcal{B}_I(d_{t-1})(q), \min_{g \in \mathcal{G}_t} \{C_g + Ld(q, r_g)\} \right\}.$$

Consequently, $\Pi_t(I, \cdot)$ is L -Lipschitz.

Proof. The group definition uses all retained representatives in its whole cell and its own stream. Its canonical key omits the requesting state. Once the child tables and streams are fixed, the payload (r_g, C_g) is the same for every requester. Thus the displayed global cone family does not depend on q . Each r_g is eligible and $C_g \geq 0$.

The global minimum is at most the local minimum that defines Π_t . Conversely, consider a group cone below the Bellman fallback at q . That fallback is at most $w_0(I, q)$, so $Ld(q, r_g) < w_0(I, q)$. Lemma 7.9 puts (b_g, r_g) in $\mathcal{R}(I, q)$. The local rule reaches its whole-cell group and includes this same payload. Every omitted group cone is at least the fallback. The two minima agree.

Lemma 7.10 makes the Bellman fallback L -Lipschitz. Each selected group cone is also L -Lipschitz. Their global minimum has the same Lipschitz bound, including at states outside the comparison tree. $\square$

Lemma 7.17. *For a good input, every outcome satisfies*

$$\max(c/a, |z|) \leq d_t \leq d_{t-1} \leq w_0,$$

$$d_t(I, q) \leq (1 + \delta)d_t(I, q') + Ld(q, q'), \quad \mathcal{B}(d_t) \leq d_t \leq \mathcal{B}(d_{t-1}) \quad \text{at nonleaves.}$$

The spatial bound for d_0 holds without the factor $1 + \delta$.

Proof. Induct first on physical height and then on t . Every extra candidate is a current-child wide action with an endpoint cone. Earlier child values dominate current ones. Thus both Π_t and Y_t dominate $\mathcal{B}(d_t)$, while $\Pi_t \leq \mathcal{B}(d_{t-1})$. The clipped update of Definition 7.14 gives the Bellman inequalities. Monotonicity follows from $\mathcal{B}(d_0) \leq d_0$ at $t = 1$ and from the preceding lower Bellman inequality thereafter. The connection inequalities of Lemma 5.3 preserve the lower bounds.

Lemma 7.16 makes Π_t L -Lipschitz at every parent state. The bounds $\Pi_t/(1 + \delta) \leq d_t \leq \Pi_t$ imply the displayed spatial inequality. At leaves all assertions follow from the guarantee for Q in Definition 7.3, since $3 \leq L$. $\square$

7.8 Functional table requests

Definition 7.18. Canonicalize a local U key to its supplied seed-table key and a d_0 key to the warmup key at index T . The functional operation $\text{GET}(K)$ returns the complete mathematical payload specified by the definitions above for the resulting full key. For a finite list $\mathcal{L}$ of scalar keys, $\text{GATHER}(\mathcal{L})$ canonicalizes, sorts, and deduplicates them, and returns the local map from each distinct canonical key to its GET value. The scalar payload of an online record is its actual child-cost sum; the sampled action is retained as that owner's decision. The group payload is its winning representative and actual cost. An auxiliary rejection or missing-candidate sentinel is not a numerical value.

These operations specify mathematical reads. Memoization can implement them without changing values or owner decisions. After a gather, a displayed table value in the code is a read from this local map; it does not request the evaluator again. Repeated sampled occurrences retain their multiplicity in arithmetic even when they share one canonical scalar value. The call $\text{LEAF}(I, q)$ reads the shared $Q(I, q)$ value with key (leaf, ν, I, q) .

7.9 Exact procedural realization

The routines below realize the complete definitions. Their inputs include the fixed pass, copy, metric, and global parameters. Each state, wide list, prefix minimum, and action retains the physical child label and original target indices. The Bellman helper is a finite local computation, not another mathematical table. Prefix minima, gathered arrays, representative lists, scores, and optimizer iterates are scratch data. The full scalar keys in Definition 7.15 fix every recursive value independently of its requester.

Algorithm 3 Warmup entries and local representatives

Require: A fixed pass and copy, complete input U , physical node I , and state q ; $0 \leq j \leq T$ for WARMUP, and internal I for LOCAL.

Ensure: The specified warmup value or the complete local representative list with whole-cell identifiers.

```
1: procedure WARMUP( $j, I, q$ )
2:   if  $I$  is a physical leaf then
3:     return LEAF( $I, q$ )
4:   end if
5:   if  $j > 0$  then
6:     return BELL( $w_{j-1}, I, q$ )
7:   end if
8:    $s \leftarrow \text{GET}(\text{key}_U(I, q))$ .
9:   if  $s = 0$  then
10:    return 0
11:  end if
12:  Enumerate the initial scales and radius- $s$  centers of Definition 7.6.
13:  Gather their  $U$  keys, then retain the eligible centers.
14:  if no center is eligible then
15:    return  $R$ 
16:  end if
17:  return the minimum initial cone, using the gathered values.
18: end procedure

19: procedure LOCAL( $I, q$ )
20:    $w_{0,q} \leftarrow \text{GET}(\text{key}_{w_0}(I, q))$ .
21:   if  $w_{0,q} = 0$  then
22:     return the empty list
23:   end if
24:   Initialize an empty representative list.
25:   for each local scale  $b$  of Definition 7.8 do
26:     Report all whole cells meeting the radius- $w_{0,q}$  mass square around  $q$ .
27:     Enumerate the ordered scale- $b$  net pairs  $\mathcal{R}_b$  in these cells.
28:     Gather  $\text{key}_{w_0}(I, r)$  for every  $r \in \mathcal{R}_b$ .
29:     Retain exactly the eligible  $(b, r)$ , with their whole-cell identifiers.
30:   end for
31:   return the representative list.
32: end procedure
```

Algorithm 4 Bellman propagation and the warmup mean

Require: The fixed pass and copy, node I , and state q ; for BELL, internal I and a specified finite nonnegative child table f .

Ensure: The finite local Bellman value or the specified warmup mean.

```
1: procedure BELL( $f, I, q$ )
2:    $v \leftarrow \text{GET}(\text{key}_{w_0}(I, q))$ .
3:   if  $v = 0$  then
4:     return 0
5:   end if
6:    $\mathcal{R} \leftarrow \text{LOCAL}(I, q)$ ; generate its wide child-state lists.
7:   Gather the union of required keys  $\text{key}_f(I_i, s)$ .
8:   Evaluate the local Bellman minimum using  $v$  and the gathered array.
9:   return this minimum, with fallback  $v$ .
10: end procedure

11: procedure MEAN( $I, q$ )
12:   Gather  $\text{key}_{w_j}(I, q)$  for  $j = 0, \dots, T - 1$ .
13:   return their arithmetic mean.
14: end procedure
```

Algorithm 5 A complete retained band

Require: Internal node I , scale b , candidate pair r , and the fixed pass and copy.

Ensure: Rejection or the complete band, child labels, and rounded canonical action.

```
1: procedure BAND( $I, b, r$ )
2:   if  $r$  is not an ordered scale- $b$  net pair then
3:     return rejection
4:   end if
5:    $w_r \leftarrow \text{GET}(\text{key}_{w_0}(I, r)).$ 
6:   if  $w_r < b/(4a)$  or  $w_r > 8Fb$  then
7:     return rejection
8:   end if
9:   Generate all wide child-state lists and gather their mean keys.
10:  Compute  $Z$  from Definition 7.11, using those means.
11:  if  $Z \leq 0$  or  $Z = +\infty$  then
12:    return rejection
13:  end if
14:  Compute  $\hat{v}$  and  $p_0$  from that Definition.
15:  if  $|\hat{v}| > 1 - \tau$  then
16:    return rejection
17:  end if
18:  Report radius- $6b$  net candidates at every internal cut.
19:  if an outside endpoint fails the radius- $6b$  test then
20:    return rejection
21:  end if
22:  Compute the finite prefix minima only at reported shifts, using the same means.
23:  Construct  $\mathcal{C}_{b,r}$  with all ordering, endpoint, radius, and band constraints of Definition 7.11.
24:  if  $\mathcal{C}_{b,r} = \emptyset$  then
25:    return rejection
26:  end if
27:  Gather child  $w_0$  only at states occurring on complete actions in  $\mathcal{C}_{b,r}$ .
28:  Compute  $(h_i)_i$  and  $h_\Sigma$  from Definition 7.11.
29:  if  $h_\Sigma > 64Fb$  then
30:    return rejection
31:  end if
32:  Take the first complete action in the fixed order and round both endpoints separately as specified there.
33:  if  $\beta(b, r) \notin \mathcal{A}_{b,r}$  or some  $|z(I_i, s_i(\beta(b, r)))| > 2h_i$  then
34:    return rejection
35:  end if
36:  return  $(\mathcal{C}_{b,r}, (h_i)_i, h_\Sigma, \beta(b, r)).$ 
37: end procedure
```

Algorithm 6 One whole-cell group choice

Require: Internal node I , scale b , whole cell e , index $1 \leq t \leq S$, and this group's private stream.

Ensure: No candidate, or the first minimum-score representative and its actual current-index cost.

```
1: procedure GROUP( $I, b, e, t$ )
2:   Enumerate all ordered scale- $b$  net pairs in the whole cell.
3:   Obtain each GET(keyband( $I, b, r$ )); let  $\mathcal{R}_e$  contain all passing representatives.
4:   if  $\mathcal{R}_e = \emptyset$  then
5:     return no candidate
6:   end if
7:   for each dyadic label  $h \in [b/M, 64Fb]$ , in increasing order do
8:     Compute  $k_h$  from Definition 7.14.
9:     if  $k_h < M$  then
10:      Draw  $J_h$  as  $k_h$  independent uniform child labels with replacement from this group's stream.
11:    else
12:       $J_h \leftarrow (1, \dots, M)$ .
13:    end if
14:    Use the same  $J_h$  for every representative in  $\mathcal{R}_e$ .
15:  end for
16:  Initialize an empty scalar-key set  $\mathcal{K}$ .
17:  for each  $r \in \mathcal{R}_e$ , label  $h$ , and occurrence  $i$  in  $J_h$  do
18:    if  $h_i(r) = h$  then
19:      Add key $d_t$ ( $I_i, s_i(\beta(b, r))$ ) to  $\mathcal{K}$ .
20:    end if
21:  end for
22:   $\text{val} \leftarrow \text{GATHER}(\mathcal{K})$ .
23:  Compute every score  $\sigma_t(r)$  of Definition 7.14 from  $\text{val}$ , retaining sample multiplicities and the connection charge.
24:  Choose the first representative  $r_*$  of minimum score.
25:  Gather all  $M$  current child keys of  $\beta(b, r_*)$ .
26:  Retain the lists  $J_h$  and the winning representative as the owner's decisions.
27:  return  $r_*$  and its actual cost from Definition 7.14, using the gathered values.
28: end procedure
```

Algorithm 7 An online action from earlier child indices

Require: Internal node I , local scale b , query q , index $1 \leq t \leq S$, and this online owner's private stream.

Ensure: No candidate, or one sampled whole action's actual current child-cost sum.

```
1: procedure ONLINE( $I, b, q, t$ )
2:    $r \leftarrow r_b(q)$ ;  $\text{band} \leftarrow \text{GET}(\text{key}_{\text{band}}(I, b, r))$ .
3:   if  $\text{band}$  is rejected then
4:     return no candidate
5:   end if
6:   List the distinct coordinates  $(i, s)$  occurring on complete band actions.
7:   Gather key $d_j$ ( $I_i, s$ ) at these coordinates for  $j = 0, \dots, t-1$ .
8:   Form the action vectors and  $v_t$  from Definition 7.14.
9:   Run the finite optimizer of Proposition 7.13 with  $h_{\text{opt}} = H$ , using only the gathered arrays.
10:  Draw one whole action  $\alpha_t$  exactly from its returned dyadic mixture, using this owner's stream.
11:  Gather key $d_t$ ( $I_i, s_i(\alpha_t)$ ) for  $i = 1, \dots, M$ .
12:  Retain  $\alpha_t$  as the owner's decision.
13:  return the action's actual child-cost sum.
14: end procedure
```

Algorithm 8 The clipped refinement update

Require: Fixed pass and copy, node I , state q , and $1 \leq t \leq S$; d_0 aliases w_T .

Ensure: The specified finite value $d_t(I, q)$, without an internal clamp to R .

```
1: procedure REFINE( $t, I, q$ )
2:   if  $I$  is a physical leaf then
3:     return LEAF( $I, q$ )
4:   end if
5:    $w_{0,q} \leftarrow \text{GET}(\text{key}_{w_0}(I, q))$ .
6:   if  $w_{0,q} = 0$  then
7:     return 0
8:   end if
9:   Compute the Bellman fallback with BELL( $d_{t-1}, I, q$ ).
10:   $\mathcal{R} \leftarrow \text{LOCAL}(I, q)$ .
11:  for each distinct whole-cell group  $(b, e)$  reached by  $\mathcal{R}$  do
12:    Obtain GET( $\text{key}_{\text{group}}(I, b, e, t)$ ).
13:    Retain its representative and actual cost if it has a candidate.
14:  end for
15:  for each local scale  $b$  of Definition 7.8 do
16:    Obtain GET( $\text{key}_{\text{online}}(I, b, q, t)$ ).
17:    Retain  $(b, C)$  if it returns an actual cost  $C$ .
18:  end for
19:  Compute  $\Pi_t$  from the Bellman fallback and retained group cones, as in Definition 7.14.
20:  Compute  $Y_t$  from retained online cones, taking  $+\infty$  when none exists.
21:  return the clipped update of  $\Pi_t$  and  $Y_t$  in Definition 7.14.
22: end procedure
```

The initial cones use exactly the scale, radius, and eligibility rules in Definition 7.6. For local representatives, enumerate the whole meeting cell before applying eligibility. Gather the w_0 values at all reported centers before that test. The whole-cell identity makes a group independent of the state that first reaches it.

The band routine applies every rejection test in Definition 7.11 on every input. It gathers label values only for child states that lie on complete band actions. After separate rounding, the selected group action keeps its full connection charge.

Each sampled group label uses exactly $\log_2 M$ fair bits, since M is a power of two. Repeated samples keep their multiplicity in the score but share the same gathered scalar key. The label test precedes key creation. The winner's actual current cost is evaluated after selection. The online optimizer uses only gathered earlier-index values during its iterations. The only current-index calls from this owner are the M child values of its single sampled action.

The refinement update includes the Bellman fallback and has no internal clamp to R . Repeated table reads use the same full key and owner decisions.

Lemma 7.19. *Fix one supplied complete input table U , an invocation namespace, and all its potential owner streams. The rules in this section determine a unique complete scalar value and auxiliary payload at every key of that invocation. Complete child histories depend only on U and current-copy owner streams at that child and its descendants. Conditional on U and these histories, current parent group and online streams retain their joint independent fair-bit laws.*

Proof. Order records lexicographically by within-pass stage, remaining physical height, and phase. Supplied seed values and shared leaf values are base values at stage zero. Warmup w_j has stage $1 + j$, a mean has stage $T + 1$, a band has stage $T + 2$, and refinement, group, and online records at index t have stage $T + 1 + t$. The alias $d_0 = w_T$ has stage $T + 1$. Seed, warmup, and refinement records have phase 2, group and online records phase 1, and mean, band, and leaf records phase 0.

At a physical leaf, a warmup or refinement entry reads only its shared leaf value. At an internal node, w_0 reads the supplied seed at its query and at every reported initial center. The local representative helper reads same-node w_0 values at the query and every reported center before applying eligibility. For $j > 0$, a warmup entry uses this helper and reads earlier-index warmup values at physical children. A mean reads same-node warmup indices below T . A band reads same-node w_0 , child means, and child w_0 values on complete band actions.

A scalar refinement entry reads same-node w_0 , the local helper's center values, and earlier-index child values for its Bellman fallback. It also reads same-node group and online records with smaller phase. These records read same-node bands at smaller stage, or at smaller phase when $t = 1$. An online record reads earlier child histories to construct its distribution. A group or online record reads current-index child values only for its tested score terms or its selected action. These current-index reads decrease physical height, even when numerical source intervals repeat. Thus every recursive read decreases the order.

All remaining operations are finite minima, rational calculations, finite action enumeration, or the finite optimizer and bounded sampler proved above. Induction on the finite order defines each value uniquely and excludes a return to an unfinished caller, regardless of input accuracy.

The same induction shows that a complete child table uses only its input U and owner streams in its physical subtree. It excludes all current parent owners. Those streams are jointly independent of U and of the descendant streams, by Definition 7.15. Conditioning on complete child histories, which are functions of these other data, preserves their joint independent laws. $\square$

A complete child history includes all mathematical states and all indices at that child, whether evaluated or not. The realized reached-key set or request trace can reveal a parent sample and

cannot replace these complete histories in the conditioning of Lemma 7.19.

8 Accuracy of weighted refinement

Fix a good input table U . The auxiliary comparison alignment follows true optimal costs. It lets us charge the refinement error across the source tree, including the bias of approximate leaf estimates.

8.1 Comparison tree and curvature

Start from a fixed state of true cost $D > 0$. The comparison tree stops at physical leaves and true zero-cost states. At a zero-cost terminal, set $Q = 0$ for this analysis. At each nonterminal, choose $b \leq c < 2b$ and set $r = r_b(q)$ and $e = d(q, r)$. Fix an optimal weighted alignment for (I, r) by a deterministic tie rule. Choose ordered points on it at the child cuts, then round its internal target anchors to the scale- b net. Let q_i be the resulting child states and α^* their shared-endpoint action. This comparison tree depends on true costs and fixed rounding rules, not on refinement randomness.

Lemma 8.1. *The chosen (b, r) is a representative in the local scale range, and α^* is a wide action with shift radius $6b$. Moreover,*

$$c(I, r) \leq \sum_i c(I_i, q_i) \leq (1 + 4M/P)c, \quad c \leq Le + \sum_i c(I_i, q_i).$$

Writing C_{term} for the sum of terminal true costs,

$$C_{\text{term}} \leq (1 + 8MJ/P)D \leq 2D, \quad \sum_{\text{nonterm}} c \leq 2JD, \quad \sum_{\text{nonterm}} e \leq 4JD/P.$$

Terminal table values equal Q , and their sum $\tilde{C}_{\text{term}}$ lies between C_{term} and $(1 + \varepsilon_{\text{leaf}})C_{\text{term}}$.

Proof. Endpoint regularity gives $c - e \leq c(I, r) \leq c + e$. Lemma 7.7 and $e \leq 2b/P$ imply representative eligibility and the local scale bounds. Along the optimal path, shift variation is at most $c(I, r)$ by Lemma 5.3. Internal rounding adds at most b/P per anchor, keeping the shift radius below $6b$.

Before rounding, the subpath costs sum to $c(I, r)$. Each internal anchor changes two child endpoints, adding at most $2(M - 1)b/P$ in total. This proves the upper child-cost bound. The rounded children retain shared endpoints, so concatenation gives $c(I, r) \leq \sum_i c(I_i, q_i)$. Moving the parent's endpoints back to q then gives $c \leq Le + \sum_i c(I_i, q_i)$.

At each level, replace every nonterminal frontier state by its comparison children and keep every terminal frontier state. The frontier's total true cost grows by a factor of at most $1 + 4M/P$. There are at most J replacement levels. Since $4MJ/P \leq 1/2$, the total multiplier is at most $1 + 8MJ/P \leq 2$. Summing over levels and using $e \leq 2c/P$ proves the frontier bounds. Lemma 5.6 gives the terminal approximation bound. At a terminal zero-cost state, all table values are zero by the good-input upper bound. $\square$

At a nonterminal write

$$m_q = m(I, q), \quad m_i = m(I_i, q_i), \quad W = \sum_i m_i, \quad G = W - Z, \quad R_q = m_q - W,$$

and $z_i = z(I_i, q_i)$, $z_r = z(I, r)$. Shared endpoints give $\sum_i z_i = z_r$. The preceding bounds yield

$$0 < \max\{c(I, r)/a, |z_r|\} \leq Z \leq W \leq 4Fc < 8Fb.$$

Define

$$K = \sum_i \frac{z_i^2}{m_i} - \frac{z_r^2}{W} = \sum_{i:m_i>0} \frac{(z_i - vm_i)^2}{m_i}, \quad v = z_r/W,$$

with $0/0 = 0$. The lower invariant makes this convention legitimate.

Lemma 8.2. *For $h_q = Le + w_0(I, q)/T$,*

$$\begin{aligned} \sum (K + G) &\leq C_{\text{term}} + 3\tilde{C}_{\text{term}} + 3\sum e + 4\sum h_q, \\ \sum (K + G) &\leq 5000F(1 + J/P + J/T)D. \end{aligned}$$

All sums are over nonterminals.

Proof. Averaging the warmup Bellman comparisons and minimizing their fixed action gives

$$m_q - (w_0 - w_T)/T \leq Le + Z.$$

This gives $R_q + G \leq h_q$, $R_q^+ \leq h_q$, and $G \leq R_q^- + h_q$. The sum of $R_q = m_q - \sum_i m_i$ telescopes to $m_{\text{root}} - \tilde{C}_{\text{term}}$. Combining these bounds gives

$$\sum R_q^- \leq \sum h_q + \tilde{C}_{\text{term}}, \quad \sum G \leq 2\sum h_q + \tilde{C}_{\text{term}}.$$

Let $E(I, q) = z(I, q)^2/m(I, q)$, with $E(I, q) = 0$ when $m(I, q) = z(I, q) = 0$. These energy quantities are used only under the good-seed invariants and are not evaluated by the algorithm. The imbalance and endpoint bounds give $|z_q| \leq m_q$, $|z_r - z_q| \leq e$, and $|z_r| \leq W$. Since $e \leq 2am_q/P \leq m_q$, they also give $|z_r| \leq 2m_q$. Expand the energy difference as

$$\frac{z_q^2}{m_q} - \frac{z_r^2}{W} = \frac{z_q^2 - z_r^2}{m_q} + \frac{z_r^2(W - m_q)}{m_q W}$$

The bounds $|z_q| \leq m_q$ and $|z_r| \leq 2m_q$ make the first term at most $3e$. If $W \leq m_q$, the second term is nonpositive. If $W > m_q$, the bounds $|z_r| \leq 2m_q$ and $|z_r| \leq W$ give $z_r^2/(m_q W) \leq 2$. The second term is then at most $2(W - m_q) = 2R_q^-$. Consequently

$$K \leq \sum_i E(I_i, q_i) - E(I, q) + 3e + 2R_q^-.$$

At a terminal, $m = Q \geq c \geq |z|$, so its energy is at most c . When we sum the energy inequality over the tree, internal energies cancel. The remaining terminal energies sum to at most C_{term} , and the root energy is nonnegative. Adding the bound for $\sum G$ proves the first asserted inequality.

Finally, Lemma 8.1 gives

$$\sum h_q \leq 512FJD/P + 4FJD/T.$$

Since $\tilde{C}_{\text{term}} \leq (1 + \varepsilon_{\text{leaf}})C_{\text{term}} \leq 4D$, the right side of the first asserted inequality is at most

$$14D + (2048F + 12)JD/P + 16FJD/T,$$

which is at most $5000F(1 + J/P + J/T)D$. □

Call a nonterminal bad if $K > bH^{-1/4}$ or $G > bH^{-1/4}$.

Corollary 8.3. *The comparison tree satisfies*

$$\sum_{\text{bad}} c \leq 10000FH^{1/4}(1 + J/P + J/T)D.$$

Proof. At a bad node, $c < 2b < 2H^{1/4}(K + G)$. Sum this bound and apply Lemma 8.2. $\square$

In the accuracy analysis, this cost sum is multiplied by $4F/S$. Substituting the parameters gives $O(\ell^4 H^{-1/2}/\chi)D$.

Lemma 8.4. *At a non-bad comparison node with $|\hat{v}| \leq 1 - \tau$, the comparison action α^* belongs to the retained band.*

Proof. Let p_j^* be the comparison shift at cut j , and put $W_j = \sum_{i \leq j} m_i$. The comparison prefix is feasible for $\Phi_j(p_j^*)$, so $\Phi_j(p_j^*) \leq W_j$. Join a minimizing prefix to the comparison suffix at its actual target boundary, whose mass is $M_x(l_j) + p_j^*$. The resulting wide action has cost $\Phi_j(p_j^*) + W - W_j$. Its cost is at least Z , giving

$$W_j - G \leq \Phi_j(p_j^*) \leq W_j.$$

Shared endpoints give $p_j^* - p_0 = \sum_{i \leq j} z_i$. Also $|v - \hat{v}|W = |z_r|G/Z \leq G$. Weighted Cauchy-Schwarz gives $\sum_i |z_i - vm_i| \leq \sqrt{KW}$. Write the deviation as

$$\begin{aligned} p_j^* - p_0 - \hat{v}\Phi_j(p_j^*) &= \sum_{i \leq j} (z_i - vm_i) + (v - \hat{v})W_j \\ &\quad + \hat{v}(W_j - \Phi_j(p_j^*)). \end{aligned}$$

The three terms are bounded by $\sqrt{KW}$, G , and G , respectively. Hence

$$|p_j^* - p_0 - \hat{v}\Phi_j(p_j^*)| \leq \sqrt{KW} + 2G.$$

The band test allows deviation $\tau\theta b/M$. After multiplying by M/b , the preceding bound is at most

$$\sqrt{8\ell} H^{-3/40} + 2H^{-1/5} \leq \ell^{-44} = \tau\theta,$$

at the fixed cutoff. To check this, multiply the first term by ℓ^{44} and use $\ell^{200} \leq H^{1/100}$. Its power of H is at most $-3/40 + 44.5/20000 < 0$. The second term has a smaller power of H . Both terms are below half the required bound at $\chi \geq 2^{1000}$. Radius and shared endpoints follow from Lemma 8.1.

Any band child state is within endpoint distance $4\theta b/M$ of q_i . By Lipschitzness of w_0 and dyadic ceiling,

$$h_\Sigma \leq 2b + 2 \sum_i w_0(I_i, q_i) + 8L_0\theta b \leq 2b + 4F \sum_i c(I_i, q_i) + 8L_0\theta b \leq 64Fb.$$

The rounding and imbalance guards then pass by the explicit estimates following their definition. $\square$

8.2 Sampling estimates

For a band action put $x_t(\alpha) = \sum_i d_t(I_i, s_i(\alpha))$.

Lemma 8.5. *For every retained band and every $\alpha \in \mathcal{C}_{b,r}$,*

$$V_r(\beta(b, r); d_t) \leq (1 + \delta)x_t(\alpha) + (6L + 2)\theta h_\Sigma, \quad d_t(I_i, s_i(\beta(b, r))) \leq 2h_i.$$

Proof. Two states of child i in the band are within endpoint distance $4\theta b/M$. Canonical rounding moves its endpoints by at most $2\theta h_i$. Apply the spatial bound in Lemma 7.17 and sum over children. The connection cost introduced by rounding is at most $2\theta h_\Sigma$. For the second bound, use the Lipschitz bound on w_0 , the definition of h_i , and $d_t \leq w_0$. $\square$

Lemma 8.6. *Condition on complete child tables. With probability at least $1 - H^{-4}$, all sampled scores in one group differ from actual rounded-action costs by at most γb . On that event, for every member (b, r) , every parent state q , and every $\alpha \in \mathcal{C}_{b,r}$,*

$$\Pi_t(q) \leq Ld(q, r) + (1 + \delta)x_t(\alpha) + e_*b, \quad e_* = 2\gamma + 512LF\theta \leq \kappa\delta.$$

Proof. For a sampled scale h , normalize a uniform-label summand as

$$Z_i = (M/b)\mathbf{1}_{h_i=h}d_t(I_i, s_i(\beta(b, r))).$$

Lemma 8.5 gives

$$0 \leq Z_i \leq 2Mh/b, \quad \mathbb{E}Z_i \leq 128F, \quad \text{Var}(Z_i) \leq 256FMh/b.$$

With at least Q_1Mh/b samples, Bernstein's inequality at normalized error $\gamma/(2\ell)$ gives failure at most $2\exp(-\ell^{37}/3000)$. Exact scales have no failure.

There are at most $\log_2(64FM) + 2 \leq 2\ell$ labels. A cell side contains at most $1 + 16\theta P \leq P + 1$ net points, so a group has at most $(P + 1)^2$ members. Lemma 7.2 bounds the full union failure by

$$4\ell(P + 1)^2 e^{-\ell^{37}/3000} \leq H^{-4}.$$

Summing the errors over labels gives score error at most γb .

The winning actual cost is at most any member's actual cost plus $2\gamma b$. At a member center r , the local rule reaches its whole cell. Lemma 7.16 transfers the bound to every parent state q . Representatives in one cell differ by endpoint distance at most $2\theta b$. Add the resulting cone displacement to the rounded-cost bound in Lemma 8.5, using $h_\Sigma \leq 64Fb$. Lemma 7.2 bounds the total error coefficient by $2\gamma + 512LF\theta \leq \kappa\delta$. The conditioning fixes all bands and child values. It leaves the fresh group streams independent with their original laws. $\square$

Lemma 8.7. *Fix a parent state q and a local scale b for which $r = r_b(q)$ has a retained band. Condition on the complete child tables. For every $\alpha^* \in \mathcal{C}_{b,r}$ and every integer $0 \leq s \leq S$, the online actions satisfy*

$$\mathbb{E} \sum_{t \leq s} [x_t(\alpha_t) + \kappa(x_{t-1}(\alpha_t) - x_t(\alpha_t))] \leq \sum_{t \leq s} x_t(\alpha^*) + h_\Sigma \left(\frac{\ln d_{\text{act}}}{\eta} + 2\kappa + sH^{-3} \right).$$

Here d_{act} is the number of action coordinates, and

$$d_{\text{act}} \leq M(321P)^2, \quad \ln d_{\text{act}} \leq 4(p_* + 21)\chi.$$

Proof. Put $f_t(i, s) = d_t(I_i, s)/h_i$. Lemma 7.17 and the definition of h_i put these values in $[0, 1]$ and make them nonincreasing. Averaging all action vertices gives a positive vector in the action hull. Its block masses equal h_i/h_Σ . The spatial bound and band diameter give within-block cost variation at most $\delta h_i + 4L\theta b/M$. The range of the normalized decrement is consequently at most

$$2\delta + 8L\theta \leq 3\delta \leq \kappa/\eta.$$

Lemma 7.2 verifies the required decrement bound. Apply Proposition 7.12 with $d = d_{\text{act}}$. Proposition 7.13, used with $h_{\text{opt}} = H$, supplies mean error at most H^{-3} at each iteration, which adds at most sH^{-3} to the normalized bound. The conditioning fixes the child histories and leaves the parent streams independent with their original laws.

Each wide endpoint lies in a mass interval of length $20b$. Net separation gives at most $1 + 20b/g_b \leq 321P$ choices. There are M child blocks, so $d_{\text{act}} \leq M(321P)^2$. The logarithmic estimate is Lemma 7.2. $\square$

8.3 Clipped regret and refinement accuracy

At comparison states define, for analysis only, $\bar{d}_t = \max(c, d_t)$ and

$$\mathcal{R}_t(I, q) = \bar{d}_t(I, q) - Le - \sum_i \bar{d}_t(I_i, q_i).$$

Lemma 8.8. *Every nonterminal satisfies*

$$\sum_{t=1}^S \mathcal{R}_t \leq \sum_i (\bar{d}_0 - \bar{d}_S)(I_i, q_i) \leq \sum_i (m_i - |z_i|) \leq W - |z_r| \leq 4Fc.$$

At a non-bad node failing the velocity test, the bound is at most $(H^{-1/4} + 4\tau F)c$.

Proof. The earlier-index Bellman action gives $d_t(q) \leq Le + \sum_i d_{t-1}(q_i)$. After clipping the child values, this right side is at least c by Lemma 8.1. Since it bounds both c and $d_t(q)$, it also bounds $\bar{d}_t(q)$. Subtracting the current clipped child sum gives

$$\mathcal{R}_t \leq \sum_i (\bar{d}_{t-1} - \bar{d}_t)(I_i, q_i).$$

Summing over t proves the first inequality.

For a child with $d_0 \leq c_i$, monotonicity makes both clipped endpoints equal to c_i . For any other child, $\bar{d}_0 = d_0 \leq m_i$ and $\bar{d}_S \geq |z_i|$. Thus its total clipped decrement is at most $m_i - |z_i|$. Sum over children and use $\sum_i z_i = z_r$ to obtain the remaining general bounds. At a non-bad node failing the velocity test, $|z_r|/Z > 1 - \tau$. Since $Z = W - G \leq W$, this gives $W - |z_r| \leq G + \tau W \leq (H^{-1/4} + 4\tau F)c$. $\square$

Lemma 8.9. *At a non-bad comparison node passing the velocity test, conditional on complete child tables,*

$$\mathbb{E} \sum_{t=1}^S \mathcal{R}_t \leq h_\Sigma \left(\frac{\ln d_{\text{act}}}{\eta} + 2\kappa + SH^{-3} \right) + 2\delta c + 2SH^{-4}Fc.$$

Proof. Coverage supplies $\alpha^* \in \mathcal{C}_{b,r}$. Write $A_t(\alpha) = Le + x_t(\alpha)$. Let $t_\times$ be the first index with $\min_{\alpha \in \mathcal{C}} A_t(\alpha) < c$, or $S + 1$ if there is no such index. This crossing index is fixed by the conditioned child tables. We bound regret before the crossing, at the crossing, and after it.

For $t < t_\times$, all action costs are at least c . Let E_t be the group's failure event from Lemma 8.6. We claim

$$\bar{d}_t(q) \leq A_t(\alpha_t) + \kappa(x_{t-1}(\alpha_t) - x_t(\alpha_t)) + 2Fc \mathbf{1}_{E_t}.$$

If $d_t(q) \leq A_t(\alpha_t)$, clipping preserves this assertion because $A_t(\alpha_t) \geq c$. Otherwise abbreviate $A_t = A_t(\alpha_t)$. The sampled action is an online candidate, so $Y_t \leq A_t$. The clipped update then

forces $\Pi_t > (1 + \delta)A_t$ and $d_t = \Pi_t/(1 + \delta)$. The earlier Bellman option gives $\Pi_t \leq A_t + \Delta x$, where $\Delta x = x_{t-1}(\alpha_t) - x_t(\alpha_t)$. Consequently $\Delta x > \delta A_t \geq \delta b$. On group success, Lemma 8.6 gives

$$d_t - A_t \leq \frac{e_* b - \delta L e}{1 + \delta} \leq \frac{\kappa \delta b}{1 + \delta} \leq \kappa \Delta x.$$

On failure the extra $2Fc$ suffices by $\bar{d}_t \leq 2Fc$.

If a crossing occurs, choose an action attaining it. On group success, its score bound gives $\Pi_{t_x} < c + 2\delta c$. The comparison-child sum plus Le is at least c , so the clipped regret at this index is at most $2\delta c$. Group failure adds at most $2Fc$. At every later index, the crossing action and child monotonicity give $\mathcal{B}(d_{t-1})(q) < c$. Hence $\bar{d}_t(q) = c$, and its regret is nonpositive.

Apply Lemma 8.7 to the fixed prefix before the crossing. Its comparator cost $x_t(\alpha^*)$ is at most the clipped comparison-child sum subtracted in $\mathcal{R}_t$. The connection term Le cancels. There are at most S failure indicators, each with conditional probability at most H^{-4} . Adding their expectations proves the bound. This step uses no independence between the sampled action and group failure. $\square$

Lemma 8.10. *Fix an integer table $U \in [0, R]$ satisfying $c \leq U \leq Ac$ at every state, where $A \leq A_{\text{init}}$ is a power of two. Put $a = \max(1, A/F)$. The refinement table satisfies, at every state,*

$$d_S \geq c/a \quad \text{on every outcome,} \quad \mathbb{E}(d_S - c)_+ \leq c/\ell.$$

The expectation fixes the entire input table U . Every zero-cost state has value zero.

Proof. The lower bound is Lemma 7.17. At zero cost, $w_0 = 0$ gives the claimed zero value. Now assume that the root cost D is positive. At a terminal, every clipped value equals Q . Summing the definition of $\mathcal{R}_t$ over the comparison tree cancels internal values and gives

$$\bar{d}_t(\text{root}) = \tilde{C}_{\text{term}} + L \sum e + \sum \mathcal{R}_t.$$

Average over t and use monotonicity to bound the final root value. The terminal term satisfies $\tilde{C}_{\text{term}} - D \leq (C_{\text{term}} - D) + 2\varepsilon_{\text{leaf}}D$. This is where the leaf approximation error enters.

For bad nodes, combine Lemma 8.8 with Corollary 8.3. For other nodes failing the velocity test, use the sharper decrement bound. For covered nodes, use Lemma 8.9, $h_\Sigma \leq 64Fc$, and the action-coordinate bound in Lemma 8.7. Taking conditional expectations at each node and then summing uses the tower property. With $\sum c \leq 2JD$, these bounds give

$$\begin{aligned} \frac{\mathbb{E}(d_S - D)_+}{D} &\leq 2\varepsilon_{\text{leaf}} + \mathcal{E}, \quad \mathcal{E} = (8M + 512F)J/P \\ &\quad + 40000F^2H^{1/4}(1 + J/P + J/T)/S \\ &\quad + \frac{2J}{S} \left[H^{-1/4} + 4\tau F + 64F \left(\frac{4(p_* + 21)\chi}{\eta} + 2\kappa \right) + 2\delta \right] + 132FJH^{-3}. \end{aligned}$$

Substitute $J \leq 100H/\chi$, $T \geq H^{3/4}$, $S = H/\ell^2$, and $F = \ell$. Retaining all contributions gives the following nine-term bound:

$$\begin{aligned} \mathcal{E} &\leq \frac{100(8M + 512\ell)H}{\chi P} + 40000\ell^4 H^{-3/4} \\ &\quad + \frac{4 \cdot 10^6 \ell^4 H^{1/4}}{\chi P} + \frac{4 \cdot 10^6 \ell^4 H^{-1/2}}{\chi} + \frac{200\ell^2 H^{-1/4}}{\chi} \\ &\quad + \frac{26400}{\chi \ell} + \frac{51200(p_* + 21)}{\ell^2} + \frac{400}{\chi \ell^8} + \frac{13200\ell}{\chi H^2}. \end{aligned}$$

The coefficient 26400 combines 800 from the velocity term and 25600 from the 2κ term. The entropy contribution is $51200(p_* + 21)/\ell^2$, using the action-coordinate estimate in Lemma 8.7.

Each displayed term is at most $1/(16\ell)$ at the stated cutoff. For the sixth, seventh, and eighth terms, multiply by 16ℓ and use respectively

$$\frac{422400}{\chi} < 1, \quad \frac{819200(\chi + 21)}{\ell} \leq \frac{819200(\chi + 21)}{\chi^3} < 1, \quad \frac{6400}{\chi\ell^7} < 1.$$

For each remaining term, multiplying by ℓ gives at most $10^8 H^{-1/5}$: use $\ell \leq H^{1/20000}$, $M \leq H^{1/20}$, and $P = H^{\chi+20}$. The cutoff makes this quantity smaller than $1/16$. Finally $2\varepsilon_{\text{leaf}} = 2\ell^{-3} \leq 1/(16\ell)$. Consequently

$$\mathcal{E} + 2\varepsilon_{\text{leaf}} \leq \frac{10}{16\ell} < \frac{1}{\ell}.$$

□

9 The bounded-integer approximation scheme

Fix an integer metric v whose positive values are at most N^3 . Apply the preceding constructions with $w = v$. Write $R = M_v(x) + M_v(y)$ and $D_v = \text{WED}_v(x, y)$. Then $N \leq R \leq N^4$. The common parameters are

$$H = \text{leastpow}_2(\lceil \log_2(N + R + 2) \rceil), \quad \chi = \log_2 H, \quad \ell = \text{leastpow}_2(\max\{1, \chi^3\}).$$

Use this same H for every coarse entry, refinement copy, and pass. On the large-input branch, $\chi \geq 2^{1000}$ and $\ell\varepsilon_{\text{core}} \geq 10$, where $\varepsilon_{\text{core}} \in (0, 1)$ is the core accuracy. Thus $p_* = \chi$, $B = H^{p_*}$, $A_{\text{init}} = BH^4$, and $F = \ell$. The remaining parameters and the labeled physical source tree are those of the fixed refinement construction.

9.1 The initial table on the complete state family

Lemma 9.1. *At the stated large-input cutoff, the coarse estimator defines a random integer table $U^{(0)} \in [0, R]$ on the complete physical state family. With probability at least .99, simultaneously at every state,*

$$c(I, q) \leq U^{(0)}(I, q) \leq A_{\text{init}}c(I, q).$$

Every zero-cost state returns zero on every execution. Shared preprocessing is deterministic and uses $N \text{ poly}(\log N)$ work and polynomial space. Conditional on the history before an entry's first request, its expected work is at most

$$(1 + |I|) 2^{O(H/\chi + \chi^2)},$$

uniformly in the target interval and without conditioning on accuracy. Every entry computation terminates and uses polynomial space on every fair-bit sequence.

Proof. Use Theorem 6.7 with per-cost exponent $C = 3$. The complete source tree has at most $2M^J \leq 2M \max(1, |x|)$ labeled nodes. There are at most $(|y| + 1)(|y| + 2)/2$ target intervals. Since $M \leq N$, the complete state family has fewer than N^6 entries at the stated cutoff. Different physical labels remain different entries, even when their numerical source intervals agree. This indexed family is fixed before any entry stream is exposed.

The concentration count in the proof of Lemma 6.4, with $C = 3$ and $s = 6$, bounds its failure probability by

$$100HN^{11}e^{-H^3/768}.$$

Here $H < 8\log_2 N + 4$ for $N \geq 2$, and $H \leq N$ at the stated cutoff. The failure probability is at most

$$100N^{12}e^{-H^3/768} < .01.$$

Indeed, $N \leq 2^H$, and $H^3/768 > 12H \ln 2 + \ln 10000$ for $H \geq 128$. The stated cutoff is larger than this threshold.

Lemma 6.6 gives the entry overhead factor $2^{O(H/p_* + p_* \log_2 H)} = 2^{O(H/\chi + \chi^2)}$. Each entry reads only completed deterministic prefix arrays and reporting structures. It uses its own private stream and workspace, and it does not request another initial entry. Before its first request, that stream is unexposed, so the conditional work bound follows from the same calculation. Lemma 6.5 and the fixed finite guess range give termination and polynomial space on every execution. $\square$

9.2 The pass schedule and its accuracy

Set $A_0 = A_{\text{init}}$. While $A_{p-1} > F$, define

$$a_p = A_{p-1}/F, \quad A_p = 4a_p.$$

Let p_{med} be the number of these median passes, and set $p_{\text{fin}} = p_{\text{med}} + 1$ and $s_{\text{pass}} = p_{\text{fin}}$. This schedule depends only on the numerical parameters. It is computed before any random table entry is requested. Every A_p is a power of two, and every a_p used in a median pass is an integer power of two.

In median pass p , apply the refinement to the preceding complete table $U^{(p-1)}$ using H^2 fresh independent copies, indexed by $c_{\text{copy}} = 1, \dots, H^2$. At each state, let

$$U^{(p)}(I, q) = \min \left\{ R, \max \left\{ 0, \text{os}_{H^2/2+1}(\lceil a_p d_S^{p, c_{\text{copy}}}(I, q) \rceil : 1 \leq c_{\text{copy}} \leq H^2) \right\} \right\},$$

where os_k denotes the k th smallest value. The copy streams are independent conditional on the complete preceding table. After the last median pass, use one fresh refinement copy with input $U = U^{(p_{\text{med}})}$, factor $A = A_{p_{\text{med}}}$, and $a = 1$. The mathematical core output is

$$Q_{\text{core}} = \left\lfloor \min \{ R, \max \{ 0, d_S^{p_{\text{fin}}, 1}(I_{\text{root}}, (0, |y|)) \} \} \right\rfloor.$$

These definitions specify complete tables. They do not require the implementation to materialize them.

Lemma 9.2. *The deterministic schedule satisfies*

$$s_{\text{pass}} = O((p_* + 1)\chi / \log \ell) = O(\chi^2 / \log \chi) < H.$$

Proof. Each median pass decreases $\log_2 A$ by $\log_2 F - 2 > 0$. Since $\log_2 A_0 = (p_* + 4)\chi$, the total number of passes is at most

$$2 + \frac{(p_* + 4)\chi}{\log_2 F - 2}.$$

Use $p_* = \chi$ and $\chi^3 \leq F < 2\chi^3$. The resulting bound is smaller than $H = 2^\chi$ at the stated cutoff. $\square$

Lemma 9.3. *On the large-input branch,*

$$\mathbb{P}\{D_v \leq Q_{\text{core}} \leq (1 + \varepsilon_{\text{core}})D_v\} > 4/5.$$

Proof. If $D_v = 0$, the initial table and every subsequent table return zero at the root on every outcome. Assume $D_v > 0$. Condition on a complete preceding table satisfying $c \leq U^{(p-1)} \leq A_{p-1}c$ at every state. At a fixed state (I, q) , put $c_q = c(I, q)$. Lemma 8.10 gives $d_S^{p, c_{\text{copy}}} \geq c_q/a_p$ on every outcome. Thus each scaled value $\lceil a_p d_S^{p, c_{\text{copy}}} \rceil$ is at least c_q . For a positive-cost state, the same lemma and Markov's inequality give

$$\mathbb{P}\{d_S^{p, c_{\text{copy}}} > 2c_q\} \leq 1/\ell.$$

When $d_S^{p, c_{\text{copy}}} \leq 2c_q$, the integrality of a_p and c_q implies $\lceil a_p d_S^{p, c_{\text{copy}}} \rceil \leq 2a_p c_q$. The probability that at least half the copies fail is at most

$$2^{H^2} \ell^{-H^2/2} \leq e^{-H^2}.$$

If fewer than half fail, the upper middle order statistic is at most $2a_p c_q \leq A_p c_q$. Clamping preserves the lower bound because $c_q \leq R$. Zero-cost states stay zero.

The state count is at most $N^6 \leq e^{6H \ln 2} < e^{10H}$, so one transition fails with probability at most e^{10H-H^2} . This union bound does not require independence across states. Apply it to the first failed transition after a good preceding table. Lemma 9.1 and Lemma 9.2 bound the probability of any required bad seed by

$$.01 + H e^{10H-H^2} < 1/10.$$

At the end, $A_{p_{\text{med}}} \leq F$. Conditional on any fixed good final seed, Lemma 8.10 applies to the fresh final copy with $a = 1$. Its root value is at least the integer D_v , and

$$\mathbb{P}\{d_S^{p_{\text{fin}}, 1} > (1 + \varepsilon_{\text{core}})D_v\} \leq \frac{1}{\ell \varepsilon_{\text{core}}} \leq 1/10.$$

Clamping cannot increase the value and preserves its lower bound. Flooring also preserves that bound because D_v is an integer. Combining the two failure bounds proves the assertion. $\square$

9.3 The coherent cache implementation

Use the canonical identities and private owners of Definition 7.15 in the fresh core-run namespace ν . A cache record has status absent, active, or complete. Completed payloads and their stored owner decisions are immutable. The program draws bits only when an owner is first reached. Shared prefix arrays, nets, and reporting structures are deterministic.

Put $B_{\text{idix}} = T + S$. Assign the scalar ranks

$$\begin{aligned} \text{rk}(U^{(0)}) &= 1, \\ \text{rk}(w_j^{p, c_{\text{copy}}}) &= (p-1)(B_{\text{idix}} + 2) + 2 + j, \\ \text{rk}(d_t^{p, c_{\text{copy}}}) &= (p-1)(B_{\text{idix}} + 2) + 2 + T + t, \\ \text{rk}(U^{(p)}) &= p(B_{\text{idix}} + 2) + 1. \end{aligned}$$

Here c_{copy} is the copy index, and $d_0 = w_T$. A mean has the rank of w_T , a band the rank of d_1 , and a group or online record at index t the rank of d_t . A leaf record has rank zero. Give scalar seed, warmup, and refinement records phase 2, group and online records phase 1, and mean, band, and leaf records phase 0. Order keys lexicographically by rank, remaining physical height, and phase.

A median seed reads same-node final refinement records of rank one less. At a physical leaf, a warmup or refinement record reads its shared rank-zero leaf record. At an internal node, w_0 reads the preceding seed at its query and reported centers, all of rank one less. The local helper reads same-node w_0 values. Warmup propagation and the refinement's Bellman fallback read lower-rank child values. Means read same-node warmup indices below T , and bands read lower-rank child means and warmup values. A scalar refinement record reads same-node group and online records with smaller phase. These records read same-node bands with smaller rank, or smaller phase at index one, and lower-rank earlier child histories. Their current-index scalar reads have equal rank and smaller physical height. Every persistent read decreases the displayed key order.

Algorithm 9 Coherent lazy evaluation and batched scalar reads

Require: Complete keys in one core-run namespace and their defining routines.

Ensure: Immutable completed records and one GET per distinct canonical scalar key in a gather.

```

1: procedure GET( $K$ )
2:   Canonicalize  $K$ , including the supplied-input identity and  $d_0 = w_T$ .
3:   if Cache[ $K$ ] is complete then
4:     return its stored payload.
5:   end if
6:   assert that Cache[ $K$ ] is absent.
7:   Mark Cache[ $K$ ] active.
8:   Run its defining routine, using its private stream when needed.
9:   Store its payload and owner decisions immutably; mark it complete.
10:  return its stored payload.
11: end procedure

12: procedure GATHER( $\mathcal{L}$ )
13:   Canonicalize, sort, and deduplicate the scalar keys in  $\mathcal{L}$ .
14:   Initialize an empty local map  $\text{val}$ .
15:   for each distinct key  $K$ , in sorted order do
16:      $\text{val}[K] \leftarrow \text{GET}(K)$ .
17:   end for
18:   return  $\text{val}$ .
19: end procedure

```

Wide lists, prefix minima, representative lists, Bellman scores, gathered arrays, and optimizer iterates are local scratch data. Every repeated scratch read is charged. An initial seed key runs the standalone initial-entry routine. A median seed key gathers its H^2 final-copy values and applies the order statistic and clamps already defined.

Lemma 9.4. *GET never encounters an active record. It terminates on every fair-bit sequence and returns the complete-table value for its key. The resulting output has the distribution of Q_{core} .*

Proof. Every recursive call decreases the displayed key order. Its ranks, heights, and phases are nonnegative integers, so recursive evaluation is well founded. The standalone initial routine is finite by Lemma 6.5. All other local enumerations have fixed finite domains. Proposition 7.13 gives a finite optimizer and a sampler with a bounded number of fair bits. Thus every defining routine terminates after its dependencies terminate. An active key cannot be reached from itself through decreasing keys.

Induction on the same order shows that each completed payload equals its mathematical record with the same owner stream. The program never redraws a completed owner or creates a new child value for a new caller. Coupling its reached owner streams with the potential streams of the complete construction proves the distributional assertion. $\square$

Accuracy conditions on complete child histories, including unreached entries. Those histories exclude current parent group and online streams. Work conditions instead on the exposed execution before a first request. An absent initial entry has exposed none of its private stream. The realized cache and request trace are not the conditioning objects in the accuracy argument.

9.4 Finite arithmetic and unconditional work

9.4.1 Finite returned values and local computation

Every table used as a seed is clamped to $[0, R]$. On any such seed, the local w_0 routine returns zero, a cone value, or default R . Its tested centers are at mass sup-distance at most $U(I, q) \leq R$, and $\tilde{U} \leq R$. Consequently

$$0 \leq w_0 \leq (2L_0 + 1)R.$$

All connection and cone charges are integers. Warmup values are integers, and means have denominator T . Every refinement update is nonnegative and at most its local w_0 , irrespective of seed accuracy.

Let $Q_{\text{den}} = \ell^{10} + 1$, and let $h(I)$ be remaining physical height. Induction on height, for all indices together, shows that the denominator of every returned $d_t(I, q)$ divides $Q_{\text{den}}^{h(I)}$. Leaves return the integer Q . A parent candidate adds an integer charge to child values, or uses an integer warmup value. Minima and maxima select arguments. The only new denominator comes from dividing by $1 + \delta = Q_{\text{den}}/\ell^{10}$. Thus returned values have $O(H + J \log \ell) = O(H)$ bits. Sampling scores and optimizer mixtures choose an action; they do not average recursive costs into a returned value.

All band and rounded-state guards are exact rational comparisons. Their inputs, and the online linear coefficients, have $H^{O(p_*+1)}$ bits. The dyadic labels satisfy $b/M \leq h_i \leq h_\Sigma \leq 64Fb$. Every Mh_i is an integer, so the action vectors share the integer denominator Mh_Σ , of $O(H)$ bits. Each wide endpoint has at most $321P$ candidates. Thus the optimizer dimension is at most $M(321P)^2$, and its input list has at most $(321P)^{2M}$ actions. Proposition 7.13 gives deterministic local bit work and storage bounded by

$$2^{O(M(p_*+1)\chi)} H^{O(p_*+1)} = 2^{O(H^{1/20}\chi^2 + \chi^2)} = 2^{o(H \log \chi / \chi)}.$$

The numerical iterations reuse gathered earlier-index values. Only the sampled whole action requests current-index child values.

9.4.2 Distinct recursive dependencies and repeated reads

The initial scale ratio is $O(A^3)$, and its radius-to-net-spacing ratio is $O(PA^2)$. Later local scale ratios are $128aF$. Both scale counts are $O((p_* + 1)\chi) = O(\chi^2) = O(\ell)$. In a later search, $w_0(I, q)/b \leq 16F$. The query square meets only $O((1 + F/\theta)^2)$ whole cells per scale. Each cell contains $O(P^2)$ representatives. Binary searches in the net arrays generate the lists without scanning unrelated target positions. Cell indices have $O(H)$ bits.

Each child has $O(P^2)$ wide states. Gathering their distinct earlier-index values and histories, over all local representatives, cells, and scales, uses $D_{\text{earlier}} = H^{O(p_*+1)}$ scalar inputs. Action enumeration has $(O(P))^{2M}$ local steps. Prefix minima are evaluated only at the finite candidate boundary shifts and use the same gathered values.

For current-index requests, fix a group, scale label h , and child. Rounded states use the common net spacing $D_h = \max(1, \theta h/16)$. Their first endpoint ranges over mass length $O(b)$. After fixing it,

the guard $|z| \leq 2h$ confines the second endpoint to mass length $4h$. Net separation bounds their number by

$$O((1 + b/D_h)(1 + h/D_h)) = O(\theta^{-2}(1 + b/h)).$$

If $k_h < M$, then $k_h \leq 2Q_1 Mh/b$ and $h/b < 1/Q_1$. Multiplying the state count by the number of sampled labels gives $O(M\theta^{-2}Q_1)$ distinct inputs for this scale label. If $k_h = M$, then $b/h < 2Q_1$, and summing all labels gives the same bound. There are $O(\ell)$ labels and $\ell^{O(1)}$ local groups. Evaluating a group winner adds M calls per group. Each online draw adds M calls per local scale. Hence one refinement entry has at most

$$D_{\text{same}} = M\ell^{O(1)}$$

same-pass, same-index child dependencies. The test $h_i = h$ precedes each group request, so a zero indicator does not create a dependency.

These bounds count distinct dependencies. A local dictionary associates each key with its completed scalar value, including values first needed after a winner is selected. Repeated scoring reads keep their multiplicities but reuse those values. There are $O(P^2)$ representatives per group, at most M sampled labels per scale label, and $\ell^{O(1)}$ groups and labels. Direct group scoring uses at most $MP^2\ell^{O(1)}$ lookups, besides the earlier-value reads and selected actions. Key construction and each dictionary lookup have polynomial-in- H bit cost. Thus all repeated lookups fit within the local $H^{O(p_*+1)}$ budget. Full action enumeration and optimizer iterations have the separate local bound already proved; they do not create one recursive call per action.

9.4.3 The dependency count and source-width sum

For this count, fix the potential owner streams and expand each auxiliary record into its scalar requests, retaining its realized owner decisions. The bounds above hold for every such decision. In the resulting scalar request graph, every equal-rank edge descends one physical child. Every other edge decreases rank, including the H^2 calls of a median seed. Its fanout is absorbed in D_{earlier} . Put $K_{\text{dep}} = s_{\text{pass}}(T + S + 2) + 1$. Lemma 9.2 and $T = o(S)$ give

$$K_{\text{dep}} = O\left(\frac{H(p_* + 1)\chi}{\ell^2 \log \ell}\right) = O\left(\frac{H}{\chi^4 \log \chi}\right).$$

A path ending at physical depth j has at most j equal-rank edges and at most K_{dep} rank-decreasing edges. No edge moves to a parent or an unrelated source node. Unfolding the dependency graph bounds its evaluations at depth j by

$$(j + 1)(K_{\text{dep}} + 1)2^{j+K_{\text{dep}}} M^j \ell^{C_1 j} H^{C_2(p_*+1)K_{\text{dep}}}$$

for absolute constants C_1, C_2 . The logarithm beyond the factor M^j is at most

$$\begin{aligned} O(J \log \ell + J + K_{\text{dep}} + (p_* + 1)K_{\text{dep}} \log H) &= O\left(\frac{H \log \chi}{\chi} + \frac{H}{\chi^2 \log \chi}\right) \\ &= O(H \log \chi / \chi). \end{aligned}$$

Thus at most $M^j 2^{O(H \log \chi / \chi)}$ evaluations occur at that depth on every execution, including inaccurate seeds.

A physical interval at depth j has width at most $\lceil |x|/M^j \rceil$. At each first initial-entry request, condition on its exposed history and charge Lemma 9.1. The stream of that absent entry is still

private. Summing over the deterministic request-slot bound, and adding the local work, gives the source-width factor

$$\sum_{j=0}^J M^j (1 + \lceil |x|/M^j \rceil) = O((|x| + 1)(J + M)).$$

Here $M^J \leq M|x|$ on the nontrivial source branch. The factors $J + M$ and the number of depths contribute only polynomial-in- H overhead.

The logarithm of all remaining overhead is bounded by

$$O\left(\frac{H \log \chi}{\chi} + \frac{H}{\chi^2 \log \chi} + \frac{H}{\chi} + \chi^2 + H^{1/20} \chi^2\right) = O(H \log \chi / \chi).$$

The middle terms include the concrete seed work. The last term includes action enumeration and finite optimization. Net preprocessing uses $O(NH \log R)$ binary work. The leaf reporting structure and the seed preprocessing use $N \text{ poly}(\log N)$ work. Leaf queries use $O(\ell^3 H \log N)$ binary work and $O(\ell^3)$ metric queries. These costs, and a fixed uniform deterministic polylogarithmic overhead per metric query, fit within the same bound. The resulting unconditional expected work is

$$(N + 1) 2^{O(H \log \chi / \chi)} = N^{1+O(\log \log \log N / \log \log N)}.$$

The conditioning used here is the exposed first-request history. No charge is conditioned on a good complete seed.

There are polynomially many possible persistent keys. Their descriptions have $O(H)$ bits, and balanced dictionaries add only polynomial-in- H lookup cost. Each band list and optimizer workspace has the finite bounds already proved. Together with Lemma 6.5, the stored records and local scratch use polynomial space on every execution.

9.5 The integer-core entry point

Algorithm 10 The bounded-integer entry point

Require: Strings x, y , integer metric oracle v with positive costs at most N^3 , and canonical accuracy encoding σ_{core} for $\varepsilon_{\text{core}} \in (0, 1)$.

Ensure: An integer output in $[0, R]$; every exact branch returns D_v .

```
1: procedure INTEGER-CORE( $x, y, v, \sigma_{\text{core}}$ )
2:    $N \leftarrow |x| + |y|$ ; test equality by zero metric values at corresponding occurrences.
3:   if  $x = y$  then
4:     return 0
5:   end if
6:   if either string is empty or  $|x| \leq 1$  then
7:     return EXACT-DP( $x, y, v$ ).
8:   end if
9:   Query every occurrence's gap cost and form both prefix-mass arrays.
10:   $R \leftarrow M_v(x) + M_v(y)$ .
11:   $H \leftarrow \text{leastpow}_2(\lceil \log_2(N + R + 2) \rceil)$ .
12:  Inspect at most  $H^2 + 12$  bits of  $\sigma_{\text{core}}$ .
13:  if this prefix does not contain the complete encoding then
14:    return EXACT-DP( $x, y, v$ ).
15:  end if
16:  Decode  $\varepsilon_{\text{core}}$  from the inspected prefix.
17:   $\chi \leftarrow \log_2 H$ ;  $\ell \leftarrow \text{leastpow}_2(\max\{1, \chi^3\})$ .
18:  if  $\chi < 2^{1000}$  or  $\ell \varepsilon_{\text{core}} < 10$  then
19:    return EXACT-DP( $x, y, v$ ).
20:  end if
21:  Compute the fixed refinement parameters using this common  $H$ .
22:  Build the deterministic reporting structures and nested mass nets.
23:  Define the labeled physical source tree and state domain implicitly.
24:  Allocate a fresh core-run namespace  $\nu$  and its implicit private-owner rule.
25:  Compute the median schedule and  $p_{\text{fin}}$ .
26:  Initialize an empty cache in namespace  $\nu$ .
27:   $K_{\text{root}} \leftarrow \text{key}_{d_S^{p_{\text{fin}}, 1}}(I_{\text{root}}, (0, |y|))$ .
28:   $Q_{\text{root}} \leftarrow \text{GET}(K_{\text{root}})$ .
29:  return  $\lfloor \min\{R, \max\{0, Q_{\text{root}}\}\} \rfloor$ .
30: end procedure
```

Theorem 9.5. *INTEGER-CORE is one uniform fair-bit algorithm for canonically encoded rational accuracies $\varepsilon_{\text{core}} \in (0, 1)$. For every such fixed accuracy, every pair of strings of total length N , and every integer metric v with positive costs at most N^3 , it returns an integer $Q \in [0, R] \subseteq [0, N^4]$ satisfying*

$$\mathbb{P}\{D_v \leq Q \leq (1 + \varepsilon_{\text{core}})D_v\} > 4/5.$$

It returns zero on equal strings. It terminates on every fair-bit sequence and uses polynomial workspace on every execution. Its large-input expected work, including deterministic preprocessing, all random sampling, and finite binary computation, is

$$(N + 1)2^{O(H \log \chi / \chi)} = N^{1+O(\log \log \log N / \log \log N)},$$

uniformly over the allowed metrics, strings, and decoded accuracies that pass the branch tests. The constant in this large-branch bound is absolute and does not depend on the accuracy encoding. In particular, for every fixed accuracy encoding there are a deterministic function $r_{\varepsilon_{\text{core}}}(N) \rightarrow 0$ and a constant $K_{\varepsilon_{\text{core}}}$, independent of the metric and strings, such that for $N \geq 1$ the expected work is at most $K_{\varepsilon_{\text{core}}}N^{1+r_{\varepsilon_{\text{core}}}(N)}$. The same assertions permit a fixed uniform deterministic polylogarithmic overhead per metric query. Repeated complete runs use independent owner streams.

Proof. On the large-input branch, Lemma 9.4 identifies the implemented output with the complete construction. Lemma 9.3 gives its approximation probability. The unconditional work and workspace bounds have been proved from the concrete initial table and finite optimizer. Since $N \leq R \leq N^4$, we have $H = \Theta(\log N)$ and $\chi = \Theta(\log \log N)$, which gives the displayed exponent. The bounds are uniform over all allowed metrics, including executions with inaccurate initial or intermediate tables.

The remaining branches call exact dynamic programming or return the exact zero answer. The bounded accuracy inspection reads only $O(H^2)$ bits before any accuracy-dependent test. If it succeeds, decoding and testing use polynomial-in- H work. For each fixed encoding, the inspection and numerical cutoff conditions eventually cease as N grows. These inspection and numerical-cutoff branches occur only in a bounded range of lengths for that accuracy and can be absorbed in $K_{\varepsilon_{\text{core}}}$. Its final floor is finite rational division. Fresh namespaces and independent owner streams give the stated independence of repeated runs. $\square$

Remark 9.6. For unit-cost edit distance on explicitly stored strings with $O(\log N)$ -bit integer symbols, INTEGER-CORE gives a $(1 + \varepsilon)$ approximation with probability greater than $4/5$ for every fixed rational $\varepsilon \in (0, 1)$. Its expected running time on the word RAM with $O(\log N)$ -bit words is

$$N^{1+O(\log \log \log N / \log \log N)}.$$

Indeed, the unit-cost metric $v(a, b) = \mathbf{1}_{\{a \neq b\}}$ on the alphabet together with $\perp$ has positive costs equal to one, so Theorem 9.5 applies. Metric queries reduce to symbol equality tests. The core already charges all finite binary computation and sampling, so the same bound holds on the word RAM without normalization or exact real arithmetic.

10 The approximation scheme for arbitrary metric costs

Return to the original metric w and put $D = \text{WED}_w(x, y)$ and $U_0 = M_w(x) + M_w(y)$. The normalized metric from Section 4 has bounded integer costs. We apply the bounded-integer scheme with fresh randomness and transfer its answer to the original metric.

10.1 Uniform algorithm, confidence, and expected work

Theorem 10.1. *There is one uniform fair-bit algorithm in the stated exact-real model that takes a canonically encoded rational $\varepsilon \in (0, 1]$ as input. For each fixed accuracy, its worst-case expected running time is $N^{1+o(1)}$, uniformly over all original metric costs and their ratios. It uses polynomial workspace on every outcome. It returns a value $\hat{D}$ with*

$$\mathbb{P}\{D \leq \hat{D} \leq (1 + \varepsilon)D\} \geq \frac{99}{100} \frac{2944}{3125} > \frac{5}{6}.$$

It returns zero exactly for equal strings, on every outcome. Its value is always in $[0, U_0]$. In addition,

$$\mathbb{P}\{D \leq \hat{D}\} \geq \frac{2944}{3125} > \frac{11}{12},$$

including all bracket outcomes.

Proof. First test equality and handle empty inputs. Their answers are exact. We also handle $|x| \leq 1$ by exact linear-time dynamic programming. Define H_0 to be the least power of two at least $\lceil \log_2(N + 2) \rceil$. Inspect at most $H_0^2 + 8$ bits of the accuracy encoding. If it is not complete within that prefix, compute D by exact dynamic programming without reading the remaining suffix. This fallback is valid without knowing the accuracy value. Otherwise the inspected numerator and denominator have $O(H_0^2)$ bits. Test $\varepsilon N < 361$ by integer cross-multiplication, and use exact dynamic programming if the test holds. These steps precede the construction of η_{norm} or any accuracy-dependent integer search range. For each fixed accuracy encoding, both fallback conditions eventually cease. Rational control calculations and conversion of the inspected integers to exact real registers use only $\text{poly}(\log N)$ work. They can be implemented by bit scans and repeated doubling.

In the remaining branch run Lemma 4.4 once. Construct the normalized oracle from Lemma 4.5, and set $\varepsilon_{\text{core}} = \varepsilon/3$. Canonically encode this rational as σ_{core} . Run the complete core 5 times with fresh independent randomness. Only deterministic preprocessing may be shared across these runs without an additional conditional-independence argument. Let Q be the median of the 5 outputs. Since the present strings are distinct, $\bar{D} \geq 1$. Clamp the median to $[1, \bar{R}]$, and return

$$\hat{D} = \min\{U_0, \eta_{\text{norm}} \min\{\bar{R}, \max\{1, Q\}\}\}.$$

Both clamps preserve a successful median: $1 \leq \bar{D} \leq \bar{R}$, and $D \leq U_0$. They also ensure a strictly positive finite output on distinct strings and an output at most U_0 on every outcome.

Confidence. Condition on the complete bracket history $\mathcal{B}$, including its revealed randomness. For every such history, the normalized metric is fixed, satisfies the core promise, and is independent of the fresh core streams. Each run succeeds with probability at least $4/5$. Its success indicator is independent of the others under this conditioning. The probability that at most two runs succeed is at most

$$\sum_{j=3}^5 \binom{5}{j} (1/5)^j (4/5)^{5-j} = \frac{181}{3125} < \frac{1}{12}.$$

Thus, uniformly in $\mathcal{B}$, the median lies in $[\bar{D}, (1 + \varepsilon/3)\bar{D}]$ with probability at least $p_5 = 2944/3125$. On this median event, Lemma 4.5 gives $\hat{D} \geq D$, even if the bracket's lower endpoint is invalid. On its intersection with $L_{\text{br}} \leq D$, the good-bracket bound in Lemma 4.5 gives

$$\hat{D} \leq (1 + \varepsilon/3)^2 D \leq (1 + \varepsilon)D,$$

because $2\varepsilon/3 + \varepsilon^2/9 \leq 7\varepsilon/9 \leq \varepsilon$ for $\varepsilon \leq 1$. Integrating the uniform conditional median probability over the good bracket histories gives success at least $(99/100)p_5 > 5/6$. This product bound does not require unconditional independence of the bracket event and the median event.

Uniform expected work, including failed brackets. On every bracket history, the normalized metric has positive integer costs at most N^3 . Theorem 9.5 gives the same conditional expected-work bound $K_{\varepsilon/3}N^{1+r_{\varepsilon/3}(N)}$ for every run, without conditioning on either approximation event. A normalized query costs $O(\log N)$ original oracle operations, and the remaining control arithmetic has only polylogarithmic overhead. The tower property consequently bounds the total expected core work, over all bracket outcomes, by

$$5K_{\varepsilon/3}N^{1+r_{\varepsilon/3}(N)} \text{polylog } N = N^{1+o(1)}.$$

The bracket costs deterministically $O(N \log^2 N)$. Gap-prefix arrays, reporting structures, and endpoint nets belong to the one normalized metric. Their construction is included in Theorem 9.5 and can be shared when deterministic. The inspection and numerical-cutoff fallbacks occur only in a bounded range of lengths for each fixed accuracy, so they preserve the pointwise asymptotic bound.

Workspace and finite randomness. The bracket has the deterministic register and bit bounds in Lemma 4.4. The normalized oracle stores finitely many original real values and bounded integer control data. Core arrays contain only bounded integers and the core's finite rationals. Execute the 5 runs sequentially, retaining only their scalar answers. The polynomial core workspace and exact dynamic-programming fallbacks imply polynomial workspace on every outcome. The bracket uses a bounded number of fair bits; the core uses its own charged fair-bit implementation. $\square$

10.2 Proof of the main theorem

Proof. Theorem 9.5 supplies the bounded-integer algorithm and its quantitative expected-work bound. Theorem 10.1 supplies the normalization, approximation guarantee, and output properties. A normalized query has polylogarithmic overhead, and the wrapper makes a constant number of core runs. Thus its expected running time is

$$N^{1+O(\log \log \log N / \log \log N)}.$$

The workspace and termination guarantees follow from the same two theorems. The exact branches cover all remaining input lengths for each fixed accuracy. $\square$

For a fixed rational accuracy $\varepsilon > 1$, use accuracy 1. The resulting factor 2 is at most $1 + \varepsilon$.

10.3 Accuracy chosen from the input length

Proof of Corollary 1.2. Let H_0 be the wrapper parameter, which depends only on N , and put

$$\chi_0 = \log_2 H_0, \quad \ell_0 = \text{leastpow}_2(\max\{1, \chi_0^3\}), \quad \varepsilon_N = \min\{1, 30/\ell_0\}.$$

Run the wrapper with the canonical encoding of ε_N . This choice precedes the bracket and normalization, so it has no dependence on the normalized costs.

For sufficiently large N , we have $\ell_0 \geq 32$, $\varepsilon_N = 30/\ell_0$, and $\varepsilon_{\text{core}} = 10/\ell_0$. The normalized metric satisfies $N \leq \bar{R} \leq N^4$ on every bracket outcome once $\varepsilon_N N \geq 361$. Since $\bar{R} \geq N$, its core parameter satisfies $H \geq H_0$. Hence $\chi \geq \chi_0$ and $\ell \geq \ell_0$, giving $\ell \varepsilon_{\text{core}} \geq 10$.

The accuracy encodings pass both bounded inspections for sufficiently large N . To see this explicitly, write $\ell_0 = 2^k$. The reduced fractions are $\varepsilon_N = 15/2^{k-1}$ and $\varepsilon_{\text{core}} = 5/2^{k-1}$. Their

Algorithm 11 The original-metric wrapper

Require: Strings x, y , original metric oracle w , and canonical accuracy encoding σ .

Ensure: The scalar wrapper output, with the stated exact fallbacks.

```
1: procedure METRIC-SCHEME( $x, y, \sigma$ )
2:   Test  $x = y$  by oracle equality on the original occurrence handles.
3:   if  $x = y$  then
4:     return 0
5:   end if
6:   if either string is empty or  $|x| \leq 1$  then
7:     return EXACT-DP( $x, y, w$ )
8:   end if
9:    $N \leftarrow |x| + |y|$ .
10:   $H_0 \leftarrow \text{leastpow}_2(\lceil \log_2(N + 2) \rceil)$ .
11:  Inspect at most  $H_0^2 + 8$  bits of  $\sigma$ .
12:  if this prefix does not contain the complete encoding then
13:    return EXACT-DP( $x, y, w$ )
14:  end if
15:  Decode  $\varepsilon$  from the inspected encoding; set  $\varepsilon \leftarrow \min\{\varepsilon, 1\}$ .
16:  if  $\varepsilon N < 361$  then
17:    return EXACT-DP( $x, y, w$ )
18:  end if
19:  Query all original occurrence gap costs; set  $U_0 \leftarrow M_w(x) + M_w(y)$ .
20:   $(L_{\text{br}}, U_{\text{br}}) \leftarrow \text{BRACKET}(x, y, w)$ .
21:   $\eta_{\text{norm}} \leftarrow \varepsilon L_{\text{br}} / (3N)$ ;  $\varepsilon_{\text{core}} \leftarrow \varepsilon / 3$ .
22:  Define the oracle  $\bar{w}$  by the bounded integer search in Lemma 4.5.
23:  Compute normalized gap costs and set  $\bar{R} \leftarrow M_{\bar{w}}(x) + M_{\bar{w}}(y)$ .
24:  Canonically encode  $\varepsilon_{\text{core}}$  as  $\sigma_{\text{core}}$ .
25:  for  $\nu = 1, \dots, 5$  do
26:    Allocate a fresh random-owner namespace for run  $\nu$ .
27:     $Q_\nu \leftarrow \text{INTEGER-CORE}(x, y, \bar{w}, \sigma_{\text{core}})$ .
28:    Release this run's private workspace and retain  $Q_\nu$ .
29:  end for
30:   $Q \leftarrow$  the third smallest of  $Q_1, \dots, Q_5$ .
31:  return  $\min\{U_0, \eta_{\text{norm}} \min\{\bar{R}, \max\{1, Q\}\}\}$ .
32: end procedure
```

code lengths are $2k + 6$ and $2k + 4$, respectively. Since $k = O(\log \chi_0)$, these lengths fit within the wrapper’s $H_0^2 + 8$ and the core’s $H^2 + 12$ inspection limits. Also $\ell_0 = O((\log \log N)^3)$, so $\varepsilon_N N \geq 361$ eventually holds. Finally, $\chi \geq \chi_0$ implies that the absolute cutoff $\chi \geq 2^{1000}$ eventually holds on every bracket outcome. Thus the inspection and numerical-cutoff fallbacks occur only in a bounded range of input lengths, with an absolute bound independent of the original metric.

On the large branch, accuracy affects only the normalization, the encoding inspections, and the final Markov test. The cost cap leaves the integer search range at N^3 . The coarse estimator, refinement parameters, pass schedule, and dependency count depend on H , not on ε_N . The absolute large-branch bound in Theorem 9.5 applies uniformly:

$$(N + 1)2^{O(H \log \chi / \chi)} \text{polylog } N = N^{1+O(\log \log \log N / \log \log N)}.$$

The bracket work and accuracy construction fit within this bound. The bounded range of these fallbacks is absorbed into one absolute constant. This argument uses the large-branch estimate directly, rather than the fixed-accuracy constants of that theorem.

The confidence calculation of Theorem 10.1 applies to this accuracy. On its success event the transferred factor is $(1 + \varepsilon_N/3)^2 \leq 1 + \varepsilon_N$. For $N \geq 4$, $\chi_0 \geq \log_2 \log_2 N$, so

$$\varepsilon_N \leq \frac{30}{(\log_2 \log_2 N)^3}.$$

This proves the approximation bound. Inputs with $N < 4$ are handled exactly. $\square$

AI assistance. OpenAI’s Codex assisted with proof development, parameter calculations, literature research, and drafting. Anthropic’s Claude assisted with reviewing proofs, proposing parameter refinements, comparing related work, and refining the exposition.

References

- [1] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966. URL: <https://nymity.ch/sybilhunting/pdf/Levenshtein1966a.pdf>.
- [2] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970. doi:10.1016/0022-2836(70)90057-4.
- [3] Robert A. Wagner and Michael J. Fischer. The string-to-string correction problem. *Journal of the ACM*, 21(1):168–173, 1974. doi:10.1145/321796.321811.
- [4] Peter H. Sellers. On the theory and computation of evolutionary distances. *SIAM Journal on Applied Mathematics*, 26(4):787–793, 1974. doi:10.1137/0126070.
- [5] Xiao Mao and Aviad Rubinstein. Approximation schemes for edit distance and LCS in quasi-strongly subquadratic time. In *Proceedings of the 58th Annual ACM Symposium on Theory of Computing*, pages 744–754, 2026. doi:10.1145/3798129.3800789.
- [6] OpenAI. An Almost-Linear Approximation Scheme for Edit Distance. OpenAI Math Release preprint, September 2026. September 24, 2026 manuscript, repository revision adc7f1241b42. URL: <https://github.com/openai/math/blob/adc7f1241b42e322a6451854ab7e4b4c146bf78a/preprints/An-Almost-Linear-Approximation-Scheme-for-Edit-Distance-September-24-2026/paper.pdf>.

- [7] Ethan Mader, Borna Tavasoli, and Jihan Wang. A strongly subquadratic $(3 + \varepsilon)$ -approximation for weighted edit distance over arbitrary metrics, September 2026. arXiv:2609.14873v1. URL: <https://arxiv.org/abs/2609.14873>.
- [8] Debarati Das, Evangelos Kipouridis, and Tomasz Kociumaka. Metric weighted edit distance: A $(3 + \varepsilon)$ -approximation in $\tilde{O}_\varepsilon(N^{1.6})$ time, September 2026. arXiv:2609.20796v1. URL: <https://arxiv.org/abs/2609.20796>.
- [9] Egor Gorbachev. Factor three approximation for edit distance, October 2026. arXiv preprint 2610.01311v1. URL: <https://arxiv.org/abs/2610.01311v1>.
- [10] William Kuszmaul. Dynamic time warping in strongly subquadratic time: Algorithms for the low-distance regime and approximate evaluation. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics*, pages 80:1–80:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2019.80>, doi:10.4230/LIPIcs.ICALP.2019.80.
- [11] Daniel S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343, 1975. doi:10.1145/360825.360861.
- [12] Eugene W. Myers and Webb Miller. Optimal alignments in linear space. *Computer Applications in the Biosciences*, 4(1):11–17, 1988. doi:10.1093/bioinformatics/4.1.11.
- [13] William J. Masek and Michael S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20(1):18–31, 1980. doi:10.1016/0022-0000(80)90002-1.
- [14] Maxime Crochemore, Gad M. Landau, and Michal Ziv-Ukelson. A subquadratic sequence alignment algorithm for unrestricted scoring matrices. *SIAM Journal on Computing*, 32(6):1654–1673, 2003. doi:10.1137/S0097539702402007.
- [15] Esko Ukkonen. Algorithms for approximate string matching. *Information and Control*, 64(1–3):100–118, 1985. doi:10.1016/S0019-9958(85)80046-2.
- [16] Gad M. Landau and Uzi Vishkin. Fast string matching with k differences. *Journal of Computer and System Sciences*, 37(1):63–78, 1988. doi:10.1016/0022-0000(88)90045-1.
- [17] Gad M. Landau and Uzi Vishkin. Fast parallel and serial approximate string matching. *Journal of Algorithms*, 10(2):157–169, 1989. doi:10.1016/0196-6774(89)90010-2.
- [18] Gad M. Landau, Eugene W. Myers, and Jeanette P. Schmidt. Incremental string comparison. *SIAM Journal on Computing*, 27(2):557–582, 1998. doi:10.1137/S0097539794264810.
- [19] Eugene W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1:251–266, 1986. doi:10.1007/BF01840446.
- [20] Zheng Zhang, Scott Schwartz, Lukas Wagner, and Webb Miller. A greedy algorithm for aligning DNA sequences. *Journal of Computational Biology*, 7(1–2):203–214, 2000. doi:10.1089/10665270050081478.
- [21] Arturs Backurs and Piotr Indyk. Edit distance cannot be computed in strongly subquadratic time (unless SETH is false). *SIAM Journal on Computing*, 47(3):1087–1097, 2018. doi:10.1137/15M1053128.

- [22] Karl Bringmann and Marvin Künnemann. Quadratic conditional lower bounds for string problems and dynamic time warping. In *Proceedings of the 56th Annual IEEE Symposium on Foundations of Computer Science*, pages 79–97, 2015. doi:[10.1109/FOCS.2015.15](https://doi.org/10.1109/FOCS.2015.15).
- [23] Amir Abboud, Thomas Dueholm Hansen, Virginia Vassilevska Williams, and Ryan Williams. Simulating branching programs with edit distance and friends: Or: A polylog shaved is a lower bound made. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, pages 375–388, 2016. doi:[10.1145/2897518.2897653](https://doi.org/10.1145/2897518.2897653).
- [24] Ziv Bar-Yossef, T. S. Jayram, Robert Krauthgamer, and Ravi Kumar. Approximating edit distance efficiently. In *Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science*, pages 550–559, 2004. doi:[10.1109/FOCS.2004.14](https://doi.org/10.1109/FOCS.2004.14).
- [25] Tugkan Batu, Funda Ergün, and Süleyman Cenk Sahinalp. Oblivious string embeddings and edit distance approximations. In *Proceedings of the 17th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 792–801, 2006. doi:[10.1145/1109557.1109644](https://doi.org/10.1145/1109557.1109644).
- [26] Rafail Ostrovsky and Yuval Rabani. Low distortion embeddings for edit distance. *Journal of the ACM*, 54(5):23:1–23:16, 2007. doi:[10.1145/1284320.1284322](https://doi.org/10.1145/1284320.1284322).
- [27] Alexandr Andoni and Krzysztof Onak. Approximating edit distance in near-linear time. *SIAM Journal on Computing*, 41(6):1635–1648, 2012. doi:[10.1137/090767182](https://doi.org/10.1137/090767182).
- [28] Alexandr Andoni, Robert Krauthgamer, and Krzysztof Onak. Polylogarithmic approximation for edit distance and the asymmetric query complexity. In *Proceedings of the 51st Annual IEEE Symposium on Foundations of Computer Science*, pages 377–386, 2010. doi:[10.1109/FOCS.2010.43](https://doi.org/10.1109/FOCS.2010.43).
- [29] Diptarka Chakraborty, Debarati Das, Elazar Goldenberg, Michal Koucký, and Michael Saks. Approximating edit distance within constant factor in truly sub-quadratic time. *Journal of the ACM*, 67(6):36:1–36:22, 2020. doi:[10.1145/3422823](https://doi.org/10.1145/3422823).
- [30] Elazar Goldenberg, Aviad Rubinfeld, and Barna Saha. Does preprocessing help in fast sequence comparisons? In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 657–670, 2020. doi:[10.1145/3357713.3384300](https://doi.org/10.1145/3357713.3384300).
- [31] Emily Fox, Ethan Mader, Kent Quanrud, Bornha Tavasoli, and Jihan Wang. A faster 3-approximation for sublinear edit distances. Manuscript, June 2026. URL: https://ethan-mader.com/assets/files/sublinear_edit_distance.pdf.
- [32] Joshua Brakensiek and Aviad Rubinfeld. Constant-factor approximation of near-linear edit distance in near-linear time. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 685–698, 2020. doi:[10.1145/3357713.3384282](https://doi.org/10.1145/3357713.3384282).
- [33] Michal Koucký and Michael E. Saks. Constant factor approximations to edit distance on far input pairs in nearly linear time. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 699–712, 2020. doi:[10.1145/3357713.3384307](https://doi.org/10.1145/3357713.3384307).
- [34] Alexandr Andoni and Negev Shekel Nosatzki. Edit distance in near-linear time: It’s a constant factor. *SIAM Journal on Computing*, 55(4):708–786, 2026. doi:[10.1137/21M1392322](https://doi.org/10.1137/21M1392322).

- [35] Debarati Das, Jacob Gilbert, MohammadTaghi Hajiaghayi, Tomasz Kociumaka, and Barna Saha. Weighted edit distance computation: Strings, trees, and Dyck. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023)*, pages 377–390. Association for Computing Machinery, 2023. URL: <https://arxiv.org/abs/2302.04229>, doi:10.1145/3564246.3585178.
- [36] Alejandro Cassis, Tomasz Kociumaka, and Philip Wellnitz. Optimal algorithms for bounded weighted edit distance. In *64th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2023)*, pages 2177–2187. IEEE Computer Society, 2023. URL: <https://arxiv.org/abs/2305.06659>, doi:10.1109/FOCS57990.2023.00135.
- [37] Egor Gorbachev and Tomasz Kociumaka. Bounded edit distance: Optimal static and dynamic algorithms for small integer weights. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC 2025)*, pages 2157–2166. Association for Computing Machinery, 2025. URL: <https://arxiv.org/abs/2404.06401>, doi:10.1145/3717823.3718168.
- [38] Panagiotis Charalampopoulos, Tomasz Kociumaka, and Shay Mozes. Dynamic string alignment. In *31st Annual Symposium on Combinatorial Pattern Matching (CPM 2020)*, volume 161 of *Leibniz International Proceedings in Informatics*, pages 9:1–9:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CPM.2020.9>, doi:10.4230/LIPIcs.CPM.2020.9.
- [39] Itai Boneh, Egor Gorbachev, and Tomasz Kociumaka. Bounded weighted edit distance: Dynamic algorithms and matching lower bounds. In *33rd Annual European Symposium on Algorithms (ESA 2025)*, volume 351 of *Leibniz International Proceedings in Informatics*, pages 45:1–45:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2025.45>, doi:10.4230/LIPIcs.ESA.2025.45.
- [40] Panagiotis Charalampopoulos, Tomasz Kociumaka, and Philip Wellnitz. Pattern matching under weighted edit distance. In *66th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2025)*, pages 1927–1935. IEEE Computer Society, 2025. URL: <https://arxiv.org/abs/2510.17752>, doi:10.1109/FOCS63196.2025.00102.
- [41] Omer Gold and Micha Sharir. Dynamic time warping and geometric edit distance: Breaking the quadratic barrier. *ACM Transactions on Algorithms*, 14(4):50:1–50:17, 2018. doi:10.1145/3230734.
- [42] Tugkan Batu, Funda Ergün, Joe Kilian, Avner Magen, Sofya Raskhodnikova, Ronitt Rubinfeld, and Rahul Sami. A sublinear algorithm for weakly approximating edit distance. In *Proceedings of the 35th Annual ACM Symposium on Theory of Computing*, pages 316–324, 2003. doi:10.1145/780542.780590.
- [43] Elazar Goldenberg, Robert Krauthgamer, and Barna Saha. Sublinear algorithms for gap edit distance. In *Proceedings of the 60th Annual IEEE Symposium on Foundations of Computer Science*, pages 1101–1120, 2019. doi:10.1109/FOCS.2019.00070.
- [44] Tomasz Kociumaka and Barna Saha. Sublinear-time algorithms for computing & embedding gap edit distance. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*, pages 1168–1179, 2020. doi:10.1109/FOCS46700.2020.00112.

- [45] Joshua Brakensiek, Moses Charikar, and Aviad Rubinfeld. A simple sublinear algorithm for gap edit distance, 2020. URL: <https://arxiv.org/abs/2007.14368>, [arXiv:2007.14368](#), [doi:10.48550/arXiv.2007.14368](#).
- [46] Karl Bringmann, Alejandro Cassis, Nick Fischer, and Vasileios Nakos. Almost-optimal sublinear-time edit distance in the low distance regime. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 1102–1115, 2022. [doi:10.1145/3519935.3519990](#).
- [47] Karl Bringmann, Alejandro Cassis, Nick Fischer, and Tomasz Kociumaka. Faster sublinear-time edit distance. In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 3274–3301, 2024. [doi:10.1137/1.9781611977912.117](#).
- [48] Elazar Goldenberg, Tomasz Kociumaka, Robert Krauthgamer, and Barna Saha. Gap edit distance via non-adaptive queries: Simple and optimal. In *Proceedings of the 63rd Annual IEEE Symposium on Foundations of Computer Science*, pages 674–685, 2022. [doi:10.1109/FOCS54457.2022.00070](#).
- [49] Djamal Belazzougui and Qin Zhang. Edit distance: Sketching, streaming, and document exchange. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science*, pages 51–60, 2016. [doi:10.1109/FOCS.2016.15](#).
- [50] Tomasz Kociumaka, Ely Porat, and Tatiana Starikovskaya. Small-space and streaming pattern matching with k edits. In *Proceedings of the 62nd Annual IEEE Symposium on Foundations of Computer Science*, pages 885–896, 2021. [doi:10.1109/FOCS52979.2021.00090](#).
- [51] Sudatta Bhattacharya and Michal Koucký. Locally consistent decomposition of strings with applications to edit distance sketching. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 219–232, 2023. [doi:10.1145/3564246.3585239](#).
- [52] Michal Koucký and Michael E. Saks. Almost linear size edit distance sketch. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, pages 956–967, 2024. [doi:10.1145/3618260.3649783](#).
- [53] Diptarka Chakraborty, Elazar Goldenberg, and Michal Koucký. Streaming algorithms for embedding and computing edit distance in the low distance regime. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, pages 712–725, 2016. [doi:10.1145/2897518.2897577](#).
- [54] Sudatta Bhattacharya, Sanjana Dey, Elazar Goldenberg, and Michal Koucký. Many flavors of edit distance. In *Proceedings of the 44th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science*, volume 323 of *Leibniz International Proceedings in Informatics*, pages 11:1–11:16, 2024. [doi:10.4230/LIPIcs.FSTTCS.2024.11](#).